Breaking ECDSA on a fault-tolerant quantum computer

The ECDSA.fail optimisations, the Pareto frontier, and the cost of quantum error correction

A companion to the `shor/` project

Sources: arXiv, IACR ePrint, GitHub (the ECDSA.fail challenge and its supplemental materials), and the published material of Google Quantum AI, IBM Quantum, IonQ, Quantinuum, PsiQuantum, QuEra and Alice & Bob.

12 October 2026

Abstract

Shor's algorithm breaks the elliptic-curve digital signature algorithm by computing a discrete logarithm on secp256k1, and the cost of doing so is dominated by one reversible circuit: the in-place addition of a classically known point to a quantum accumulator. The ECDSA.fail open-autoresearch challenge (May–October 2026) cut the width × work product of that circuit from 2,715 × 3,960,753 to 1,151 × 1,299,453 by the paper's data cutoff and to 1,244 × 797,905 on the 7 October leaderboard, while a separate low-width track reached 792 logical qubits. Part I of this document explains every optimisation family that produced that movement—the six inversion architectures (Kaliski, dialog GCD, Jump-2 with the base-5 codec, ping-pong, Skywalk, Leapfrog), the arithmetic levers (temporary-AND adders, chunked adders with measured carry erasure, truncated comparators, pseudo-Mersenne folds, Karatsuba squaring), the qubit levers (peak owners, budgets, venting, wire loans, register-shared Euclid, ghosting), measurement-based uncomputation, the post-passes, and the nonce-hunting economics that make approximate circuits pass a finite validator—with the circuit diagrams of the supplemental notes redrawn, and it gives the Pareto frontier in both the challenge's (Q, T) plane and the leaderboard's time axis, placed next to the external operating points of Google, Schrottenloher, TrailMix, Luo et al. and IonQ, and next to the circuits built in `shor/`. Part II is about what it costs to run such a circuit on noisy hardware: the threshold theorem and the overhead of error correction, the stabiliser formalism with encoding and decoding algorithms spelled out in full for the small codes, the surface and toric codes (lattice, syndrome extraction, matching, union-find and neural decoders, lattice surgery), colour codes, and quantum LDPC codes (hypergraph products, bivariate bicycle codes such as IBM's gross code, belief propagation with ordered-statistics post-processing, IBM's bicycle architecture and IonQ's walking-cat architecture), the bounds of coding theory with the codes placed on the rate–distance plane, fault-tolerant Clifford gates (transversality, Eastin–Knill, lattice surgery, automorphisms, Pauli frames), fault-tolerant T gates (magic-state injection, distillation, cultivation, code switching), and finally the physical-qubit estimates for a Clifford+T computation, worked out for the ECDSA.fail circuits and compared with the published superconducting, trapped-ion, neutral-atom and cat-qubit estimates. Every encoding circuit and the surface-code threshold plot were checked by simulation (Stim and PyMatching); the scripts are in the appendix.

How to read this document

Part I (Section 1–Section 4) is about the *logical* circuit: what ECDSA.fail optimised, how each optimisation works, what it costs in the $Q \times T$ metric, and where the frontier stands. **Part II** (Section 5–Section 10) is about turning a logical circuit into a physical machine: codes, decoders, fault-tolerant gates, and the resulting physical-qubit and runtime estimates.
Notation. $p = 2^{256} - 2^{32} - 977$ is the secp256k1 field prime, r the (prime) group order, $n = 256$. Q is the peak number of live logical qubits of a circuit and T its average number of *executed* Toffoli gates (CCX/CCZ/AND) on the challenge's validation distribution; the challenge score is $S = Q \times T$. A $[\![n, k, d]\!]$ code uses n physical qubits to encode k logical qubits with distance d; p also denotes a physical error rate where no confusion is possible, and p_L a logical error rate. Numbers taken from a source are cited at the point of use; numbers computed for this document are marked as such and come from the scripts in Appendix B.

Contents

I The logical circuit: ECDSA.fail 4

1 The target: ECDSA, secp256k1, and one reversible point addition 4
1.1 What a quantum computer has to compute . . . 4
1.2 The affine group law and the in-place mixed addition . . . 5
1.3 The ECDSA.fail challenge: harness, metric, validity . . . 5

2 The optimisations of ECDSA.fail 6
2.1 Where the cost sits: the six-stage pipeline of `8e9c9a2` . . . 6
2.2 Six ways to divide: the inversion architectures . . . 8
2.2.1 Kaliski's almost-inverse (the baseline, 29 May–2 June) . . . 8
2.2.2 The dialog GCD: record the walk, replay it (2–19 June) . . . 9
2.2.3 Jump-2 and the base-5 codec (TrailMix, 19 June–21 August) . . . 10
2.2.4 Ping-pong: fixed-depth signed binary division (21 August–29 September) . . . 11
2.2.5 Skywalk: Stein's GCD with a trit history (29 September–7 October) . . . 12
2.2.6 Leapfrog: a route-free multi-step 2-adic walk (from 7 October) . . . 12
2.3 The arithmetic layer: where the Toffolis are . . . 13
2.3.1 Temporary-AND adders, Cuccaro bridges, and the carry trade . . . 13
2.3.2 Chunked adders and measured carry erasure . . . 14
2.3.3 Truncated comparators and width schedules . . . 14
2.3.4 Pseudo-Mersenne folds . . . 15
2.3.5 Karatsuba squaring with pseudo-Mersenne modulo (stage 4) . . . 15
2.3.6 Constants, strength reduction, and fusion . . . 15
2.4 The qubit layer: owning the peak . . . 16
2.4.1 Peak owners, co-binders, and budgets . . . 16
2.4.2 Venting, wire loans, parking, recompute-later . . . 16
2.4.3 Register-shared Euclid and the 825-qubit circuit . . . 17
2.4.4 Ghosting: measurement-based release and later recomputation . . . 17
2.5 Measurement-based uncomputation, in general . . . 18
2.6 Post-passes, census strips, and the nonce economy . . . 19
2.7 Making the circuit window-compatible . . . 19

3 The Pareto frontier 20
3.1 Three constructions, and which one is a frontier . . . 20
3.2 Reading the frontier . . . 23

4 The `shor/` implementation against the ECDSA.fail catalogue 24
4.1 Coverage . . . 25
4.2 What the package does not have, and what it would take . . . 26

II Fault tolerance: from logical to physical qubits **28**

5 Why error correction, and what it costs **28**
5.1 Noise, and the discretisation of errors . . . 28
5.2 The stabiliser formalism . . . 28
5.3 Encoding . . . 29
5.4 Decoding, in general . . . 30
5.5 The bounds, and a map of the codes . . . 31

6 The codes: small codes, surface and toric codes **33**
6.1 Small codes . . . 33
6.1.1 Shor's $[\![9,1,3]\!]$ code . . . 33
6.1.2 Steane's $[\![7,1,3]\!]$ code . . . 34
6.1.3 The perfect $[\![5,1,3]\!]$ code . . . 34
6.1.4 The $[\![15,1,3]\!]$ Reed–Muller code and the $[\![4,2,2]\!]$ detection code . . . 35
6.2 The toric and surface codes . . . 35
6.2.1 Kitaev's toric code . . . 35
6.2.2 The planar (rotated) surface code . . . 36
6.2.3 Decoding the surface code . . . 37
6.2.4 The threshold, measured . . . 38

7 The codes: colour codes and quantum LDPC codes **39**
7.1 Colour codes . . . 39
7.1.1 Two-dimensional colour codes . . . 39
7.1.2 Three-dimensional colour codes, gauge colour codes . . . 40
7.2 Quantum LDPC codes . . . 41
7.2.1 Hypergraph product codes . . . 41
7.2.2 Bivariate bicycle codes and IBM's gross code . . . 41
7.2.3 Trapped ions: IonQ's walking-cat architecture . . . 44
7.2.4 Asymptotically good quantum LDPC codes . . . 44
7.3 Other codes in use . . . 44

8 Fault-tolerant Clifford gates **45**
8.1 Transversal gates and their limits . . . 45
8.2 Fault-tolerant syndrome extraction . . . 45
8.3 Lattice surgery: the surface code's logical Cliffords . . . 46

9 Fault-tolerant T gates **48**
9.1 Why T is the expensive gate . . . 48
9.2 Magic-state injection and distillation . . . 48
9.3 Code switching and gauge fixing . . . 49
9.4 What a Toffoli costs, by architecture . . . 50

10 Physical-qubit estimates **50**
10.1 From a logical circuit to a machine . . . 50
10.2 Model S: a surface-code machine at $p = 10^{-3}$ . . . 51
10.3 Per-gate accounting . . . 52
10.4 The published estimates . . . 52
10.5 What the logical optimisations buy, physically . . . 53

11 Conclusion **54**

A Sources and how they were checked **55**

B Scripts and data **56**

Part I
The logical circuit: ECDSA.fail

1 The target: ECDSA, secp256k1, and one reversible point addition

1.1 What a quantum computer has to compute

ECDSA on secp256k1 signs with a secret scalar $k \in [1, r)$ whose public key is $Q = [k]P$, where P is the standard generator of the curve

$$E:\ y^2 = x^3 + 7 \quad \text{over } \mathbb{F}_p, \qquad p = 2^{256} - 2^{32} - 977, \qquad r = \#E(\mathbb{F}_p) \text{ prime.}$$

Verifying a signature needs only Q; forging one needs k; and recovering k from (P, Q) is the elliptic-curve discrete logarithm problem (ECDLP). Classically the best attacks are generic $\Theta(\sqrt{r}) \approx 2^{128}$ group operations. Shor's algorithm [83, 93] solves it in polynomial time by phase estimation on the two-dimensional "probe" function

$$f(u, v) = [u]P + [v]Q = [u + kv]P, \qquad f(u - k,\, v + 1) = f(u, v), \tag{1}$$

whose period lattice is generated by $(-k, 1)$ and $(r, 0)$: prepare two uniform superpositions over $u, v \in [0, 2^m)$ with $2^m \gtrsim r$, compute f coherently into a point register, Fourier-transform both scalar registers, and measure a pair (s, t) that satisfies $t \equiv ks \pmod r$ with constant probability, from which $k = ts^{-1} \bmod r$ and the candidate is verified classically by $[k]P \stackrel{?}{=} Q$ [33].

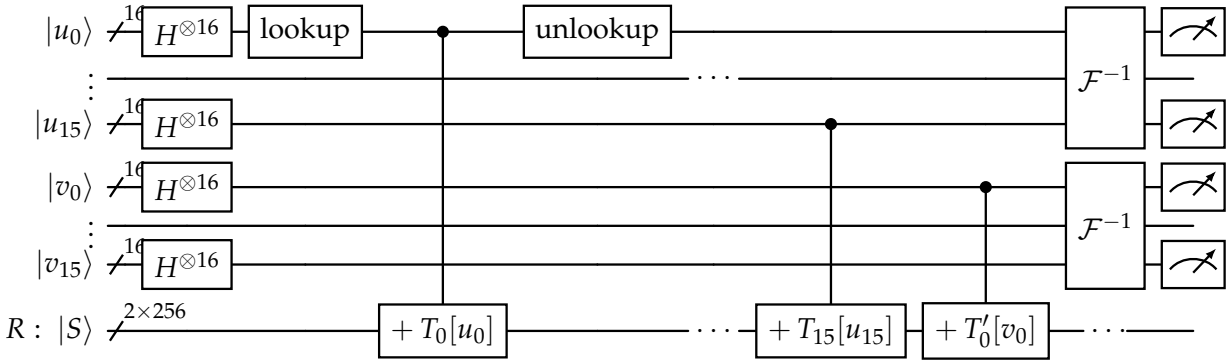


Figure 1: Shor's ECDLP circuit with the scalars split into 16-bit windows ($n = 256$, $w = 16$): each window $z \in \{u_j, v_j\}$ is a coherent address into a classical table $T_j[i] = [i\,2^{16j}]P$ (or $[i\,2^{16j}]Q$); a QROM lookup materialises the selected point, the accumulator R is updated in place by one *mixed* point addition, and the lookup is uncomputed. The scalar registers pass through unchanged and are read out by (semiclassical) inverse Fourier transforms. The accumulator starts at a fixed point S, never at $\mathcal{O}$, so that the affine formulas stay valid [54, 90]. Redrawn after Supplemental Note D of [75].

Windowing and the cost formula. With w-bit windows there are $\lceil n/w \rceil = 16$ windows per scalar, i.e. 32 conceptual point additions; the first window is replaced by a direct table lookup of the 2^{2w} possible starting points (or 2^{18} entries in IonQ's version [54]) and the last three windows are dropped and recovered by a classical search over 2^{48} candidates, which leaves $2n/w - 4 = 28$ executed point additions [5, 54, 75]. Each addition also pays three table lookups of 2^{16} entries (the coordinates x_A, y_A and the derived value $3x_A$), which both IonQ and ECDSA.fail price at $3 \cdot 2^{16}$ Toffolis. This gives the proxy that ECDSA.fail uses to convert its per-addition numbers into whole-attack numbers (its eq. "full-attack"):

$$\text{Toffolis} \approx 28\,(T + 3 \cdot 2^{16}), \qquad \text{logical qubits} \approx Q + 16, \tag{2}$$

and, if one point addition fails on a fraction e of inputs, a union bound $1 - 28e$ on the success probability. The last three windows can be dropped because the classical post-processing verifies each candidate against the curve; Ekerå's analysis [33] shows a few extra qubits per scalar register restore the success probability that the $r \nmid 2^m$ rounding would otherwise lose. The Fourier transforms contribute nothing to the Toffoli count and can be done semiclassically on one recycled qubit per register [50]; every published estimate therefore reports only the point arithmetic.

1.2 The affine group law and the in-place mixed addition

For $R = (x_R, y_R)$ and $A = (x_A, y_A)$ with $x_R \neq x_A$,

$$\lambda = \frac{y_R - y_A}{x_R - x_A}, \qquad x_{R'} = \lambda^2 - x_R - x_A, \qquad y_{R'} = \lambda(x_A - x_{R'}) - y_A. \tag{3}$$

Three subtractions, one squaring, one multiplication and one *division*. The division is the whole problem: the divisor $x_R - x_A$ is a quantum value, so the inverse has to be computed by a reversible circuit on a superposition and then uncomputed. ECDSA.fail's best circuits implement the group law as the seven register updates of Table 1, which overwrite the X and Y registers in place and never allocate separate output coordinates (Supplemental Note D, [75]).

step	update	X register afterwards	Y register afterwards
1	$X \leftarrow x_R - x_A;\ Y \leftarrow y_R - y_A$	$d_x = x_R - x_A$	$d_y = y_R - y_A$
2	$Y \leftarrow Y \cdot X^{-1}$	d_x	λ
3	$X \leftarrow X + 3x_A$	$x_R + 2x_A$	λ
4	$X \leftarrow X - Y^2$	$x_A - x_{R'}$	λ
5	$Y \leftarrow Y \cdot X$	$x_A - x_{R'}$	$\lambda(x_A - x_{R'})$
6	$Y \leftarrow Y - y_A$	$x_A - x_{R'}$	$y_{R'}$
7	$X \leftarrow x_A - X$	$x_{R'}$	$y_{R'}$

Table 1: In-place mixed addition $R' = R + A$ in seven reversible register updates (Supplemental Note D of [75], after [90]). Steps 2 and 5 are the two multiplicative steps; after steps 3–4, $X = (x_R - x_A) + 3x_A - \lambda^2 = x_A - x_{R'}$. Step 5 is *exactly* the inverse of the division unitary of step 2 applied to a different operand: $U_{\text{div}} : |X\rangle|Y\rangle \mapsto |X\rangle|YX^{-1}\rangle$, so $U_{\text{div}}^{-1} : |X\rangle|Y\rangle \mapsto |X\rangle|YX\rangle$.

The generic-case domain

The seven steps are correct only when $x_R \neq x_A$ (step 2 divides by d_x) and $x_A \neq x_{R'}$ (step 5 multiplies by $x_A - x_{R'}$; in the register-shared low-Q circuit this quantity is also inverted). The excluded inputs $R \in \{A, -A, -2A\}$ have density $O(1/p)$ and are not detected; Shor's algorithm tolerates them because it only needs the addition to be right on *most* inputs. The windowed variant additionally has to handle the table row $A = \mathcal{O}$ (Section 2.7).

1.3 The ECDSA.fail challenge: harness, metric, validity

ECDSA.fail [75] is a public benchmark launched by Eigen Labs on 30 May 2026: a Rust harness (adapted from the code Google released with [5], Zenodo record 19597130) in which contestants edit `src/point_add/` to emit an *op stream* for one mixed point addition, and a trusted evaluator (`eval_circuit.rs`, which never imports contestant code) runs that stream on 9,024 test inputs. The interface is: two 256-qubit registers `target_x`, `target_y` holding the accumulator, two 256-*bit* classical registers `offset_x`, `offset_y` holding the addend, and the circuit must overwrite the targets with the affine sum. The gate set is X, Z, CX, CZ, CCX, CCZ, SWAP, reset R, and Hmr ("Hadamard–measure–reset": a demolition measurement in the X basis whose outcome becomes a classical bit), with gates optionally conditioned on classical bits (`PushCondition`/`PopCondition`).

The score. $S = Q \times T$ with

$Q = 1 + \max\{\text{qubit index referenced in the op stream}\}$ (= the high-water mark of live qubits),
$T =$ average over the 9,024 shots of the number of CCX/CCZ gates *executed*.

Executed, not emitted: a Toffoli under a classical condition counts only on the shots where the condition holds, which is what makes measurement-based uncomputation "free" (Section 2.5) and why T is not a static gate count, a T-depth, or a fault-tolerant hardware cost. Clifford gates, measurements and resets cost nothing.

The three channels. Every run reports, for all 9,024 inputs, (i) classical mismatches `cls` (wrong output point, per shot), (ii) phase-garbage batches `pha` (a non-zero accumulated global phase across the 64-shot batch—a leftover $(-1)^{m \cdot f}$ from a measurement whose fix-up did not match, or a reset of a qubit that was not $|0\rangle$, which the simulator models by XOR-ing a random bit into the phase), and (iii) ancilla-garbage batches `anc` (a scratch qubit left non-zero at the end). A valid run is `0/0/0`; a valid run that beats the incumbent is *promoted* and becomes a commit on `main`.

Fiat–Shamir islands, the tail nonce and λ. The test inputs are not fixed: a SHAKE256 stream seeded with the domain tag, the op count and every field of every op draws the scalar pairs (k_1, k_2), the points $[k_1]G, [k_2]G$, and also the random outcomes of every R and Hmr. A circuit therefore cannot be tuned to a known test set—but it can be tuned to *some* test set. Every op stream ends with 48 pairs of self-cancelling $X; X$ gates whose targets encode a 48-bit *tail nonce*: it costs nothing and changes the hash, hence the "island" (inputs plus measurement outcomes) on which the circuit is judged. A deliberately approximate circuit is wrong on a small set of *hard inputs*; if λ is the expected number of failing shots on a random island, a random nonce yields a clean island with probability roughly $e^{-\lambda}$, and contestants *hunt* nonces (classical pre-filters on GPUs replay the circuit's own failure predicate at tens of thousands of nonces per second, then a full simulation confirms the survivors). The practitioners price every truncation in "Toffoli per unit of λ" and work against explicit λ caps (18 and 19.5 in the September–October packages) [57]. This is the central integrity caveat of the whole leaderboard, stated by the paper itself: *validator-clean is not value-exact*; the paper re-evaluates 42 accepted submissions on a common 50,000-input corpus and reports a retry-adjusted score $Q \times T / \hat{p}$, with $\hat{p} \approx 0.998$ for the best circuit [75].

Where the material in Part I comes from

The main paper [75] (arXiv:2609.09582, v2 of 19 Sept. 2026, data cutoff 26 July 2026 09:21:55 UTC); its Supplemental Notes A–F, published as TeX and PDF in the GitHub repository `jieyilong/ecdsafail-supplemental-materials` [76] (A: trajectory, B: anatomy of the optimisations, C: census of the 404 scored commits, D: the best $Q \times T$ circuit `8e9c9a2`, E: the low-Q circuit `b6f2b0a`, F: the window-compatible variant); the challenge repository `Layr-Labs/ecdsafail-challenge` [32] (cloned at commit `c672dda`, 11 Oct. 2026); and the per-commit wiki `benhuang2025/ecdsafail-wiki` [57], which documents all 584 accepted commits through 7 Oct. 2026 with their diffs, 18 concept pages, and leaderboard snapshots. Where the wiki's audit disagrees with the paper's census it says so, and so do we.

2 The optimisations of ECDSA.fail

2.1 Where the cost sits: the six-stage pipeline of `8e9c9a2`

The best product-score circuit at the data cutoff, submission `8e9c9a2` (source commit `60d61859fa69`, Q = 1,151, T = 1,299,453, $Q \times T$ = 1,495,670,403), executes the seven steps of Table 1 as six source-level stages (Figure 2). The two multiplicative steps, stages 2 and 5, share one engine—a recorded Euclidean walk whose transcript is replayed on the coefficient

pair—and between them account for the great majority of the Toffolis and for the qubit peak; everything else is a modular add/sub of a classical constant or a specialised square. Reading the pipeline left to right is the quickest way to see which optimisation family acts where.

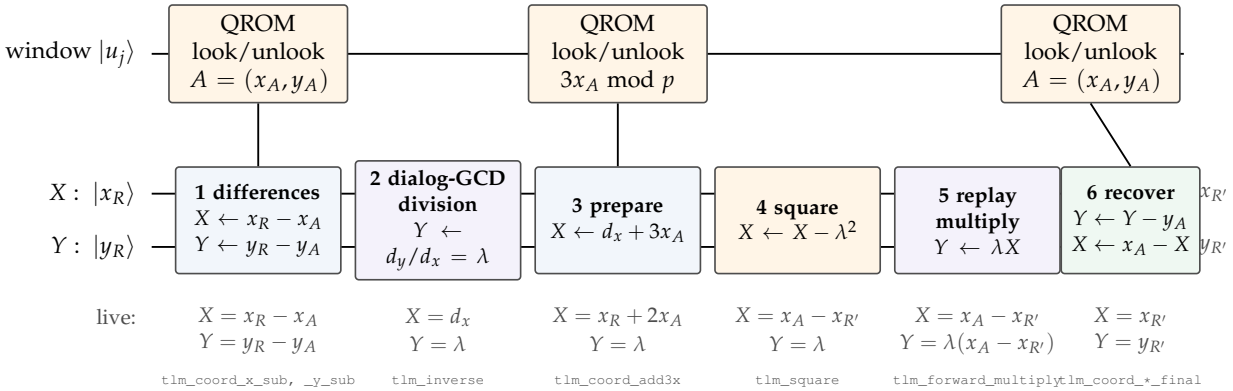


Figure 2: The six source-level stages of `8e9c9a2` and the live contents of the two coordinate registers after each (Supplemental Note D). In the challenge, x_A, y_A are classical bits; the QROM boxes are what a windowed Shor circuit must add (Section 2.7). Stages 2 and 5 are the same Jump-2 dialog-GCD engine run in the "inverse" and "forward" directions.

Nine families. Supplemental Note B classifies the 400 scored source commits of the public competition (to 26 July) into nine optimisation families, six of which have classical-compiler names and three of which are specifically reversible: *windowing*, *algorithmic substitution*, *dead-code and redundancy elimination*, *register and scratch allocation*, *codec/encoding*, *fusion*, *strength reduction*, *constant propagation / classical specialisation*, and *uncompute strategy* [75]. Table 2 gives the counts and the share of the realised score reduction each family receives in the paper's bookkeeping. Because the taxonomy is organised by *what a commit did*, while a reader wants to know *how the circuit works*, the rest of this section is organised by circuit component instead—inversion, arithmetic, qubits, uncomputation, post-passes, windowing—and names the family of each lever as it comes up.

family	commits	%	Δ share	typical levers
dead-code / redundancy elimination	244	61.0	39%	width truncation (149), iteration/threshold tuning (37), fold/carry truncation (37), dead-gate strips (21)
register and scratch allocation	50	12.5	21%	ancilla-lifetime / venting (30), scratch reuse (17), in-place mutation (3)
uncompute strategy	22	5.5	8%	measurement-based uncompute (all 22)
constant propagation / classical specialisation	19	4.8	≈ 0	classical $3x_A$, $(2-q)x_A$, forced-branch prefixes, SAT-proven rows
strength reduction	14	3.5	–	fast-adder reroute (9), Solinas/NAF folds (5)
fusion	12	3.0	18%	gate/op fusion (7), boundary cswap-merge (5)
windowing	8	2.0	13%	the Jump-2 inversion base, windowed Solinas shift
codec / encoding	7	1.8	≈ 0	base-5 / K5 transcript codecs (low-Q branch)
algorithmic substitution	2	0.5	small	Karatsuba squarer swaps
verifier search (nonce rerolls) and other	22	5.5	–	identity-tail nonce changes, padding

Table 2: The census of the 400 scored competition commits to 26 July 2026 (Supplemental Notes B and C of [75]). "Δ share" is the fraction of the gross frontier reduction attributed to the 69 largest improvements that a family receives; the four structural families (allocation, fusion, windowing, uncompute) take $\approx 60\%$ of it across only 26 commits, whereas dead-code elimination is frequent but incremental. The wiki's diff-level re-audit of the 180 later commits (to 7 Oct.) adds a change-type "inversion/division algorithm swap" for the three architecture changes of Section 2.2 [57].

2.2 Six ways to divide: the inversion architectures

Every frontier circuit computes $\lambda = d_y/d_x$ and later multiplies by $x_A - x_{R'}$ with a *binary extended-Euclidean walk* of one kind or another; the choice of walk is the single biggest determinant of both Q and T, and the competition replaced it five times (Table 3). The common skeleton is: (i) a forward walk on the pair $(u, v) = (p, d)$ that drives it to $(1, 0)$ or to a fixed point, recording its branch decisions coherently (they depend on the superposed d and cannot be measured); (ii) a *replay* of the same decisions as linear updates on a coefficient pair seeded with the numerator, which turns y into $y \cdot d^{-1}$ (division) or $y \cdot d$ (multiplication, replay in the opposite direction); (iii) a backward walk that consumes the record and restores d. Only the walk rule, what is recorded, and how the record is stored differ.

	Kaliski	dialog GCD	TrailMix Jump-2	ping-pong	Skywalk / Leapfrog
dates (frontier)	29 May–2 Jun	2–19 Jun	19 Jun–21 Aug	21 Aug–29 Sep	29 Sep–7 Oct / 7 Oct–
rule	qrisp/HJNRS almost-inverse	Stein's binary GCD	Stein, two halvings per step	fixed alternation, no compare	Stein, even rail halves / 3 add-halves + barrel
coefficients	(r, s) carried live	replayed from record	replayed, fused into walks	replayed, fused	replayed, fused
record per step	3 bits (f, a, b)	2 bits $(b_0, b_0 b_1)$	5 symbols in 7 qubits/3 steps	1 sign bit	trit (1.6 q) / 5 qubits
steps per walk	$\approx$ 397–404	399 $\to$ 259 (K=2)	258–261	≈ 700	≈ 393 / 139
(Q, T) at handover	2006×2.55M	1168×1.43M	1150×1.285M	1250×0.889M	1174×0.945M / 1244×0.798M

Table 3: The inversion architectures that held the ECDSA.fail frontier, in order; dates and operating points from the wiki's concept pages and the 7 Oct. leaderboard snapshot [57].

2.2.1 Kaliski's almost-inverse (the baseline, 29 May–2 June)

The organisers' reference circuit (2,715 qubits, 3,942,753 Toffolis unpadded) computed λ with two runs of the reversible Kaliski/Montgomery almost-inverse in the form of Häner et al. [53]: state

(u, v_w, r, s) initialised to $(p, d_x, 0, 1)$, a termination flag f, one history qubit per iteration, and the straight-line iteration (qrisp step numbers)

STEP 0	$is_zero = [v_w = 0]$ (an OR chain); $m_i \oplus= f \cdot is_zero$; $f \oplus= m_i$
STEP 1–2	branch flags a_f, b_f from $u_0, v_{w,0}$ and $\ell = [u > v_w]$
STEP 3	cswap(a_f): (u, v_w) and (r, s)
STEP 4	$add_f = f \wedge \neg b_f$; $v_w \mathrel{-}= add_f \cdot u$; $s \mathrel{+}= add_f \cdot r$
STEP 6–8	$v_w \leftarrow v_w/2$ (free relabel); $r \leftarrow 2r \bmod p$
STEP 9–10	cswap(a_f) back; $a_f \oplus= \neg s_0$

run a fixed number of times, then backwards, with 2^{iters} removed by modular halvings. The Kaliski era already contains four of the recurring moves in embryo [57]: **boundary cswap merges** (STEP 9 of iteration k and STEP 3 of $k+1$ swap the same registers and cswap(a) cswap(b) = cswap$(a \oplus b)$, so one width-n Fredkin row is deleted per iteration at the cost of a one-qubit "frame" carrying a_k across the boundary: $-274{,}327$ and $-260{,}517$ Toffolis for the two channels), **in-place** v **with borrowed carries** (use the live x register as v_w, and take the adders' carry ancillas from history slots that are provably still $|0\rangle$: 2,708 $\rightarrow$ 2,310 qubits), **width truncation** (an envelope fitted to 80,000 sample inputs clips every operation to the bits that can be non-zero at iteration i, worth $\approx$ 4,300 Toffolis per margin bit), and **carry-tail truncation** (above the top set bit of $c = 2^{32} + 977$ a carry can only propagate, so the chain is cut at $33 + W$). By 2 June the frontier stood at 2,006 $\times$ 2,552,687, and then the whole inverter was replaced.

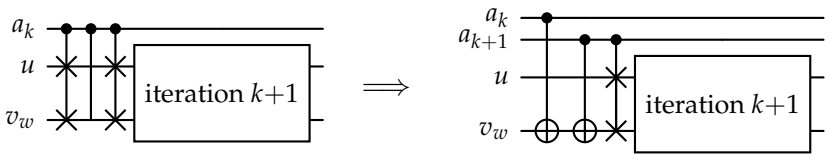


Figure 3: Boundary cswap-merge (fusion family). Left: the step-9 un-swap of iteration k and the step-3 swap of iteration $k+1$ act on the same pair. Right: the identity cswap(a_k)cswap(a_{k+1}) = cswap$(a_k \oplus a_{k+1})$ replaces two width-n Fredkin rows ($2n$ Toffolis) by one row driven by a parity formed with a free CNOT into a one-qubit frame (shown schematically; the frame qubit is cleared at the boundary that set it, so it is never live at the step-4 peak). Commits `d35d6e7/9bcc27d` and `36a2989/112d5a4` [57, 75].

2.2.2 The dialog GCD: record the walk, replay it (2–19 June)

Schrottenloher's construction [90] (following the "record and replay" idea of [61]) drops the Bézout coefficients from the walk altogether. Run Stein's binary GCD on $(u, v) = (p, a)$ alone,

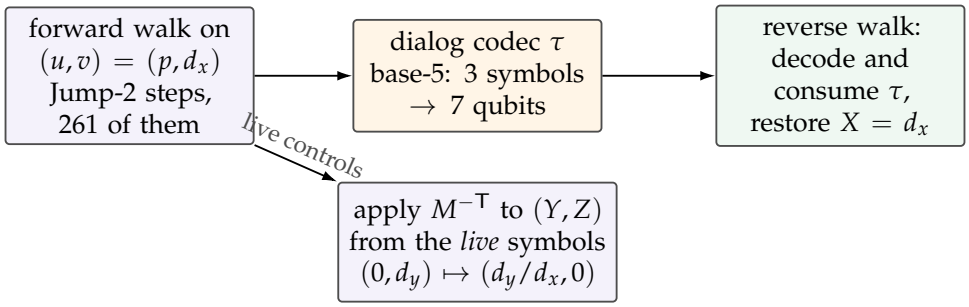
$$b_0 \leftarrow v_0; \qquad b_{01} \leftarrow b_0 \wedge [u > v]; \qquad \text{if } b_{01} : \text{swap}(u, v); \qquad \text{if } b_0 : v \leftarrow v - u; \qquad v \leftarrow v/2, \quad (4)$$

and keep only the two decision bits per step—the *dialog*. Each step is a linear map M_i on (u, v); the product $M = M_L \cdots M_1$ sends $(p, a) \mapsto (1, 0)$. Over $\mathbb{F}_p$ the constant p is 0, so replaying the same decisions on the pair $(0, y)$ gives $(y/a, 0)$: *the record of the walk on a is a program for dividing by a*, and the identical program read in the other direction (doublings instead of halvings) multiplies by a. The replay of one step is

$$\text{cswap}(b_{01}, x, y); \qquad y \leftarrow y - b_0\, x \bmod p; \qquad y \leftarrow y/2 \bmod p,$$

so the only quantum data it touches are the two coefficient registers and the two recorded bits; it never looks at a again. Since $(b_0, b_{01}) = (0, 1)$ cannot occur, three steps carry $3^3 = 27 < 32$ states and fit in five qubits instead of six, giving an asymptotic $2.355n$ qubits for the record in place of $2n$ for (r, s)—and, unlike the Kaliski design, the record is only consumed at the very end, so its lifetime is the whole circuit, which is what the later codecs and the register-sharing of

Section 2.4.3 address. The first dialog submission, `cddd5df` (2 June, 1,698 × 2,447,846), replaced both inversions at once; by 18 June (`ba3efc1`) the route stood at 1,168 × 1,431,621, almost entirely by tuning its truncations: step count (399 → 393, then 259 under K=2), the width envelope active_width$(s) = \lceil 256 - s \cdot \text{slope} + \text{margin} \rceil_{\text{even}}$ with slope 0.7075 → 1.017 and margin 37 → 10, per-step compare widths from a baked table of observed maxima of $w - \text{msb}(u \oplus v)$, body trims, apply-pass chunking, and fold windows [57].



Stage 2 (division). The same engine with the replay fused into the *reverse* walk and M^{T} instead of $M^{-\mathsf{T}}$ is stage 5 (multiplication): $(0, \lambda) \mapsto (\lambda(x_A - x_{R'}), 0)$. The transcript is a quantum register entangled with d_x; it is grown as the active widths of u, v shrink and is returned to $|0\rangle$ by the reverse walk.

Figure 4: Dialog-GCD division as implemented in `8e9c9a2` (Supplemental Note D, pp. 7 and 14 redrawn). Every halving, swap and subtraction of the walk is a linear update M of (u, v); the same controls apply the coefficient update $M^{-\mathsf{T}}$ to the pair (Y, Z), so while the walk reduces $(p, d_x) \to (1, 0)$ the numerator d_y becomes $d_y d_x^{-1} = \lambda$.

2.2.3 Jump-2 and the base-5 codec (TrailMix, 19 June–21 August)

Trail of Bits' "TrailMix" port [75] (submission `bdb1d22`) reorders the step so that the halving comes first and allows a *second* halving when the first leaves v even:

$$\begin{aligned} &v \leftarrow v/2; \qquad s_2 = [v \text{ even}]; \qquad \text{if } s_2 : \; v \leftarrow v/2; \qquad b = v_0; \\ &s = b \wedge [v < u]_{\text{top } k \text{ bits}}; \qquad \text{if } s : \; \text{swap}(u, v); \qquad \text{if } b : \; v \leftarrow v - u. \end{aligned} \tag{5}$$

Every macro-step ends with v even (the subtraction of odd u from odd v is even; if $b = 0$, v was even already), so the next unconditional halving is always valid: Jump-2 is an *exact retiming* of the binary GCD that makes about 0.95 bits of progress per recorded symbol instead of 0.71, which is why the fixed budget fell from 393 to 258–261 macro-steps (non-convergence 2.839×10^{-4} at 258, 2.86×10^{-5} at 261 over 10^7 inputs). The symbol $\sigma = (b, s, s_2)$ has three bits but only five reachable values—$s_2 = 0 \Rightarrow b = 1$ and $b = 0 \Rightarrow s = 0$—so it is a base-5 digit $d = \phi(\sigma) \in \{0, \ldots, 4\}$, and three digits pack as $z = d_0 + 5d_1 + 25d_2 \leq 124 < 2^7$: seven qubits for nine raw bits, by a reversible permutation of basis states (no decision is measured or discarded). For 261 symbols the tape is $2 + 86 \cdot 7 + 5 = 609$ qubits instead of 783 (Figure 5). The windowing family's largest entry is exactly this re-blocking of the Euclidean loop (its landing is the competition's second-largest single frontier move), and the codec family is its storage [75].

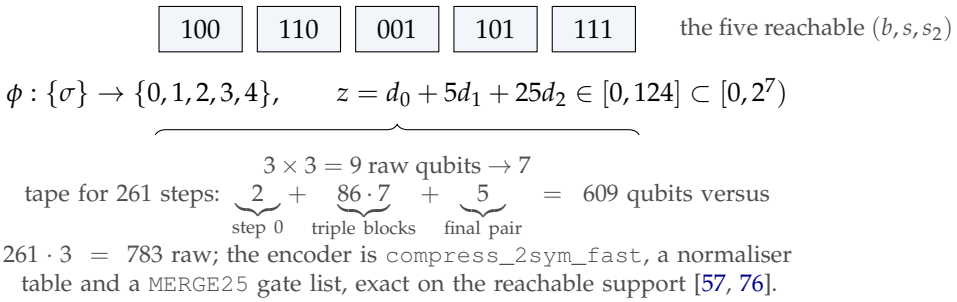


Figure 5: The base-5 transcript codec of the Jump-2 route (codec/encoding family; Supplemental Note D, p. 9). In the earlier June route the same idea appeared as the K2-pair ($6 \to 5$), K5 ($15 \to 12$), head-11 and tail-3 block codecs [57].

Width scheduling. The walk operates on shrinking registers: at step i only the active $w_i =$ SCHED_J2$[i]$ low bits of u, v are touched, the comparator looks only at GAP_J2$[i]$ top bits, and the freed high lanes host the growing tape. Note D's live-width model for stage 2,

$$Q_i \approx \underbrace{512}_{\text{Bézout pair}} + \underbrace{|\tau_i|}_{\text{compressed dialog}} + \underbrace{2w_i}_{u_i, v_i} + \underbrace{q_{\text{work},i}}_{\text{compare, carry, control}}, \tag{6}$$

states the scheduling objective: make the decrease of $2w_i$ absorb the growth of $|\tau_i|$, while hosting and measurement-based cleanup keep $q_{\text{work},i}$ below the same peak. The known bits $u_0 = 1$ and $v_0 = 0$ are "parked" while dormant and their lanes lent out; chunked adders borrow already-live registers as dirty vent space; every borrowed value is restored before its owner is active again.

2.2.4 Ping-pong: fixed-depth signed binary division (21 August–29 September)

Submission 897dda2 (teddyjfpender) replaced the GCD by a divider that needs no comparison at all. Two *odd signed* registers in a 259-bit two's-complement envelope take turns being halved in a fixed order:

$$\text{sign} = \text{target}_1 \oplus \text{source}_1, \qquad \text{target} \leftarrow \frac{\text{target} + (-1)^{\text{sign}}\,\text{source}}{2}, \tag{7}$$

where the sign is chosen so that the sum is $\equiv 2 \pmod 4$ and the result is odd again; roles alternate with the round parity, which is known at build time, so no data-dependent swap is needed. The pair $(u, v) = (p, d)$ converges to a fixed point $(\pm 1, \pm 1)$; one tape qubit per round records the sign; the replay runs the same signed halvings modulo p on $(\text{coefficient}, \text{numerator}) = (0, c)$ and ends at $(\pm c/a, \pm c/a)$, after which the terminal signs make the two registers equal and a row of CNOTs clears one of them. Each replay round is one fused cell: a chunked add with measured boundary carries, a fold by one of $\{-f, 0, +f, +2f\}$ with $f = 2^{32} + 977$ restricted to a window, and a measured overflow flag repaired by a truncated compare. The walk is cheaper per round (257 Toffolis in Julian's reconstruction, Section 4) but takes ≈ 700 rounds, and its 700-bit tape sets the peak: the memory note's plateau is

$$700\ (\text{tape}) + 256 + 256\ (\text{coefficient pair}) + 2\ (\text{terminal signs}) + 86\ (\text{chunk carry ladder}) = 1{,}300,$$

which the follow-ups brought down to 1,250 by lending the 16 known terminal wires (the sign copies in bits 1..7 of each 9-wire register and the constant bit 0), interleaving the replay with the walk so that early replay rounds share the machine with only part of the tape, and sizing every carry ladder against a peak cap (PP_WALK_MAX_QUBITS) [57]. Against the TrailMix frontier the swap cost 171 qubits and saved 332k Toffolis (1,321 × 952,707 against 1,150 × 1,284,776), the largest single score delta of the extension census.

2.2.5 Skywalk: Stein's GCD with a trit history (29 September–7 October)

`8ee337c` (mochimodev) returned to Stein's rule but removed its comparison by a change of representation: keep the pair as the unordered magnitudes $\{u+v, u\}$, exactly one of which is even; call it E and the other O; the next state is $\{E/2, |O - E/2|\}$. One *rail tick* is

1. $c = R_1[0]$; Fredkin-swap $R_1[i] \leftrightarrow R_2[i]$ on c for $i \geq 1$ and $R_2[0] \oplus\!= c$, so that the even rail is in R_1;
2. $E/2 = R_1[1..]$ is a relabelling (no gates);
3. $R_2 \leftarrow R_2 \mp E/2$, subtracting when the signs agree (signs are kept, not absolute values);
4. the wire c leaves the rail as the tape's orientation bit $\mathrm{typ}_t = \mathrm{typ}_{t-1} \oplus c_t$, and the add's sign-flip wire becomes s_t.

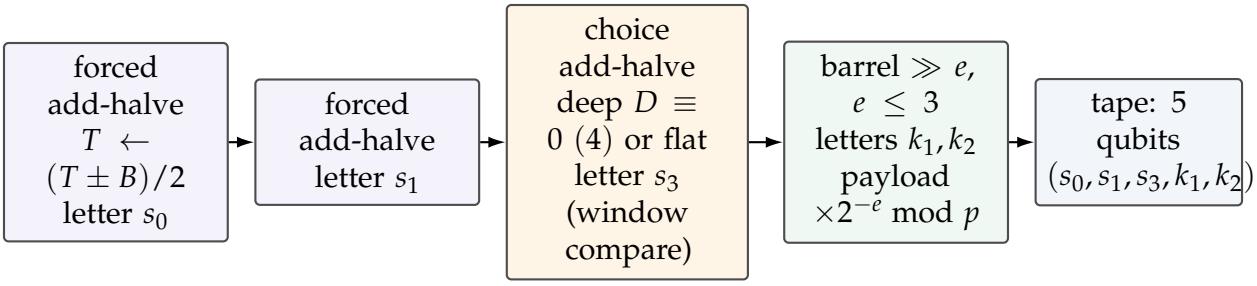
The pair (typ, s) takes three legal values, a *trit*, and five trits pack into eight qubits ($3^5 = 243 \leq 256$), i.e. 1.6 qubits per tick; about 393 ticks suffice where ping-pong needed ≈ 700, so the history is shorter and Q fell 1,250 $\rightarrow$ 1,174, while T rose 6.4% because every tick pays a full-width Fredkin row on the rails and every payload cell a conditional 256-bit route swap [57]. The payload is carried as a sum/difference ("butterfly") pair (Σ, Δ) with the division cell $\Sigma \leftarrow (\Sigma - (-1)^g \Delta)/2 \bmod p$, $g = \mathrm{typ}_t$, followed by a swap if s; seeded with $(2N, 0)$ it parks at $\Sigma = \Delta = N/d$, so the endpoint $\Delta \oplus\!= \Sigma$ costs no Toffoli.

2.2.6 Leapfrog: a route-free multi-step 2-adic walk (from 7 October)

`86221ad` (BitWonka) makes much bigger steps. Two odd rails R_0, R_1 in two's complement; at tick t the target $T = R_{t \bmod 2}$ is updated from the source $B = R_{1-t \bmod 2}$ by

1. two *forced* add-halves, letters s_0, s_1: $T \leftarrow (T + (-1)^s B)/2$ with $s = T_1 \oplus B_1$ (so the sum is $\equiv 2 \bmod 4$ and T stays odd; the sign costs two CNOTs);
2. one *choice* add-halve, letter s_3: of $T \pm B$, exactly one is $\equiv 0 \bmod 4$ (the *deep* candidate D); take it when $D \bmod 32 \neq 0$ (so its extra zeros $e = v_2(D) - 1 \leq 3$) and the step is not the rail-growing "$e = 1$ with equal signs" case (rule *sign2*); with `LF_W1` the excluded case is reopened when $|T| > 4|B|$ on a K-bit window of the top bits;
3. a *barrel*, letters $k_1 = e \,\&\, 1$, $k_2 = e \gg 1$: an arithmetic right shift of the rail by e (the e known-zero low wires rotate to the top and are sign-filled, $1.5w - 2$ Toffolis) and, on the payload, a controlled multiplication by $2^{-e} \bmod p$: with $m = P \bmod 2^e$ and $f - 1 \equiv 0 \bmod 16$, $P \cdot 2^{-e} \equiv \mathrm{rotr}_e\big(P - m(f-1)\big)$—one fold ladder whose addend bits are AND-monomials of the three bits of m, then a rotation `rot4` built from three controlled reflections ($1.5n - 2$ Fredkins).

Each tick removes $3 + e$ low bits of the target, so 139 ticks suffice (Skywalk ≈ 393, ping-pong ≈ 700); the letter is five raw qubits with no codec (695 tape qubits), the payload work per tick is three replay cells and one barrel (≈ 556 cell-equivalents per pass against Skywalk's 393 cells $+393$ route swaps), and because the target is fixed by the tick's parity there is no routing at all. The width envelope (`env_w1_peel139.txt`) is a greedy *peeling* fit over 1.92 M walks that drops the walks that never park and then repeatedly lowers whichever position saves the most priced Toffoli per walk sacrificed; the author reports $\lambda = 14.4 \pm 0.4$ under the inherited 19.5 cap. The landing went $1{,}172 \times 945{,}335 \rightarrow 1{,}243 \times 804{,}160$ (999,570,880, the first score below 10^9), and three same-day follow-ups reached $1{,}244 \times 797{,}905$ [57]. The repository head on 11 October (`c672dda`) still runs Leapfrog, with walk caps of 1,235–1,237 in its "spookyfrog" recipe variants [32].



one Leapfrog tick on rail $T = R_{t \bmod 2}$ from $B = R_{1-t \bmod 2}$:
$3 + e$ low bits of T removed; 139 ticks per walk, four rail traversals
and two payload passes per point addition; on the reverse tick s_0, s_1
are recomputed with CNOTs and s_3, k_1, k_2 are erased by measurement.

Figure 6: The Leapfrog tick ([57], concept page *leapfrog*).

Why the walks matter for fault tolerance

A point addition is four rail traversals and two payload passes of a sequential walk; the walk's steps are *inherently serial* (each depends on the previous one) and are built from ripple adders. In the challenge metric only the Toffoli count is priced, but on a fault-tolerant machine the *depth* is what the reaction time multiplies (Section 10): Julian's measurements in Section 4 put the Toffoli depth of a point addition at 0.8–0.95× its Toffoli count for every ripple-based construction. Fewer, fatter steps (Leapfrog) therefore also shorten the critical path.

2.3 The arithmetic layer: where the Toffolis are

Almost every Toffoli in any of these circuits sits inside a ripple-carry adder, and the levers of the *strength reduction*, *fusion*, *dead-code* and *uncompute* families mostly act there. The building blocks, in the order they were introduced into the frontier:

2.3.1 Temporary-AND adders, Cuccaro bridges, and the carry trade

The frontier's register-plus-register adder is Gidney's temporary-logical-AND ripple adder [38]: at position i the carry is $c_i = \mathrm{MAJ}(a_i, b_i, c_{i-1})$, computed by folding the previous carry into both operands with two CNOTs, writing the AND into a *fresh* wire with one Toffoli and XOR-ing the previous carry in. When the sum is done, each carry wire is erased with Hmr and a classically-conditioned CZ on the two operands—zero Toffolis (Section 2.5). The textbook Cuccaro adder [27] keeps the carry inside the operand (MAJ then UMA) at $\approx 2n$ Toffolis and one ancilla. The two sit at opposite ends of one trade:

n-bit adder	Toffolis	owned carry wires
temporary-AND ripple, with carry-out ("vented")	n	$n-1$
temporary-AND ripple, wrapped mod 2^n	$n-1$	$n-2$
Cuccaro MAJ/UMA	$\approx 2n$	≈ 1
one Cuccaro "bridge" stage inside a measured ripple	$+1$ per stage	-1 per stage

"Bridges" (`bridge.rs`) are single Cuccaro stages spliced into a measured ripple where the live count would otherwise exceed the cap: each trades one extra CCX for one fewer carry wire, and the number of bridges per replay cell is a per-round profile (`I35_PROFILE`) tuned against the peak. For a *classical* addend c the carry out of position i is identically zero below the first position that can generate one, so `add_const` emits $n - 2 - \mathrm{ctz}(c)$ Toffolis (dead-low carry elision, exact); two call sites can prove the first carry zero and save more [57].

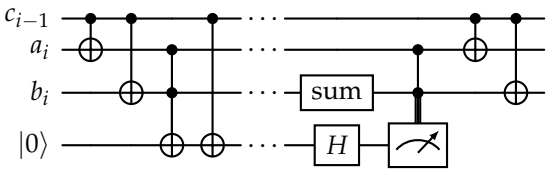


Figure 7: One slice of a temporary-AND ripple adder [38]. Left: $c_i = \mathrm{MAJ}(a_i, b_i, c_{i-1})$ is written into a clean wire with one Toffoli (the AND of the CNOT-modified operands) and one CNOT. Right: once the sum bit has been used, the carry wire is measured in the X basis (H then measure—the challenge's Hmr); outcome $m = 1$ leaves the phase $(-1)^{a'_i b'_i}$ on the two controls, which a CZ conditioned on m (the classical wire going up) removes; the CNOTs are then undone. Net: one Toffoli per carry instead of two.

2.3.2 Chunked adders and measured carry erasure

A full ladder keeps $\approx n$ carry wires live. When they do not fit under the cap the add is cut into chunks; each chunk unwinds its own carries before the next starts, so only one chunk's ladder is live at a time, and the carry left on the boundary wire between chunks is *erased by measurement and repaired by a comparison*: the carry out of sum $=$ acc $+$ addend is exactly $[\text{sum} < \text{addend}]$, so after Hmr on the boundary carry, on outcome 1 a *phase oracle* $(-1)^{[\text{sum}<\text{addend}]}$ computed on the top k bits of the two operands cancels the kickback (`compare::erase_with_compare`, `cmp_lt_phase_conditioned`). If the whole chunk is compared and its carry-in is known the erase is exact; if only the top k bits are compared it is wrong exactly when both operands agree on all k bits, probability $\approx 2^{-k}$ per call—and since the oracle runs only on the $m = 1$ shots, a comparator bit costs about 0.5 *executed* Toffolis. Chunking saves qubits and costs Toffolis; moving from two chunks to three is "a step worth hundreds of Toffoli" [57]. The same phase-oracle erase clears the modular adder's overflow flag (`ERASE_COMPARE`) and the replay cell's flag (`flag_compare`).

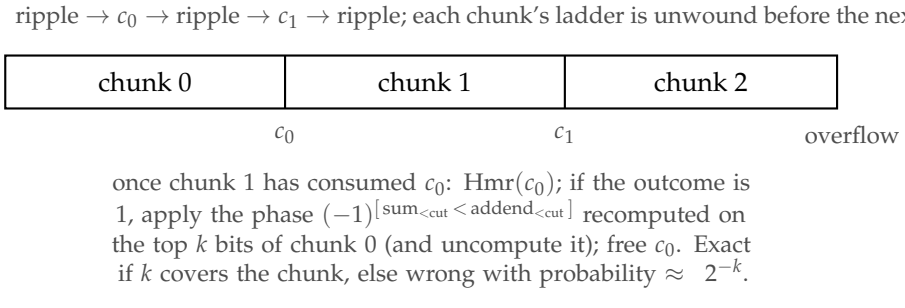


Figure 8: Chunked addition with measured boundary carries (concept pages *adders-and-carries*, *comparator-and-width-truncation* of [57]).

2.3.3 Truncated comparators and width schedules

This is the most frequent change-type of the whole competition (184 of 584 accepted commits by the wiki's count). Two cases recur. (a) A comparator that exists only to clean up a carry or flag (above) is narrowed to the top k bits; the error is a top-k tie. (b) A register that provably shrinks during the walk is narrowed round by round: ping-pong's `PP_WIDTH_SCHEDULE` ("259,258x19,257x5,...,8x9,8x2"), TrailMix's `SCHED_J2/GAP_J2`, Skywalk's and Leapfrog's envelope files. A trimmed top wire is a CNOT from the wire below it followed by a free—valid only if it was a sign copy. The schedules are *fitted*, not proved: `b5796ce` gave each round the smallest width whose violation count stayed within a λ budget of about 1.5 per draw. Per-round *shapes* then equalise the per-site error, since minimising total bits at fixed total error means equal error per site (`PP_CHUNK_SHAPE` "64:0,38:1,25:-3,0:-4", `PP_FLAG_SHAPE`). Two things dis-

tinguish an *exact* truncation (narrowing to a proven-non-zero prefix, e.g. the Kaliski invariant bitlen u + bitlen $v_w \leq 2n - i$) from a *probabilistic* one: the former adds no hard inputs; the latter is a "sell" of Toffolis for λ that must be "bought" back somewhere else to stay under the cap [57].

2.3.4 Pseudo-Mersenne folds

Because $p = 2^{256} - f$ with $f = 2^{32} + 977$, $2^{256} \equiv f \pmod{p}$: anything that spills past bit 256 is reduced by adding the spill times f into the low bits. A modular add/sub (`mod_addsub`) is: integer add with the top carry vented onto an *overflow* wire; a constant add of f into the low $fs = 33 + \texttt{FOLD_GUARD}$ bits controlled by that wire; erase the wire with a measured compare. The dropped carry off the top of the window is the whole approximation, with error $\approx 2^{-(\texttt{FOLD_GUARD}+1)}$ per call (a carry escapes only through $\texttt{FOLD_GUARD} + 1$ consecutive ones). Narrowing a window is one of the commonest Toffoli levers and the commonest *qubit* lever inside the replay ("the cell's fold is the widest thing alive in the trailing batch"). The replay cell folds by one of $\{-f, 0, +f, +2f\}$ selected by the sign, the parity of the sum and the overflow, without building an operand register (`fold_selected`); the square's high limb is folded with f written in non-adjacent form $f = 1 + 2^4 - 2^6 + 2^{10} + 2^{32}$ (five signed terms, the *signed-digit constant decomposition* of the strength-reduction family) [57, 75].

2.3.5 Karatsuba squaring with pseudo-Mersenne modulo (stage 4)

Stage 4 computes $X \leftarrow X - \lambda^2$. Writing $\lambda = L + \beta H$ with $\beta = 2^{128}$ and $U = L^2$, $V = H^2$, $W = (L + H)^2$,

$$\lambda^2 = U + \beta(W - U - V) + \beta^2 V \equiv U + \beta(W - U - V) + fV \pmod{p}, \tag{8}$$

so three half-size squares replace one full square, the $2^{256}V$ limb folds to fV, and the five signed terms of f turn fV into shifted additions. Squaring is cheaper than multiplying because the cross products $y_i y_j$ are symmetric (each is needed once) and the diagonal terms $y_i^2 = y_i$ cost nothing: the three squares have 24,512 unordered cross products, 24.9% fewer than a direct 256-bit symmetric square (32,640). The circuit-level specialisations are what make it cheap (Figure 9): each of W, U, V is built in a $2|x|$-bit register, its signed contribution is applied straight into X, and it is unbuilt before the next is built, so no 512-bit square is ever live; leaf squares are triangles of add-or-subtract rows selected by x_i (a complement frame, no controlled gates; `square_addsub_vented`, −43,467 T at `422f21d`); the split recurses while a half is at least 64–65 bits (break-even near 60 bits by the cost model $B(m) = m(m-1)/2 + 4m - 3$ vs. $K(m) = B(a) + B(b) + B(b+1) + a + 7b + 1$), limited by the qubit peak rather than the Toffoli count [57, 75]. The identity is exact; every approximation lives in the fold windows.

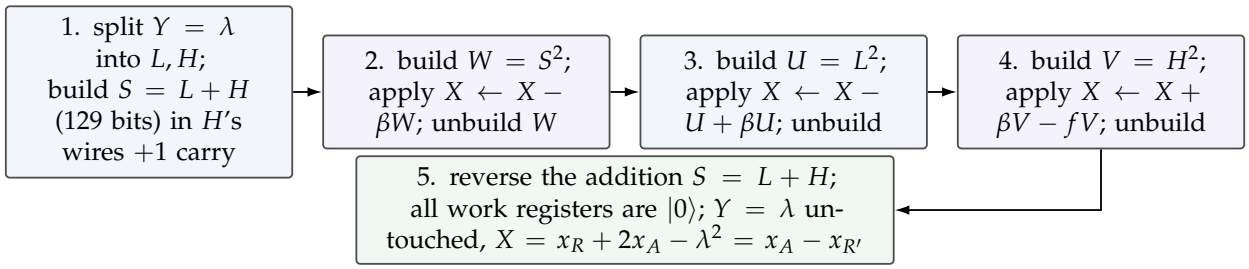


Figure 9: The build–apply–unbuild schedule of the symmetry-aware streamed Karatsuba square (Supplemental Note D, p. 13).

2.3.6 Constants, strength reduction, and fusion

Classical specialisation (constant propagation family): since x_A is classical, $3x_A \bmod p$ is computed on the host and loaded, so stage 3 is one modular addition instead of two additions, a dou-

bling and a halving (`662e267`); the final $x_A - X$ is $\overline{X} + (x_A + 1)$ with the pseudo-Mersenne correction (Figure 2, stage 6); Häner et al.'s $x \mathrel{-}= (2 - q)x_2$ is the same idea with a control. A *guaranteed-prefix* specialisation runs the first 304 Kaliski iterations with a branch-free body because the sum invariant proves they cannot terminate ($-158{,}254$ T, 3.6%). *Strength reduction*: inlining the add-c/branch/parity pattern of the Solinas reduction into Kaliski's step 8 ($-2{,}090{,}712$ T); $acc - X \equiv acc + (p - X)$ so the cheaper conditional *adder* drives the inner loop ($-256{,}016$ T); Litinski's add-subtract schoolbook rows ($n - 1$ instead of $2n - 1$ Toffolis per row); rerouting five call sites to the carry-ancilla adder where there is local headroom ($-17{,}400$ T for $+21$ qubits). *Fusion* merges exactly two adjacent operations to delete one load/unload boundary: a self-inverse comparator called twice (compute a flag, body, uncompute the flag) collapses to one MAJ sweep, read, body, read, inverse sweep ($2n$ instead of $4n$ Toffolis; $-2{,}096{,}640$ and $-2{,}087{,}232$ T for the two comparators of Kaliski's step 2); two conditional Cuccaro calls sharing an operand share one `tmp` ($-524{,}288$); the Solinas reduction's unconditional add of c followed by a conditional un-subtract collapses to one conditional add once the post-shift overflow bit is recognised as the control ($-655{,}864$); and the boundary cswap merge of Figure 3 [75]. The windowed Solinas shift (windowing family) replaces 22 consecutive no-op modular doublings between bit 9 and bit 32 of c by one classical shift-by-22 into a spill register, one sparse correction and one conditional subtract, with the reverse shift replaying the same flags against the same qubits—an earlier attempt that recomputed an independent inverse failed the phase check.

2.4 The qubit layer: owning the peak

2.4.1 Peak owners, co-binders, and budgets

Q is a single moment: the instant when the most wires are live. Making a phase cheaper does not lower Q unless that phase is where the maximum is, and the maximum is usually shared by several *co-binders*—in the ping-pong era the replay ladder and the square ladder both reached 1,278, so a one-qubit win meant lowering both budgets in the same commit. Modern builders size their scratch against a *cap* (`TLM_TARGET_Q`, `SUB4_PP_PEAK`, `PP_WALK_MAX_QUBITS`): each adder, comparator and fold looks at cap − active and picks the widest (cheapest) layout that fits, so the cap is a dial that trades qubits for Toffolis, and a trade pays exactly when

$$(Q + \Delta Q)(T + \Delta T) < QT \iff \Delta T < -\Delta Q \cdot \frac{T}{Q + \Delta Q}, \tag{9}$$

i.e. one qubit is worth T/Q Toffolis: $\approx$ 1,130 T in late July (1,151 $\times$ 1.30M) and $\approx$ 800 T at the October frontier. Many commits are nothing but such retunes (`f5ff95b`: cap 1,150 $\to$ 1,158 for $-9{,}147$ T, net -1.48M score; `087cafa`: owners 1,278 $\to$ 1,275 for $+1{,}491$ T, net -0.85M) [57].

2.4.2 Venting, wire loans, parking, recompute-later

Freeing early lowers the high-water mark because the builder's allocator hands freed indices out again (LIFO). *Loans* lend a wire whose value is known to the peak-binding code: a duplicated bit (after the fixed ping-pong walk $u, v \in \{\pm 1\}$, so bits 1..7 of each 9-wire register are sign copies: 16 wires, 1,317 $\to$ 1,301 at `152fbe0`), a constant (the odd operands' bit 0, cleared with an X), a provable zero (the square's top cross bits), or a reconstructible value (erase a retained AND with Hmr+CZ, lend, rebuild with a Toffoli). A loan costs two CNOTs: `cx(sign,q); release_clean(q); body; reacquire(q); cx(sign,q)`. *Parking* hosts a temporary inside a live wire whose value can be restored (the adder's final carry written straight into the top output bit; the Karatsuba sum kept in the input's high half: -6 qubits for $+2{,}372$ T at `51c6c31`). *Recompute-later* erases a bit in the X basis before the peak and rebuilds it afterwards from live state, cancelling the measurement phase with Z^m (`5526766`: -2 Q for $+112$ T; `265a9e9`: a tape sign rebuilt with CNOTs alone from a linear walk invariant, -1 Q at 0 T). *Rescheduling* shortens lifetimes with no new identity (erase each boundary carry as soon as the next chunk has used it;

interleave walk and replay). In the census all of these are *register and scratch allocation*; their condition for exactness is the same every time: the claimed value holds on every input when the loan starts, its source is not touched inside the body, the wire comes back at $|0\rangle$, and the repayment uses the relation that cleared it [57, 75].

2.4.3 Register-shared Euclid and the 825-qubit circuit

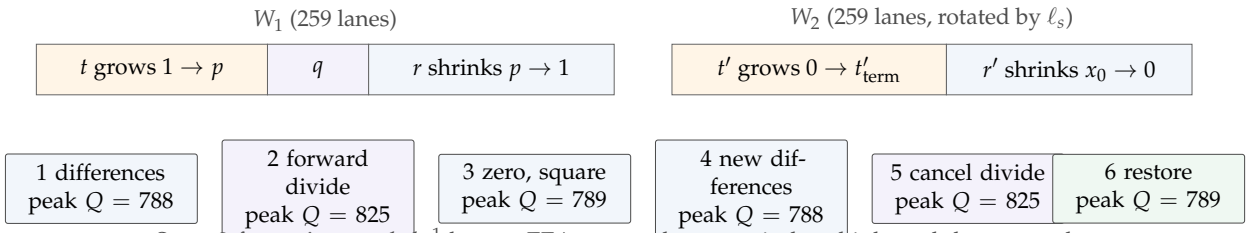
The lowest-width admitted circuit at the cutoff, `b6f2b0a` ($Q = 825$, $T = 489{,}161{,}900$), is a different architecture: TrailMix's *shrunken Proos–Zalka* route, a register-shared extended Euclidean algorithm of the kind described by Luo et al. [78], with no dialog transcript at all (Supplemental Note E). The remainders shrink while the Bézout coefficients grow, so the two pairs are packed into two 259-lane banks with movable boundaries,

$$W_1 = t \,\|\, q \,\|\, r, \qquad \operatorname{bitlen} t + 1 + \operatorname{bitlen} q + \operatorname{bitlen} r \le 259,$$
$$W_2 = \operatorname{rot}_{\ell_s}(t' \,\|\, r'), \qquad \operatorname{bitlen} t' + \operatorname{bitlen} r' \le 259,$$

length metadata $\ell_t, \ell_q, \ell_{t'}, \ell_{r'}, \ell_s$ addresses the active fields, and a fixed schedule of 1,479 reversible microsteps performs the restoring divisions $r \leftarrow r \mp 2^{\ell_s} r'$, $t' \leftarrow t' \mp 2^{\ell_s} t$ with the quotient bits built and consumed inside one round (Figure 10). The circuit carries the complete packed Euclidean state and *reverses the scheduled gate sequence* to clean it; the price is high T (92.15% of the circuit's Toffolis are in the two EEA stages), the benefit is the peak equation

$$\underbrace{259 + 259 + 257 + 37}_{812 \text{ persistent}} + \underbrace{13}_{\text{largest scheduled scratch}} = 825, \tag{10}$$

where 257 is the passenger coordinate d_y and $37 = 9_{\ell_t} + 1_{\ell_{t'}} + 7_{\ell_q} + 8_{\ell_s} + 8_{\ell_{r'}} + 4_{\text{control}}$ is the metadata. The Q826→Q825 step removed one persistent ℓ_q lane by reconstructing its high bit only where it is consumed: for a 259-bit source with bit length L, $h_7 = [128 \le L \le 255] = Z_{256} \oplus Z_{128}$ where Z_k is the zero-prefix predicate on positions $\ge k - 1$, computed as a reversible predicate, used as a control and uncomputed—a recomputation-for-storage trade licensed on the benchmark support ($\max \ell_q = 22$ observed over 18,048 inversion factors), not an all-input proof. Three disjoint Q826 host windows (an inactive t' lane hosting remainder-update scratch, an inactive ℓ_s lane hosting a coefficient carry, another t' lane hosting the rotated-swap scratch) and *dynamic cursors* (the length register itself is decremented during a scan and restored by the reverse scan, so no copied cursor is live) are the other named mechanisms. ECDSA.fail's low-Q track continued after the cutoff: 813 (`b4a51d2`) and, on the 7 October leaderboard, 792 qubits at 656.6 M Toffolis (`10177cd`) [57].



Stage 2 forms $\lambda = d_y d_x^{-1}$ by one EEA traversal, a canonical multiply and the reversed traversal. Stage 5 erases λ with a *fresh* witness: after stage 4, $d'_x = x_A - x_{R'}$ and $d'_y = \lambda d'_x = y_{R'} + y_A$, so a second EEA on d'_x recomputes $q = d'_y (d'_x)^{-1} = \lambda$ and resolves a *ghosted* copy of λ (Section 2.4.4); no history of the first inversion is retained.

Figure 10: The low-Q circuit `b6f2b0a`: the two packed banks of the register-shared EEA and the six stages with their peak widths (Supplemental Note E, pp. 4, 8 and 14).

2.4.4 Ghosting: measurement-based release and later recomputation

The low-Q route frees a whole 257-lane field register during the reverse EEA without losing coherence. If a reversible computation has produced $\sum_x \alpha_x |x\rangle |f(x)\rangle_D$, measuring each lane of D in

the X basis yields a classical string m and leaves $\sum_x \alpha_x (-1)^{m \cdot f(x)} |x\rangle$; the lanes of D are released (Figure 11). Later the same Boolean function is recomputed lane by lane into a fresh register ($d_y = \lambda d_x$ after d_x has been restored) and Z^{m_j} on each rebuilt lane contributes $(-1)^{m \cdot f(x)}$ a second time, so $(-1)^{2m \cdot f(x)} = 1$ and the explicit $|d_y\rangle$ is recovered exactly. Luo et al. use exactly this to run their point addition on three field registers [78]; it is valid whenever the later register holds the same function as the measured one—it is not erasure, and it does not assume d_y is classical.

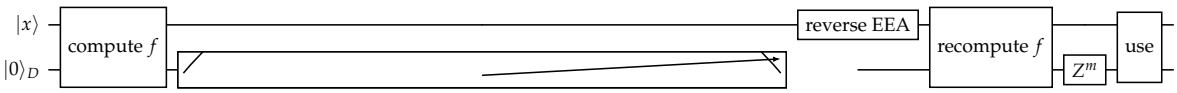


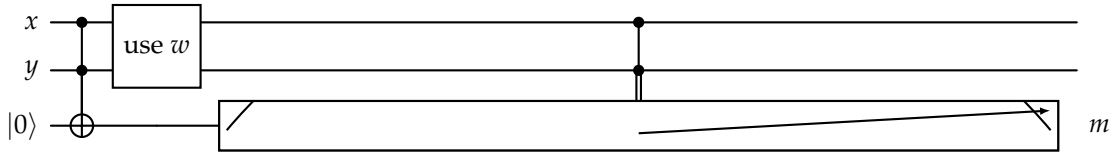
Figure 11: Ghosting (Supplemental Note E, p. 12; [78]). The register holding $f(x)$ is X-measured (Hmr, outcome string m) and freed; the phase $(-1)^{m \cdot f(x)}$ it leaves is cancelled by Z^m on a later, independently recomputed copy of $f(x)$.

2.5 Measurement-based uncomputation, in general

Every scratch value must return to $|0\rangle$ with no residual phase (the reversibility and phase checks of Section 1.3), so the question is never *whether* to uncompute but *how*. The textbook way—run the computation backwards—costs as many Toffolis as the computation. The alternative is the pattern already used three times above: measure the wire in the X basis, which resets it but leaves a known phase $(-1)^{m \cdot f}$ depending on the random outcome m; then cancel that phase with gates conditioned on m. The whole family (census: *uncompute strategy*, 22 commits, all "measurement uncompute") differs only in how cheap the fix-up is [57, 75]:

- f is an AND of two live wires, or any *quadratic* function of live wires: the fix-up is CZ/Z gates only, 0 Toffolis. Examples: the AND-uncompute of Figure 7 (`clear_and`, `measured_and_clear`); the *from-zero* adders (`5b834ba`: if an operand bit is provably $|0\rangle$ on entry the MAJ step leaves exactly $\mathrm{AND}(x, y)$ in it, so the UMA Toffoli becomes Hmr+`cz_if`); the *known-output receivers* (`add_known`, `known_stream`: when the adder's result is already known as an affine function of live wires—a reverse pass restoring a value—each carry is rebuilt with CNOTs as $c_i = a_i \oplus b_i \oplus s_i$ and then X-measured; written in terms of the sum bit and the incoming carry each carry is quadratic, so the repair is Z/CZ: an adder with *zero* Toffolis, valid on the subspace where the output really is known); the *phase-only comparator tops* (a borrow $\mathrm{MAJ}(q, a, x) = qa \oplus qx \oplus ax$ needs only its phase, three CZs instead of a Toffoli).
- f is a comparison (a chunk carry-out, an overflow flag): the fix-up is a phase oracle (compute, Z, uncompute) under `push_condition(m)`, executed on half the shots—so its *executed* Toffoli cost is half its static cost, and truncating it to k bits makes it approximate (Section 2.3.2).
- the opposite choice, *deferred coherent uncompute* (`b788efc`): keep the square's chunk carries live until the inverse pass recomputes them into the held wires (exact, no conditioned compare), paying with loans of provably-zero wires.

Two things break the idiom: a control that changes after the AND was built (the CZ then applies the wrong phase), and an approximate fix-up. A systematic error shows up as `PHASE GARBAGE` with near certainty; a rare one adds hard inputs to the phase channel. The simulator models Hmr as: draw m from the Fiat–Shamir stream, store it, phase $\oplus= q \wedge m$, $q \leftarrow 0$; a plain reset R does the same but discards m, which is why freeing a dirty wire is never safe. Hmr cannot be replayed by `emit_inverse`, so any backward pass containing it is written by hand.



compute: $w \leftarrow x \wedge y$ (one Toffoli; four T gates fault-tolerantly)
uncompute: Hmr$(w \to m)$ leaves $(-1)^{m\,xy}$; apply CZ(x, y) iff $m = 1$
cost of the uncompute: 0 Toffolis, one measurement, one feed-forward bit
condition: w must still equal $x \wedge y$ of the *current* controls

Figure 12: Gidney's temporary logical AND with measurement-based uncomputation [38], the primitive behind the "uncompute strategy" family. The challenge counts the compute Toffoli and not the uncompute, which is why its T is "executed Toffolis".

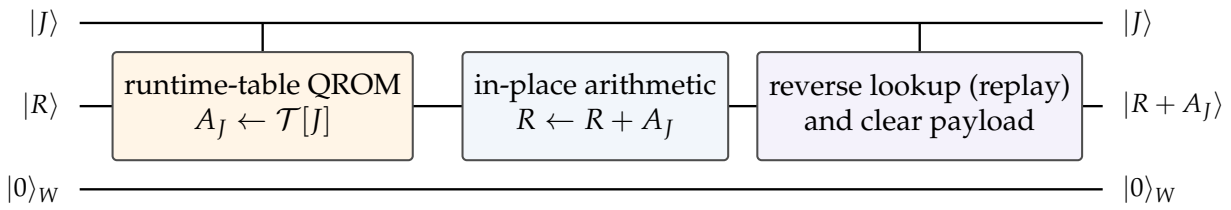
2.6 Post-passes, census strips, and the nonce economy

After the circuit is emitted and before the nonce tail is appended, `build()` runs passes over the op stream. *Exact* ones: self-inverse pair cancellation (a CCX followed by the same CCX with nothing touching its operands in between), constant propagation of $\{0, 1, ?\}$ and affine relations (drop a CCX with a known-0 control, turn one with a known-1 control into a CX), the target-fanout identity $\mathrm{CCX}(a, b \to t); \mathrm{CX}(t \to u); \mathrm{CCX}(a, b \to t) = \mathrm{CX}(t \to u); \mathrm{CCX}(a, b \to u)$ looped to closure, the four simplifiers `product`, `affine`, `quadratic`, `truth` that re-derive Boolean support facts from the stream, and *SAT-proven CCX→CX rows*: an offline solver proves that a specific Toffoli's product equals a fixed XOR of other wires at that point of the stream, and the gate is replaced by a CNOT chain (the October frontier ships 105 such rows, 102 of them in the square). *Empirical* ones: a *fire census* simulates the stream on 10^8–10^9 random inputs, records which Toffolis ever fired and which had an implied second control, and deletes or downgrades the rest, admitted in ascending risk until a λ budget is spent (`DEAD_KEYS/DOWNGRADE_KEYS`, identity-keyed so that an upstream edit cannot mis-address them, after a positional strip once caused 7,535/9,024 mismatches). As of 1 October the frontier's build ran no census strip, only the exact chain and the SAT rows [57]. Finally, *verifier search*: a nonce-only change moves the score by a few Toffolis because the average is over a different sample (-5, -9, -24 T in three admitted commits), and every real change forces a re-hunt whose expected cost grows like e^{λ}: around $\lambda \approx 8$ a hunt is minutes to hours on GPUs, at 15–20 it needs a pre-filter and many GPU-hours, beyond that it is infeasible even when the Toffoli count looks better. The measured $\lambda \approx 15.7$ of one tree (9.5 classical, 6.2 phase) gave one clean draw in $\approx$ 29,000 rather than the naive $e^{-15.7} \approx 1/6.8\,\mathrm{M}$, because classical and phase failures are correlated [57].

2.7 Making the circuit window-compatible

In the challenge A arrives as classical bits; in Shor's algorithm it is *selected coherently* by a w-qubit address J. Supplemental Note F adapts `8e9c9a2` to the interface $U_{\mathrm{win}} : |J\rangle|R\rangle|0\rangle_W \mapsto |J\rangle|R + A_J\rangle|0\rangle_W$: the circuit exposes X, Y, J and $3 \cdot 2^w$ classical table columns (x_j, y_j, $3x_j \bmod p$, with row 0 encoding $\mathcal{O}$ as zeros), conditions every payload CNOT on the runtime table bits, and *replays* each lookup after use (Figure 13). The lookup is an XOR, $L_c : |j\rangle|z\rangle \mapsto |j\rangle|z \oplus c_j\rangle$, implemented by a reversible address traversal (unary iteration over the w address bits with AND ancillas) that applies a classically conditioned CNOT to each payload bit whose table bit is one; applying it twice clears the payload exactly, $|j\rangle|c_j\rangle \mapsto |j\rangle|0\rangle$, and the traversal's routing ANDs are removed by measurement with phase correction. Stage 3 uses a $K = 2$ QROAM [77]: with $j = 2h + \ell$ the high-address traversal loads two adjacent rows into banks B_0, B_1 and ℓ-controlled

swaps select the requested one. The identity row is handled by a coherent predicate $b = [J \neq 0]$ that controls the square-subtract of stage 4 and the negation of stage 6, so $R + \mathcal{O} = R$ without measuring J. Cost: $Q = 1{,}162$ (+11, because the payload banks are cleared before the dialog-GCD peak), $T = 1{,}683{,}828$ (+29.6%; the three traversal pairs cost 328,180 executed Toffolis), serialised operations 25.21× more (table-payload CNOTs), error rate 0.1950% against the baseline's 0.1920% on a shared 100,000-input corpus—statistically indistinguishable [76].



stage-local schedule: load x_J, y_J (shared traversal) → stage 1 → replay-unload → stage 2 → load $3x_J$ (K=2 QROAM) → stage 3 → unload → stage 4 (controlled by $[J \neq 0]$) → stage 5 → load y_J, x_J → stage 6 → unload. No payload is live across the dialog-GCD peaks.

Figure 13: The runtime-table window interface of the window-compatible `8e9c9a2` (Supplemental Note F, Figs. 1–2).

3 The Pareto frontier

3.1 Three constructions, and which one is a frontier

Supplemental Note A is careful to keep three objects apart [76]. The *product-score incumbent sequence* is the chronological list of promoted submissions, each of which lowered $Q \times T$ when it landed; it is what the leaderboard shows and what Figure 15 plots. The *low-width track* is a mechanism-oriented selection of validator-clean circuits below the product-efficient regime (q980, q988, q952, q1019, q948, q828, q826, q825 in ledger order). The *admitted* (Q, T) *Pareto set* is the only one of the three that is a frontier in the multi-objective sense,

$$\mathcal{P}(\mathcal{A}) = \{c \in \mathcal{A} : \nexists\, d \in \mathcal{A} \text{ with } Q(d) \leq Q(c),\ T(d) \leq T(c),\ \text{one inequality strict}\}, \qquad (11)$$

over the set $\mathcal{A}$ of *admitted* rows: promoted ones, plus rejected ones ("rejected" is the CLI's word for a valid run that did not beat the incumbent) that carry public evidence of a complete 9,024-instance clean run. At the 26 July cutoff $\mathcal{P}$ had eight points, six of them rejected low-Q rows that a winner-only leaderboard would have discarded (Table 4).

submission	Q	T	status	interpretation
Admitted Pareto set at the data cutoff, 26 July 2026 (Table 2 of the paper)				
`b6f2b0a`	825	489,161,900	admitted	lowest admitted width (register-shared EEA)
`f1d8707`	826	479,337,428	admitted	one qubit coincides with 9,824,472 fewer Toffolis
`39a9b5f`	828	448,102,916	admitted	two more qubits, 31,234,512 fewer ($\approx$ 15.6 M per qubit)
`a3b0148`	948	54,781,961	admitted	exit from the hard-Q metadata cliff
`4352cfb`	980	28,847,321	admitted	lowest admitted T below 1,000 qubits
`1c0e0e9`	1,133	1,460,511	admitted	bridge toward the product-efficient regime
`8e9c9a2`	1,151	1,299,453	promoted	$Q \times T$ incumbent, 1,495,670,403
`a536a48`	1,158	1,292,651	promoted	lowest-T promoted point
Non-dominated rows of the 7 October 2026 snapshot, promoted or rejected with API metrics [57]				
`10177cd`	792	656,602,578	rejected	lowest width on the snapshot
`87cffbd`	794	415,821,960	rejected	
`8d26593`	795	300,862,444	rejected	
`c60617f`	807	276,427,457	rejected	
`5c4ec19`	822	250,088,365	rejected	
`3170f7b`	823	154,247,477	rejected	
`05f08d8`	833	61,654,186	rejected	
`48d934d`	838	22,019,787	rejected	
`a9d89c2`	973	11,844,334	rejected	
`9d4282b`	998	10,438,839	rejected	
`4188053`	1,011	9,329,693	rejected	
`57aa8ae`	1,072	2,791,974	rejected	
`c1c0da9`	1,112	1,739,356	rejected	
`dd3621d`	1,145	1,132,785	rejected	
`17250b2`	1,164	970,983	rejected	
`b72ce1a`	1,170	946,553	promoted	Skywalk, 6 Oct
`ee41004`	1,174	943,826	promoted	Skywalk, 3 Oct
`c039e59`	1,243	802,520	promoted	Leapfrog, 7 Oct
`a02c716`	1,244	797,905	promoted	Leapfrog, 7 Oct: $Q \times T = 992{,}593{,}820$

Table 4: The admitted Pareto set of the paper and the non-dominated set of the latest public snapshot. The post-cutoff "rejected" rows passed the evaluator (0/0/0 on their own island) but did not improve the scalar score; the paper's stricter curation (a public note with a full clean run) was not re-applied here, so the lower block is the API-reported frontier, not a re-audited one. Several intermediate promoted rows (1,171–1,173 qubits) are omitted for brevity.

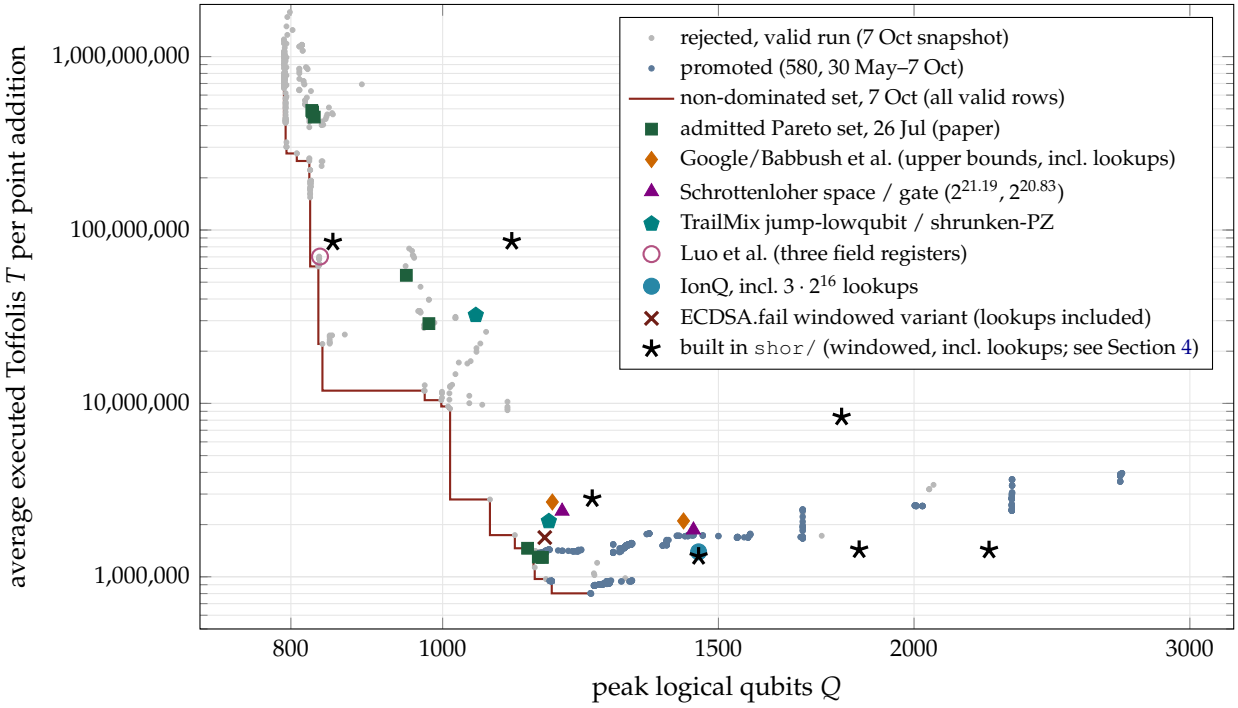


Figure 14: The (Q, T) plane. Grey and blue dots are every valid run on the 7 October snapshot (459 rejected, 580 promoted); the red staircase is their non-dominated set; green squares are the paper's admitted Pareto set at the cutoff. The external points are plotted where the sources put them, but they are *not* measured under one accounting rule: Google's two points are upper bounds attested by a zero-knowledge proof for a circuit that includes coherent lookups; Schrottenloher's, TrailMix's, IonQ's and the `shor/` points are windowed additions that include their table lookups (Section 2.7); the ECDSA.fail points are for a classical addend and executed Toffolis on the challenge distribution. The paper's own external table carries the same caveat [75].

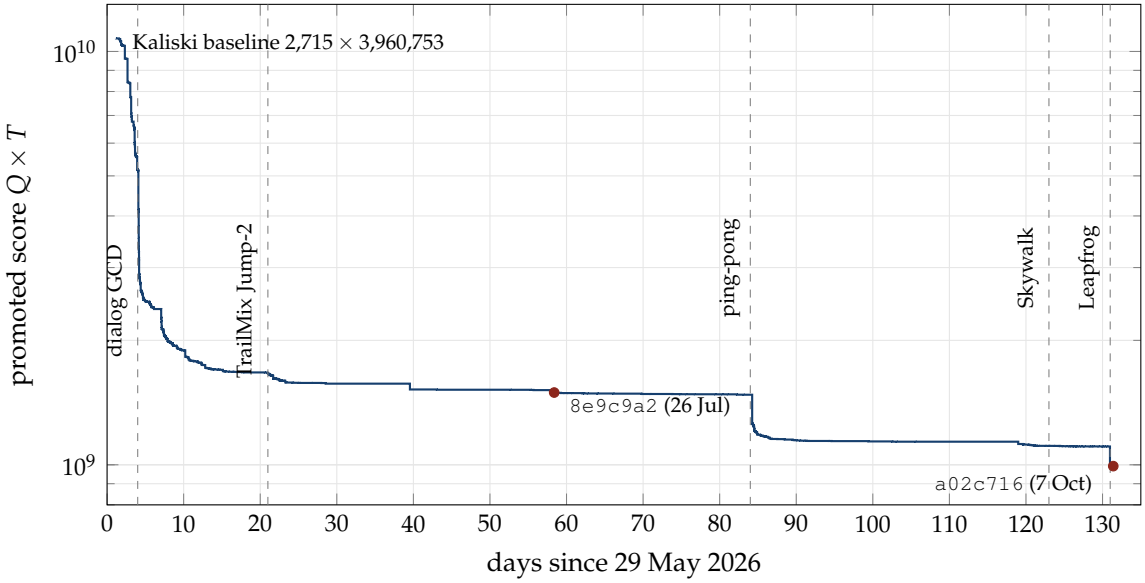


Figure 15: The product-score incumbent sequence: all 580 promoted submissions on the 7 October snapshot, with the five changes of inversion architecture marked. From 10.75×10^9 to 0.99×10^9 in 131 days; the paper's three eras (I: joint peak owners, to 5.69 B at `375fdff`; II: hosted and decoded history, to 1.83 B at `4b7d727`; III: last-consumer proofs, to 1.57 B at `71f5115`) all lie inside the first two architectures [76].

3.2 Reading the frontier

The secants are not marginal costs. Moving from 826 to 828 qubits coincides with 31.2 M fewer Toffolis, about 15.6 M per qubit; from 980 to 1,133 with more than 27 M fewer; from 1,133 to 1,151 with only 161,058 fewer, and from 1,151 to 1,158 with 6,802. These are descriptive secants between *different circuit families*, not derivatives of one design; what they evidence is a steep "hard-Q" transition among the observed routes, a cliff below $\approx$ 950 qubits where resident arithmetic state has been replaced by metadata and recomputation and the Toffoli count rises by orders of magnitude [76]. The fixed-budget slices are discontinuous for the same reason: no admitted circuit between 981 and 1,050 qubits improved on the 980-qubit row's count.

What the frontier is relative to. The baseline of the challenge was $2{,}715 \times 3{,}960{,}753 = 10.75$ B (with the 9,000-pair `DUMMY_TOFFOLIS` padding; 3,942,753 unpadded). The cutoff incumbent `8e9c9a2` is 86.1% lower (7.19×); its Q fell 57.6% and its T 67.2%; the product tip (1,151) and the Toffoli tip (1,158, `a536a48`) are different rows. The 7 October incumbent `a02c716` is 90.8% below the baseline; it spent 93 qubits to buy 502k Toffolis relative to `8e9c9a2`. Google's attested points are 3.17 B and 2.99 B retry-adjusted (their reported circuits were treated as zero-error); the paper's claim is that the challenge reached "more than 50% below" them *under its own accounting*, which excludes the coherent lookups that Google's circuits include (Section 2.7: adding them costs +11 qubits and +29.6% Toffolis) [75].

Retry-adjusted scores. Because frontier circuits are approximate, the paper also plots $Q \times T/\hat{p}$ with $\hat{p}$ the success rate on a common 50,000-input corpus and, for a full attack with 28 additions, $28\,Q'T'/(1-28e)$ with $Q' = Q + 16$, $T' = T + 3 \cdot 2^{16}$, $e = 1 - \hat{p}$; $\hat{p}$ stays near 1 on the point-addition axis (the 0.192% failure of `8e9c9a2` would cost $\approx$ 5.3% additional expected work under an independently re-runnable success model), but some nominal improvements shrink or reverse once the 28× amplification is applied, and the union bound is only a plug-in proxy: $\hat{p}$ measures classical-output agreement, not coherent circuit success, and silent errors inside Shor's algorithm accumulate coherently across additions [75, 76].

artifact	Q	Toffolis	accounting
Google/Babbush low-Q [5]	1,175	2.7 M	disclosed upper bound (ZK proof); lookups included
Google/Babbush low-gate [5]	1,425	2.1 M	same package
Schrottenloher space-optimised [90]	1,192	$2^{21.19}$	reproducible architecture, windowed, plus 16 window qubits
Schrottenloher gate-optimised [90]	1,446	$2^{20.83}$	
TrailMix `jump-lowqubit` [75]	1,169	2.09 M	open implementation prior
TrailMix `shrunken-PZ` [75]	1,050	32.3 M	architectural prior for the low-Q track
Luo et al. [78]	835	70.29 M	$3n + 6\lfloor \log_2 n \rfloor + 19$ qubits
IonQ/Häner et al. [54]	1,457	1.196 M $+3 \cdot 2^{16}$	windowed, signed-window ready; 28 additions $\Rightarrow$ 39.0 M
ECDSA.fail `8e9c9a2`	1,151	1,299,453	classical addend, executed Toffolis
ECDSA.fail windowed `8e9c9a2` (Note F)	1,162	1,683,828	coherent lookups included
ECDSA.fail `a02c716` (7 Oct)	1,244	797,905	classical addend
ECDSA.fail `b6f2b0a` / `10177cd`	825 / 792	489 M / 657 M	extreme-width evidence
`shor/`: IonQ cells, signed windows, SELECT-SWAP	1,457	1,310,693	windowed, executed; 37.1 M for 28 additions
`shor/`: dialog on the space cells	1,246	2,821,091	windowed
`shor/`: three field registers (Luo)	851	85,204,396	windowed; 2.39 G for the attack

Table 5: Neighbouring public operating points for one secp256k1 point addition, under heterogeneous accounting rules (after Table 3 of [75], extended). "Reported count" is not a uniform metric across rows.

4 The `shor/` implementation against the ECDSA.fail catalogue

The Qiskit package in `shor/shor_qiskit/` (44 modules, $\approx$ 16,700 lines, plus $\approx$ 4,400 lines of benchmarks in `bench/`) builds the same object the challenge scores—one windowed secp256k1 point addition—from exhaustively tested toy-size primitives, and counts its 256-bit instantiation hierarchically (`hier.tracing()`: build each sub-circuit once per *shape*, append it as an opaque gate carrying its counts; at toy size the hierarchical count equals the flat one on every field and the tree exports to Qualtran bloqs whose `QECGatesCost` agrees). This section records which of the ECDSA.fail levers it already contains, what it measures for them, and what it does not have. The numbers are the package's own, from `bench/*.json` and `shor-complete.tex` Part VII.

4.1 Coverage

ECDSA.fail lever (section)	in `shor/`	measured (`secp256k1`, $w = 16$)
Kaliski almost-inverse, HJNRS form (Section 2.2)	`ec_kaliski`, `ec_kaliski_opt` (unconditional rounds, postponed reduction, Montgomery form)	29% fewer Toffolis and 41% less depth than HJNRS at $n = 12$
dialog GCD, record/replay (Section 2.2.2)	`ec_eea` (as printed), `ec_gcd.Dialog` (fused 77-bit compare, width schedule, PM cells)	in-place multiply 1,955,276 → 865,588 Toffolis
IonQ's conditionally-inverted walk, Fig. 1 record packing (3 pairs in 5 qubits)	`ec_gcd.CondInv`, `fig1_compress`	720,565 per in-place multiply; record 669 qubits
Jump-2 (Section 2.2.3)	`ec_gcd.Jump2`, `Jump2Packed` (261 steps)	1,161,099 per multiply from this package's generic cells (no fused cells, no base-5 codec)
ping-pong (Section 2.2.4)	`ec_gcd.PingPong` (704 rounds)	917,565 per multiply; walk 257/round, replay 785/step; 2,117,534 per addition on 2,155 qubits
register-shared EEA, three field registers (Section 2.4.3)	`ec_luo`, `ec_luo3` (ghosting via `ec_mbu.vent/zfix`)	division 835 qubits / 42,103,470 T; windowed addition 851 / 85,204,396; attack 2.386×10^9
Gidney classical–quantum constant adder	`ec_cqadd`	$2.98n$ (dirty) / $3.96n$ (3 clean) Toffolis; mod add $11.0n$, doubling $6.0n$ (Luo's $11n$, $6n$)
temporary-AND adder, measured uncompute (Sections 2.3.1 and 2.5)	`ec_gates.and_/and_dg`, `ec_adders`	2.50× fewer T over an inversion; CDKM vs Gidney $2n$ vs $n - 1$
pseudo-Mersenne folds, truncated compares (Sections 2.3.3 and 2.3.4)	`ec_approx` (Alg. 5–11 of [90]), `PMPhase` (IonQ's phase-approximate adder, $\kappa = 65$, $\delta = 32$, 37 careful steps)	cell 712 → 352 Toffolis (336 executed); failure rates measured at toy primes
dedicated squarer with fold (stage 4)	`ec_square` (IonQ's signed-row squarer)	1,050,623 → 79,160 Toffolis per square-subtract
QROM lookups, MBU unlookup, merged phase repair, SELECT-SWAP	`ec_window`, `ec_mbu`, `select_swap_lookup`	1,310,700 → 197,364 per addition; SELECT-SWAP on the $3x$ load $-16{,}128$ at no qubit cost
signed windows	`ec_signedwin`	2^{15}-entry tables, -2.7 M over 28 additions
exact peepholes, fire census (Section 2.6)	`ec_opt.peephole`, `ec_opt.census/strip_unfired`	census at $q = 61$: exact when it covers all inputs (1,680 → 991); 2.8% of fresh inputs wrong at 300 samples
lean (space) cells, register sharing beside the walk	`ec_space` (`PMSpace`), `CondInv(share=True)`	1,246 qubits at 2,821,091 T; 1,457 at 1,310,693 executed
carry-lookahead adders	`ec_cla`	7× Toffolis for $\log n$ depth; addition depth 1,503,533 → 394,281
physical model (Section 10)	`physical.py` (Gidney-2025 surface-code model)	reproduces 897,864 qubits / 4.96 days for RSA-2048

Table 6: What the package already implements of the ECDSA.fail catalogue, with its own measurements (`bench/ec_project_256.json`, `ec_toffoli_256.json`, `ec_space_256.json`, `ec_luo3_256.json`, `ec_catalogue.json`, `estimates.json`).

Three of its results sit directly on the plane of Figure 14: the IonQ-style addition with signed windows and SELECT-SWAP on the $3x$ lookup, 1,457 qubits and 1,310,693 executed Toffolis (5.9% below IonQ's 1,392,608 at IonQ's own width), the space variant at 1,246 qubits, and the three-register Luo circuit at 851 qubits—the top of ECDSA.fail's published low-Q range. The package also makes a point that the challenge metric hides: every ripple-based construction has a Toffoli *depth* of 0.8–0.95× its Toffoli count, so the fast machine of Section 10 is throughput-bound, not reaction-bound, until carry-lookahead adders (`ec_cla`, $\approx 7\times$ the Toffolis for $\log n$ depth) are used, after which the unary-iteration lookups set a floor of $3 \cdot 2^{16} = 196{,}608$ steps per addition.

4.2 What the package does not have, and what it would take

1. **The post-August walks.** `ec_gcd` has Kaliski, the dialog, CondInv, Jump-2 and ping-pong as backends on one `backend.mul(m,x,y,q)` interface, which is exactly the right place for *Skywalk* (Section 2.2.5) and *Leapfrog* (Section 2.2.6). Leapfrog is the natural next backend: its rails are two odd two's-complement registers, its tick is three add-halves and a barrel, its letter is five raw qubits (no codec), and its payload cell is ping-pong's `replay_add_halve` run at the proxy round of the rail width—all of which the package has or can compose from `PMPhase`. The parts that need new code are the choice rule (*sign2* plus the K-bit window compare), the barrel (rot4 from three controlled reflections, and the payload's $\times 2^{-e}$ mod p via $P - m(f-1)$), and the width-envelope fit. The package's own round-budget caveat applies: it found ping-pong's 704 rounds not to be a worst case ($n = 16$ needs $3.9n$ exhaustively), so a Leapfrog port should re-measure the parking depth rather than inherit 139.

2. **The base-5 codec and the live-width schedule of stage 2.** `Jump2Packed` packs symbols but not as base-5 digits; the 609-versus-783 tape, and the Note-D objective of making the shrinking $2w_i$ absorb the growing $|\tau_i|$ (Equation (6)), are what take a Jump-2 circuit from this package's 2,217 qubits to the challenge's 1,151. The package's `CondInv(share=True)` already does the equivalent for IonQ's walk (the record grows into the lanes the width schedule frees); the same mechanism applied to `Jump2Packed` with the codec is a contained change.

3. **Fused replay cells.** The challenge's replay step costs ≈ 785 Toffolis in the package's generic cells but far less in the fused `signed_mod_add_pm_halve_fused/replay_add_halve` cell (chunked add with measured boundary carries, selected fold by $\{-f, 0, +f, +2f\}$, measured flag). The package's `PMPhase.signadd` is the same idea for IonQ's adder; the remaining gap is the fused halving-with-fold and the per-round chunk/flag/fold *shapes*.

4. **λ-priced truncation and nonce hunting.** The package measures failure rates of each approximate cell at toy primes and composes them; it does not do what every frontier submission does, which is to fit width schedules against a λ budget on the actual 256-bit distribution and to hunt an island. That machinery is not needed for an honest resource estimate (the package's circuits are either exact or carry a measured rate), but it is needed to *reproduce* a frontier score, because the frontier scores are island-conditioned. A classical pre-filter replaying the walk's failure predicate (as `dialog_gcd_classical_filter.rs` and the CUDA kernels do) would be the first step.

5. **Zero-Toffoli known-output receivers and the exact simplifier chain.** `add_known` and `known_stream` (Section 2.5) and the `product`, `affine`, `quadratic`, `truth` passes are generic and would apply to the package's reverse passes; the SAT-proven CCX→CX rows are stream-specific and not worth porting.

6. **Window compatibility is already there.** Because the package builds the windowed addition with coherent lookups (unlike the challenge interface), its numbers are the "full-attack" numbers directly; the conversion of Equation (2) is not needed for them. The identity row and the $R \in \{\pm A\}$ exceptions are handled as in Note F, with signed windows removing $\mathcal{O}$ from the tables altogether.

7. **Physical model.** `physical.py` is a surface-code model; Section 10 adds the trapped-ion (IonQ Q102), gross-code (IBM) and neutral-atom data points and explains why they are not modelled the same way.

The one-line comparison

On the challenge's own axis the package's best windowed addition is $1{,}457 \times 1{,}310{,}693 = 1.91\,\mathrm{B}$—above ECDSA.fail's $1.50\,\mathrm{B}$ (classical addend) and $1.96\,\mathrm{B}$ (windowed) and above IonQ's $2.03\,\mathrm{B}$—while being exact wherever it is not explicitly approximate, and verified end to end at toy sizes. The frontier's remaining advantage is almost entirely in the walk (Leapfrog's 139 fat ticks against 401 CondInv rounds) and in the fused replay cell; the frontier's disadvantage is that its best numbers are island-conditioned.

Part II
Fault tolerance: from logical to physical qubits

5 Why error correction, and what it costs

5.1 Noise, and the discretisation of errors

A circuit of N logical qubits and G gates run directly on physical hardware fails with probability $\approx Gp$, where p is the error per operation. The circuits of Part I have $G \approx 4 \times 10^7$ Toffolis—each itself several two-qubit gates—on $\approx$ 1,200 qubits, while the best physical two-qubit gates in 2026 sit at $p \sim 10^{-3}$ (superconducting) to 10^{-4} (trapped ions) [1, 97]. The gap of five to ten orders of magnitude is closed by quantum error correction, whose two founding facts are:

1. **Errors can be discretised.** Any single-qubit noise process is a linear combination of I, X, Y, Z (its Kraus operators expand in the Pauli basis), so a code that corrects X, Y and Z on any one qubit corrects *every* single-qubit error, and the measurement of the syndrome projects a continuous error onto one of the discrete Pauli cases. Formally, a set of errors $\{E_a\}$ is correctable by a code with projector Π iff the *Knill–Laflamme conditions* hold [65]:

$$\Pi\, E_a^\dagger E_b\, \Pi = c_{ab}\, \Pi \qquad \text{for all } a, b, \tag{12}$$

   i.e. no error maps a codeword onto another codeword, and different errors are either distinguishable or act identically on the code (*degeneracy*).

2. **Fault tolerance is possible below a threshold.** If every component fails with probability below some p_{th}, then arbitrarily long quantum computations can be carried out with arbitrarily small logical error at a cost that is polylogarithmic in the circuit size (the threshold theorem [2, 64]). The proof concatenates a distance-3 code with itself ℓ times: one level turns p into cp^2 (two faults are needed to fail), so ℓ levels give $p_\ell = (cp)^{2^\ell}/c$, which is $\le \varepsilon$ once $\ell = O(\log\log(1/\varepsilon))$, at a size overhead of $n^\ell = \text{polylog}(1/\varepsilon)$.

In practice the overhead is dominated not by the asymptotics but by the constants of the code family, so the right question is: *for a given physical error rate p and a target logical error ε per gate, how many physical qubits does a logical qubit cost, and how many does a logical gate cost?* The answer for the surface code is set by its threshold $p_{\text{th}} \approx 1\%$ and the empirical law

$$p_L(d) \approx A\left(\frac{p}{p_{\text{th}}}\right)^{(d+1)/2}, \qquad A \approx 0.1, \tag{13}$$

per code-distance-d patch per round of syndrome measurement [36, 37], so that each step $d \to d+2$ suppresses the logical error by $\Lambda = p_{\text{th}}/p$ (Google measured $\Lambda = 2.14 \pm 0.02$ on Willow at its physical noise [1]; Figure 22 shows $\Lambda \approx 10$ at $p = 10^{-3}$ for the circuit-level depolarising model simulated here). A distance-d patch costs $2d^2 - 1$ physical qubits (d^2 data, $d^2 - 1$ measurement), so a target of $\varepsilon = 10^{-15}$ per patch per round at $p = 10^{-3}$ needs $d \approx 25$ and $\approx$ 1,250 physical qubits per logical qubit before any routing space—the origin of the "$\sim 10^3$ physical qubits per logical qubit" rule of thumb and of the million-qubit estimates of Section 10.

5.2 The stabiliser formalism

Let $\mathcal{P}_n$ be the Pauli group on n qubits (tensor products of I, X, Y, Z with phases $\pm 1, \pm i$). A *stabiliser code* is defined by an abelian subgroup $\mathcal{S} = \langle S_1, \ldots, S_{n-k}\rangle \subset \mathcal{P}_n$ with $-I \notin \mathcal{S}$; its code space is the joint $+1$ eigenspace $\mathcal{C} = \{|\psi\rangle : S|\psi\rangle = |\psi\rangle\ \forall S \in \mathcal{S}\}$, of dimension 2^k when the $n-k$ generators are independent [23, 46]. Three facts carry the whole subject:

- **Detection.** For a Pauli error E, either E commutes with every generator or it anticommutes with some of them; measuring the generators returns the *syndrome* $s \in \{0,1\}^{n-k}$, $s_i = 1$ iff E anticommutes with S_i, without disturbing the encoded state, because the measurement of S_i projects onto its ± 1 eigenspaces and the code state is already an eigenstate.
- **Logical operators.** The *normaliser* $N(\mathcal{S})$ (Paulis commuting with all of $\mathcal{S}$) acts on $\mathcal{C}$; $N(\mathcal{S})/\mathcal{S}$ is the $2k$-generated group of logical Paulis $\bar{X}_j, \bar{Z}_j$. An undetectable error is an element of $N(\mathcal{S})$; it is harmless iff it lies in $\mathcal{S}$. The *distance* is $d = \min\{\mathrm{wt}(E) : E \in N(\mathcal{S}) \setminus \mathcal{S}\}$, and the code corrects all errors of weight $\le t = \lfloor (d-1)/2 \rfloor$ (Equation (12): for $\mathrm{wt}(E_a^\dagger E_b) < d$ the product is either in $\mathcal{S}$ or detectable).
- **Binary representation.** Writing a Pauli as a $2n$-bit vector $(x|z)$ (ignoring phase), products become sums mod 2 and commutation becomes the symplectic form $x \cdot z' + z \cdot x' \bmod 2$; the generators form an $(n-k) \times 2n$ *check matrix* $H = [H_X | H_Z]$ with $H_X H_Z^{\mathsf{T}} + H_Z H_X^{\mathsf{T}} = 0$, and the syndrome of $E = (x|z)$ is $s = H_X z + H_Z x$. Decoding, encoding and the bounds of Section 5.5 are all linear algebra over $\mathbb{F}_2$ on H.

A *CSS code* [22, 94] has generators that are purely X-type or purely Z-type, $H = \left[\begin{smallmatrix} H_X & 0 \\ 0 & H_Z \end{smallmatrix}\right]$ with $H_X H_Z^{\mathsf{T}} = 0$ (now H_X and H_Z denote the X- and Z-check matrices, $r_X \times n$ and $r_Z \times n$). The Z-type checks detect X errors through the classical code $\ker H_Z$ and the X-type checks detect Z errors through $\ker H_X$, independently, so a CSS code is two classical codes $C_X = \ker H_Z \supseteq \mathrm{row}(H_X)$ and $C_Z = \ker H_X \supseteq \mathrm{row}(H_Z)$ glued by the orthogonality condition; $k = n - \mathrm{rank}\, H_X - \mathrm{rank}\, H_Z$; and the transversal CNOT is automatically a logical CNOT (Section 8.1). Every code in this document except $[\![5,1,3]\!]$ is CSS.

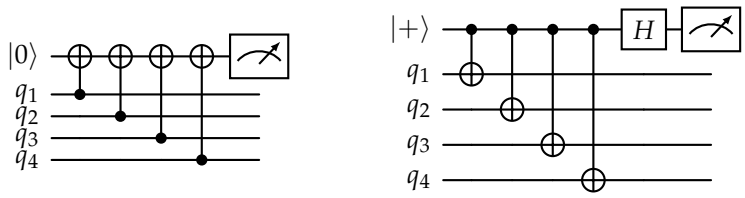


Figure 16: Measuring a weight-four Z-type check $Z_1Z_2Z_3Z_4$ (left) and X-type check $X_1X_2X_3X_4$ (right) with one ancilla. The ancilla accumulates the parity; the data qubits are untouched if nothing fails. A single ancilla fault in the middle of the left circuit can propagate to two data qubits (a *hook* error), which is why the CNOT order matters for the surface code (Section 6.2) and why cat-state or flag constructions are needed for high-weight checks (Section 8.2).

5.3 Encoding

There are two ways to prepare a logical state, and real machines use the second.

Unitary encoders (Gottesman's standard form). Bring the check matrix, by row operations over $\mathbb{F}_2$ and qubit permutations, to the standard form

$$H = \left[\begin{array}{ccc|ccc} I_r & A_1 & A_2 & B & 0 & C \\ 0 & 0 & 0 & D & I_{n-k-r} & E \end{array}\right], \qquad \bar{X} = \left[\begin{array}{ccc|ccc} 0 & E^{\mathsf{T}} & I_k & C^{\mathsf{T}} & 0 & 0 \end{array}\right], \tag{14}$$

where $r = \mathrm{rank}\, H_X$ [46]. Then (Algorithm 1) place the k data qubits last and the $n-k$ ancillas in $|0\rangle$; for each logical $\bar{X}_j$ apply its X-part to the ancillas controlled on data qubit j (the Z-part acts trivially on $|0\rangle$); then for $i = 1, \dots, r$ apply H to ancilla i followed by generator i's Pauli on the other qubits controlled on ancilla i, with an S on ancilla i when the generator has Y there. Each such block applies $(I + S_i)/\sqrt{2}$ to a state in which qubit i is $|0\rangle$, and since the projectors commute the result is $\prod_i \frac{I+S_i}{\sqrt{2}}\, \bar{X}^c |0 \cdots 0\rangle$, the encoded $|c_L\rangle$. The three small encoders of Section 6.1

were generated or checked this way with Stim (Appendix B). The cost is $O(n(n-k))$ controlled-Paulis—fine for $n = 7$, irrelevant for $n = 10^3$.

Algorithm 1 Standard-form encoder for an $[\![n, k, d]\!]$ stabiliser code (verified in `scripts/standard_form_encoder.py`)

Require: check matrix H in the standard form (14) after a qubit permutation π; data qubits $n-k, \ldots, n-1$

1: **for** $j = 0, \ldots, k-1$ **do** ▷ conditional logical X
2: **for** each ancilla q with $(E^{\mathsf{T}})_{q,j} = 1$ **do** apply $\mathrm{CX}(n-k+j \to q)$
3: **for** $i = 0, \ldots, r-1$ **do** ▷ project onto $S_i = +1$
4: apply H on qubit i
5: **for** each qubit $q \neq i$ **do** apply CX, CZ or CY from i to q according to the (x, z) entries of S_i at q
6: **if** S_i has Y on qubit i **then** apply S on qubit i
7: **return** the permutation π that maps circuit qubits back to code qubits

Measurement-based encoding. Prepare every data qubit in $|0\rangle$ (which is already a $+1$ eigenstate of all Z-type checks and of $\bar{Z}$), measure all X-type checks, and record the outcomes: the state is now a stabiliser state of the code with some X-checks at -1, which is just a known Pauli frame, or is corrected by a Pauli. This prepares $|0_L\rangle$ in one round of syndrome measurement; $|+_L\rangle$ is the same with $|+\rangle$ and the Z-checks. A non-stabiliser state ($|T_L\rangle$) cannot be prepared this way and needs injection (Section 9.2). This is how surface-code, colour-code and bicycle-code logical qubits are initialised in every architecture below; the "encoder" is the error-correction cycle itself.

5.4 Decoding, in general

Given the syndrome s the decoder must choose a correction $\hat{E}$ with $\mathrm{syn}(\hat{E}) = s$; the correction succeeds iff $\hat{E}E \in \mathcal{S}$. The best possible decoder is *maximum-likelihood* (MLD): choose the coset $\hat{E}\mathcal{S}$ of maximal total probability—degenerate decoding, since all errors in a coset are equivalent. MLD is #P-hard in general, so practical decoders approximate it:

- **Lookup tables** for small codes: a table from each of the 2^{n-k} syndromes to a minimum-weight error (Section 6.1).
- **Minimum-weight decoding**: find the lowest-weight $\hat{E}$ with syndrome s (ignoring degeneracy). For codes whose errors create *pairs* of syndrome defects (surface and toric codes, repetition codes) this is a minimum-weight perfect matching problem on a graph, solvable in polynomial time (Section 6.2.3); union-find decoders give an almost-linear-time approximation (Section 6.2.3).
- **Belief propagation** on the Tanner graph of H, which is approximate for quantum codes because of degeneracy and short cycles, repaired by ordered-statistics post-processing (BP+OSD) or by the memory/relay heuristics of Relay-BP (Section 7.2.2).
- **Learned decoders** (AlphaQubit) trained on real syndrome data [7].

Real decoding is *circuit-level*: measurements are themselves noisy, so the syndrome is measured repeatedly ($\approx d$ rounds), the decoder works on the differences between consecutive rounds (*detectors*), and a fault anywhere in the extraction circuit—a data error, a measurement error, an ancilla error that hooks onto two data qubits—is one of the "error mechanisms" of a *detector error model* that lists, for each possible fault, which detectors it flips and which logical observables it toggles. Stim builds this model from the circuit automatically, and that is what the matching decoder of Figure 22 decodes [39, 55]. The decoder must also be *fast*: a 1 μs surface-code cycle produces $\approx d^2$

bits per patch per microsecond, and the reaction time (decoding latency plus control) multiplies the critical path of every non-Clifford gate (Section 10); Google's real-time decoder ran at 63 μs average latency for a distance-5 memory over a million cycles [1], and IBM's Relay-BP is designed to be implemented on FPGAs/ASICs [79].

5.5 The bounds, and a map of the codes

For stabiliser (and, mostly, all) codes with rate $R = k/n$ and relative distance $\delta = d/n$, the classical bounds have quantum analogues [4, 23, 34, 46, 84]:

$$\text{Singleton (Knill–Laflamme):}\quad n - k \;\geq\; 2(d-1) \qquad \Rightarrow\; R \leq 1 - 2\delta; \tag{15}$$

$$\text{Hamming (non-degenerate):}\quad \sum_{j=0}^{t} 3^j \binom{n}{j} 2^k \;\leq\; 2^n \qquad \Rightarrow\; R \leq 1 - \tfrac{\delta}{2}\log_2 3 - h\big(\tfrac{\delta}{2}\big); \tag{16}$$

$$\text{Gilbert–Varshamov (existence):}\quad \sum_{j=1}^{d-1} 3^j \binom{n}{j} \;<\; 2^{n-k} \text{ suffices} \qquad \Rightarrow\; R \geq 1 - \delta\log_2 3 - h(\delta), \tag{17}$$

with h the binary entropy. The Hamming bound counts the $3^j\binom{n}{j}$ Pauli errors of weight j that a non-degenerate code must map to distinct syndromes—degenerate codes can beat it slightly, and the surface code is highly degenerate—while the Gilbert–Varshamov bound shows that random stabiliser codes achieve rate $1 - \delta\log_2 3 - h(\delta) > 0$ for $\delta < 0.189$: *good* codes exist, but random ones have dense checks. Rains' shadow enumerators and the Ashikhmin–Litsyn linear-programming bounds tighten the upper bound below Hamming for stabiliser codes [4, 84]. For codes with *geometrically local* checks in two dimensions there is the far stronger Bravyi–Poulin–Terhal bound

$$k\,d^2 \;\leq\; c\,n \qquad \text{(2D local stabiliser codes)}, \tag{18}$$

so the surface code's $k = 1$, $d = \Theta(\sqrt{n})$ is optimal for a planar layout, and any code with $k = \Theta(n)$ or $d \gg \sqrt{n}$ must have non-local checks [16, 17]—which is exactly what the hypergraph-product and bivariate-bicycle codes of Section 7.2 buy with long-range couplers, and what the asymptotically good LDPC codes of Panteleev–Kalachev ($k, d = \Theta(n)$ with constant-weight checks) show is possible in principle [69, 82]. Figure 17 places the codes of this document on the (δ, R) plane; Table 7 summarises what each family offers.

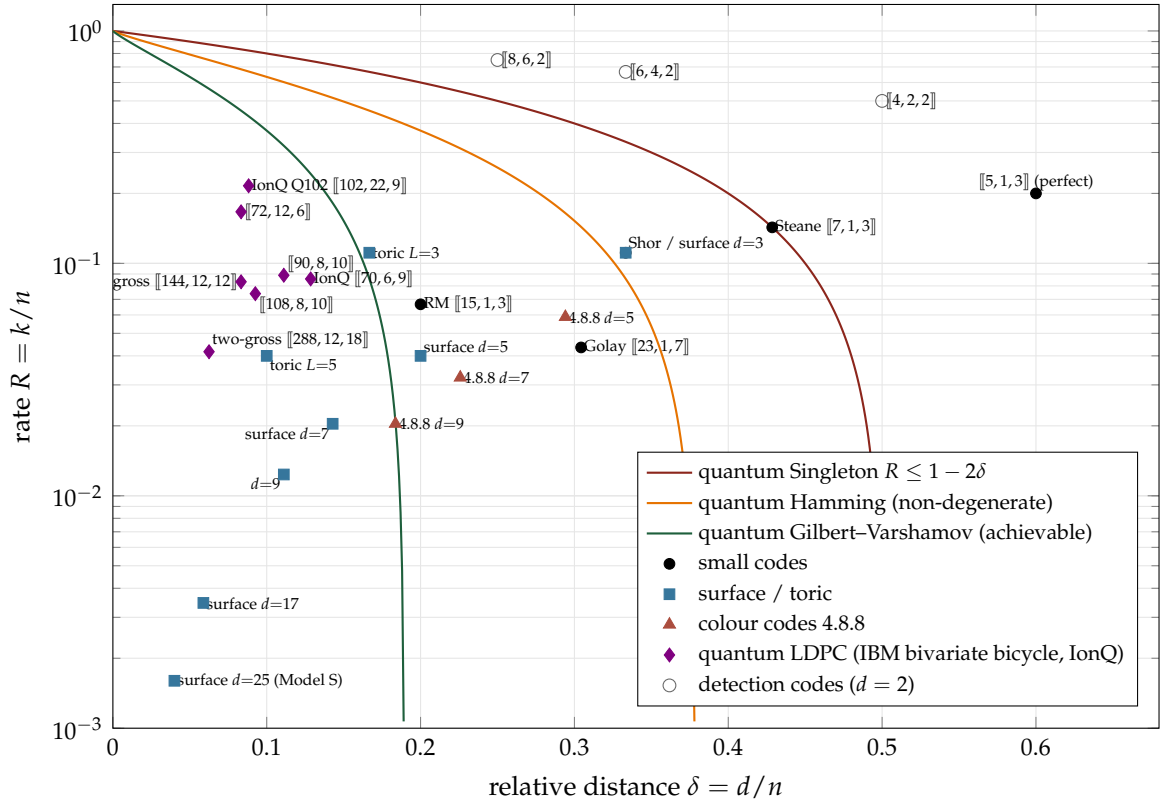


Figure 17: Rate against relative distance (logarithmic rate axis) for the codes discussed in Part II, with the three bounds (15)–(17) computed for this document (the Hamming and Gilbert–Varshamov curves stop where they reach zero). Finite codes sit far below the asymptotic curves; what the plot shows is the trade each family makes. The surface code slides toward $R \to 0$ as d grows ($d = 25$, the distance used in Section 10, is at $\delta = 0.04$, $R = 0.0016$); colour codes are somewhat denser at equal distance; the bivariate bicycle codes sit an order of magnitude higher in rate at comparable δ (the gross code at $\delta = R = 1/12$), and IonQ's $[\![102, 22, 9]\!]$ higher still; the $[\![5, 1, 3]\!]$ code is perfect—it meets the Hamming bound with equality, $1 + 3 \cdot 5 = 2^4$.

family	$[\![n,k,d]\!]$	locality	threshold (circuit-level)	native logical gates	decoder
Shor, Steane, $[\![5,1,3]\!]$, RM	$9,7,5,15$ / 1 / 3	none needed	(concatenated) $\sim 10^{-4}$–10^{-3}	Steane: transversal Clifford; RM_{15}: transversal T	lookup table
surface / toric	d^2 / 1 / d; $2d^2$ / 2 / d	2D, weight 4	$\approx$ 0.5–1% [37, 85]	lattice surgery CNOT, Pauli product measurements; T by injection	MWPM, union-find, neural
2D colour code (4.8.8, 6.6.6)	$\frac{d^2+2d-1}{2}$ / 1 / d	2D, weight 4–8	$\approx$ 0.2–0.5%	transversal H, S,CNOT	matching on restricted lattices, BP
3D colour code	e.g. $[\![15,1,3]\!]$	3D	lower	transversal T (gauge fixing)	
bivariate bicycle (IBM)	$[\![144,12,12]\!]$, $[\![288,12,18]\!]$	degree 6, two planar layers	0.8% (pseudo-threshold 0.65–0.7%) [18]	automorphisms; generalised surgery via LPU; T injected	BP+OSD, Relay-BP
three-ring QLDPC (IonQ)	$[\![70,6,9]\!]$, $[\![102,22,9]\!]$	non-local (ion transport)	–	cat-state measurements, Clifford frame	[97]
hypergraph product	$\Theta(n)$ / $\Theta(\sqrt{n})$	non-local	$\sim$	surgery	BP+OSD
good QLDPC (PK, LZ)	$\Theta(n)$ / $\Theta(n)$	expander-like	–	(theory)	linear-time decoders exist
repetition cat (Alice & Bob)	$[\![d,1,d]\!]$ on cat qubits	1D	bias-tailored	lattice surgery, injection	matching
yoked surface codes	concatenated	2D	–	storage only	

Table 7: The code families used in the architectures of Section 10. Thresholds depend strongly on the noise model and decoder; the quoted ranges are indicative.

6 The codes: small codes, surface and toric codes

This section and the next go through the code families of Table 7 one by one: what the stabilisers are, how a logical state is encoded, how the syndrome is decoded, and (in Sections 8 and 9) which logical gates the structure supports. The small codes come first because everything about them can be written down completely; the surface code because it is the baseline of every physical estimate in Section 10; colour and quantum LDPC codes because they are what IBM, IonQ and the neutral-atom proposals actually plan to build.

6.1 Small codes

6.1.1 Shor's $[\![9,1,3]\!]$ code

Shor's code [91] is the concatenation of the three-qubit phase-flip code $|\pm\rangle \mapsto |\pm\pm\pm\rangle$ with the three-qubit bit-flip code $|0\rangle \mapsto |000\rangle$, $|1\rangle \mapsto |111\rangle$:

$$|0_L\rangle = \tfrac{1}{2\sqrt{2}}(|000\rangle + |111\rangle)^{\otimes 3}, \qquad |1_L\rangle = \tfrac{1}{2\sqrt{2}}(|000\rangle - |111\rangle)^{\otimes 3}.$$

Its eight generators are the six weight-two Z-checks $Z_1Z_2, Z_2Z_3, Z_4Z_5, Z_5Z_6, Z_7Z_8, Z_8Z_9$ (bit flips inside each block) and the two weight-six X-checks $X_1\cdots X_6$, $X_4\cdots X_9$ (relative phases between blocks). The logical operators are $\bar{Z} = X_1X_2X_3$ ($\equiv X^{\otimes 9}$) and $\bar{X} = Z_1Z_4Z_7$ ($\equiv Z^{\otimes 9}$)—note the swap of letters, a consequence of the outer code being a phase-flip code. The encoder (Figure 18, left) is the two concatenation steps written out: two CNOTs spread the data qubit over the three block leaders, Hadamards turn the phase-flip code into the bit-flip basis, and six CNOTs fill each block. Decoding is a lookup (Table 8): the two Z-checks of a block locate an X error on one of

its three qubits, and the two X-checks locate the block carrying a Z error—but not which qubit, because Z_1, Z_2 and Z_3 differ by stabilisers ($Z_1Z_2 \in \mathcal{S}$) and act identically on the code. This is *degeneracy*: three different errors, one syndrome, one correction that works for all three. A Y error is corrected as an X and a Z error simultaneously.

qubit j	1	2	3	4	5	6	7	8	9
X_j (Z-checks, by block)	10 00 00	11 00 00	01 00 00	00 10 00	00 11 00	00 01 00	00 00 10	00 00 11	00 00 01
Z_j ($X_{1..6}$, $X_{4..9}$)	10	10	10	11	11	11	01	01	01

Table 8: Shor-code lookup table (computed, `scripts/syndrome_tables.py`). Each X error has its own syndrome; the three Z errors of a block share one.

6.1.2 Steane's $[\![7,1,3]\!]$ code

The Steane code [94] is the CSS code built from the classical $[7,4,3]$ Hamming code C and its dual: $H_X = H_Z = H_{\text{Ham}}$, the 3×7 matrix whose j-th column is the binary representation of j,

$$H_{\text{Ham}} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}, \qquad |0_L\rangle = \tfrac{1}{\sqrt{8}} \sum_{c \in C^\perp} |c\rangle, \qquad |1_L\rangle = \bar{X}|0_L\rangle,$$

so $k = 7 - 3 - 3 = 1$ and $d = 3$ because the Hamming code has distance 3 (and so does its dual, the simplex code, with distance 4). Three X-type and three Z-type weight-four checks; $\bar{X} = X^{\otimes 7} \equiv X_1X_2X_3$ and $\bar{Z} = Z^{\otimes 7} \equiv Z_1Z_4Z_5$. The encoder in Figure 18 (centre) is the measurement-free version of the standard-form construction: the data qubit's X is first spread to $X_1X_2X_3$ by two CNOTs, then three ancillas in $|+\rangle$ apply the three X-generators by controlled-X fans. Decoding is the Hamming decoder twice (Table 9): the three Z-check outcomes read as a binary number are the index of the flipped qubit, and the three X-check outcomes the index of the phase-flipped qubit. The Steane code is the smallest 2D colour code (Section 7.1); its transversal Clifford group is why it appears in every trapped-ion and neutral-atom logical demonstration [9, 88].

qubit j	1	2	3	4	5	6	7
(s_4, s_2, s_1) of X_j (from Z-checks) = of Z_j (from X-checks)	001	010	011	100	101	110	111

Table 9: Steane-code lookup table (computed): the syndrome is the binary index of the faulty qubit, for X and Z errors alike.

6.1.3 The perfect $[\![5,1,3]\!]$ code

The five-qubit code [8, 68] is the smallest code correcting an arbitrary single-qubit error: its 16 syndromes exactly cover the identity and the 15 single-qubit Paulis (Table 10), which is the quantum Hamming bound with equality. Its stabiliser group is generated by $XZZXI$ and its cyclic shifts $IXZZX$, $XIXZZ$, $ZXIXZ$ (the fifth shift is their product), and $\bar{X} = X^{\otimes 5}$, $\bar{Z} = Z^{\otimes 5}$. It is not CSS, so it has no transversal CNOT and its encoder needs controlled-Y and S gates; Figure 18 (right) is the standard-form circuit of Algorithm 1, verified with Stim to map $X_{\text{data}} \mapsto +XXXXX$ and $Z_{\text{data}} \mapsto +ZZZZZ$ (the leading Z on the data qubit fixes the sign of $\bar{X}$; two controlled-Z gates from the data qubit to ancillas in $|0\rangle$ that the generic algorithm emits are omitted because they act trivially).

$s_1s_2s_3s_4$	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
error	X_1	Z_3	X_5	Z_5	Z_2	X_4	Y_5	X_2	Z_4	Z_1	Y_1	X_3	Y_2	Y_3	Y_4

Table 10: $[\![5,1,3]\!]$ lookup table (computed) for generators $S_1 = XZZXI$, $S_2 = IXZZX$, $S_3 = XIXZZ$, $S_4 = ZXIXZ$: every one of the 15 single-qubit errors has a distinct non-zero syndrome.

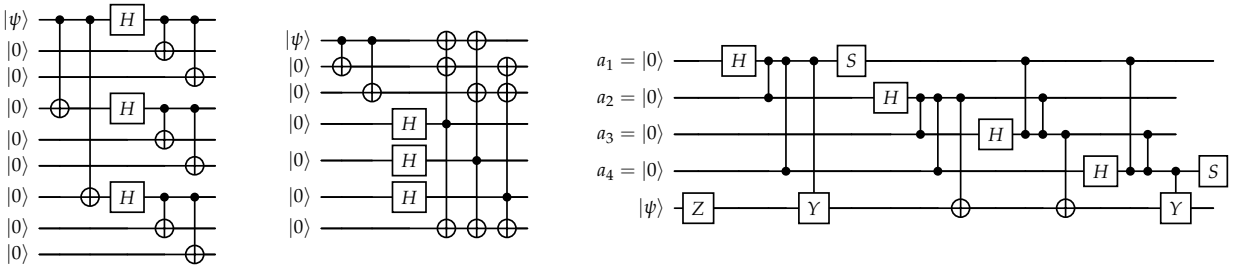


Figure 18: Encoders verified with Stim (`scripts/verify_codes.py`, `scripts/standard_form_encoder.py`). Left: Shor's code, the two concatenation steps. Centre: Steane's code, data qubit on the top wire; the three $|+\rangle$ ancillas apply the X-type generators $X_{1,2,4,7}$, $X_{1,3,5,7}$, $X_{2,3,6,7}$ (in the figure's wire order the supports are $\{1,2,4,7\}$, $\{1,3,5,7\}$, $\{2,3,6,7\}$). Right: the $[\![5,1,3]\!]$ standard-form encoder; the vertical lines ending in dots are controlled-Z gates, the boxed Y is a controlled-Y, and each S supplies the phase for a generator with Y on its pivot ($YZIZY$ and $ZIZYY$, which together with $IXZZX$ and $ZZXIX$ generate the same group as the cyclic generators).

6.1.4 The $[\![15,1,3]\!]$ Reed–Muller code and the $[\![4,2,2]\!]$ detection code

Index the 15 qubits by the non-zero 4-bit strings $v \in \mathbb{F}_2^4 \setminus \{0\}$. The X-stabilisers are the four weight-8 operators X on $\{v : v_b = 1\}$, $b = 1..4$; the Z-stabilisers are those four plus the six weight-4 operators Z on $\{v : v_b = v_{b'} = 1\}$; $\bar{X} = X^{\otimes 15}$, $\bar{Z} = Z^{\otimes 15} \equiv$ a weight-three Z (so $d = 3$, while the X-distance is 7). The CSS structure was checked and a nine-instruction standard-form encoder verified (Appendix B). What makes this code important is that every X-stabiliser has weight $\equiv 0 \pmod 8$ (the classical code is *triply even*) and $\bar{X}$ has weight $15 \equiv 7 \pmod 8$: writing $|0_L\rangle \propto \sum_{s \in \mathcal{S}_X} |s\rangle$ and $|1_L\rangle \propto \sum_s |s \oplus 1^{15}\rangle$, the transversal gate $T^{\otimes 15}$ multiplies a basis state $|x\rangle$ by $e^{i\pi|x|/4}$, which is 1 on every term of $|0_L\rangle$ and $e^{-i\pi/4}$ on every term of $|1_L\rangle$: $T^{\otimes 15} = \bar{T}^\dagger$. This is the engine of magic-state distillation (Section 9.2) and of code switching (Section 9.3); the code is also the smallest 3D colour code.

The $[\![4,2,2]\!]$ code, $\mathcal{S} = \langle XXXX, ZZZZ\rangle$, $\bar{X}_1 = X_1X_2$, $\bar{Z}_1 = Z_1Z_3$, $\bar{X}_2 = X_1X_3$, $\bar{Z}_2 = Z_1Z_2$, detects any single error (every weight-one Pauli anticommutes with one of the two checks) but cannot locate it; it is the smallest member of the $[\![2m, 2m-2, 2]\!]$ "iceberg" family used in trapped-ion and neutral-atom demonstrations where post-selection on a trivial syndrome is affordable [9].

6.2 The toric and surface codes

6.2.1 Kitaev's toric code

Place qubits on the $2L^2$ edges of an $L \times L$ square lattice with periodic boundary conditions [30, 63]. For each vertex v and each face (plaquette) p define

$$A_v = \prod_{e \ni v} X_e \quad \text{(weight 4)}, \qquad B_p = \prod_{e \in \partial p} Z_e \quad \text{(weight 4)}.$$

A vertex and a plaquette share either 0 or 2 edges, so all generators commute. Since $\prod_v A_v = \prod_p B_p = I$, there are $2(L^2-1)$ independent generators and $k = 2L^2 - 2(L^2-1) = 2$ logical qubits. A product of Z's along a closed loop of edges commutes with every A_v; if the loop is contractible it is a product of B_p's (a stabiliser), if it winds around the torus it is a logical operator. The two non-contractible cycles give $\bar{Z}_1, \bar{Z}_2$; $\bar{X}_1, \bar{X}_2$ are X-strings along non-contractible cycles of the *dual* lattice (crossing edges). The shortest non-contractible loop has L edges, so $d = L$: the toric code is $[\![2L^2, 2, L]\!]$. Its decoding problem is the prototype for everything below: a Z-error on a set of edges is a chain; A_v is violated exactly at the *endpoints* of the chain (an interior vertex meets the chain

twice); so the syndrome is a set of defects, always even in number, and the decoder's job is to pair them by a correction chain such that error+correction is a contractible loop—a stabiliser—rather than a non-contractible one.

6.2.2 The planar (rotated) surface code

Cut the torus open and the two logical qubits become one, held by a patch with two kinds of boundary [19, 37]. In the *rotated* layout of Figure 19, d^2 data qubits sit on a $d \times d$ grid, and the $d^2 - 1$ checks are the faces of the grid, coloured in a checkerboard: X-type faces measure X on their (four, or two at the boundary) corners, Z-type faces measure Z. Two opposite boundaries carry weight-two X checks, the other two carry weight-two Z checks, and a logical operator is a string that runs between the two boundaries of its own type: $\bar{Z}$ is a row of Z's between the Z boundaries (each bulk X face it crosses is touched at two corners, so it commutes with everything), $\bar{X}$ a column of X's between the X boundaries. The shortest such string has d qubits; deforming a string by multiplying with a face operator gives an equivalent representative, so each logical operator has many representatives, and this high degeneracy is also why the code decodes so well. One patch is $[\![d^2, 1, d]\!]$ on $2d^2 - 1$ physical qubits once the $d^2 - 1$ measurement ancillas are counted.

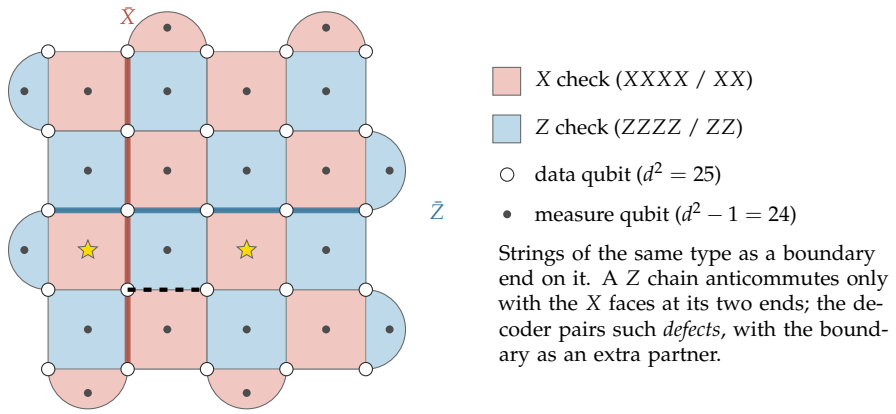


Figure 19: The rotated surface code at $d = 5$. The dashed chain is a Z error on two neighbouring qubits of the fourth row; the two stars are the only X faces it anticommutes with (the X face between them is touched twice)—the *defects* the decoder sees. The logical operators drawn are the minimum-weight representatives; multiplying $\bar{Z}$ by any Z face slides it.

Encoding. A logical $|0_L\rangle$ is prepared by initialising all data qubits in $|0\rangle$ and running one cycle of syndrome measurement: the Z faces are already satisfied, the X faces return random outcomes that are simply recorded as the reference frame (or fixed by Z corrections), and from then on the code is live. $|+_L\rangle$ is the same with $|+\rangle$ and the X faces. Reading out $\bar{Z}$ is the time-reversed operation: measure every data qubit in the Z basis, recompute the Z faces from the outcomes to get one last syndrome round (so that the readout itself is error-corrected), and take the parity along any $\bar{Z}$ string.

Syndrome extraction. Each face has one measurement qubit. One cycle (Figure 20) is: reset the ancillas; Hadamard the X-face ancillas; four layers of CNOTs in which every face talks to its four corners in a fixed order; Hadamard; measure. The order matters because of *hook errors*: an ancilla fault between the second and third CNOT propagates to the last two data qubits of the order, and if that pair were aligned with the logical operator the pair would count as a single fault that advances a chain by two, halving the effective distance. The standard orders—"Z" shape (NW, NE, SW, SE) for the X faces and "N" shape (NW, SW, NE, SE) for the Z faces—make every hook pair perpendicular to the logical string it could have helped, and they also let all faces run their

CNOTs simultaneously, because at each layer every data qubit is touched by exactly one face. The whole cycle is 8 layers; Google's Willow runs it in 1.1 μs [1], which is the "1 μs cycle" of every superconducting estimate in Section 10.

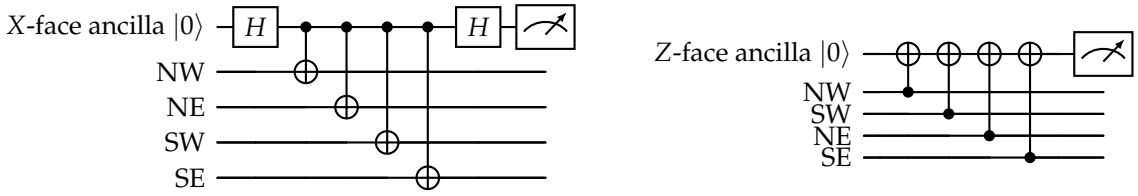


Figure 20: One surface-code cycle for an X face (left, order NW–NE–SW–SE) and a Z face (right, order NW–SW–NE–SE). The four CNOT layers of all faces run in parallel; the hook pairs (last two targets) are horizontal for X faces and vertical for Z faces, perpendicular to $\bar{X}$ and $\bar{Z}$ respectively in Figure 19.

6.2.3 Decoding the surface code

Because measurements are noisy, one round is never enough: the cycle is repeated, and the decoder works on *detectors*, the XOR of consecutive outcomes of the same face [30]. In the resulting three-dimensional space-time picture (Figure 21), every elementary fault flips at most two detectors: a data error between rounds flips the two faces beside it (a space-like edge), a measurement error flips the same face in two consecutive rounds (a time-like edge), a CNOT fault or hook flips a diagonal pair (a space-time edge), and a fault next to a boundary flips a single detector (an edge to the boundary node). The *detector error model* is the list of these edges with their probabilities; Stim derives it from the circuit, and Figure 22 was produced that way [39]. Decoding then means choosing a set of edges that explains the observed defects and is most probable:

Algorithm 2 Minimum-weight perfect matching (MWPM) decoding of a surface-code memory

Require: detector error model $G = (V, E)$ with edge probabilities p_e and logical-flip labels; observed defect set $D \subseteq V$ (odd $|D|$ is made even by a boundary node)

1: weight every edge $w_e = \log \frac{1-p_e}{p_e}$ (so that a path's weight is $-\log$ of its probability up to a constant)
2: for every pair of defects, the lightest path between them is the shortest path in (G, w) (Dijkstra); build the complete graph on D with these distances, plus a boundary partner per defect
3: find a *perfect matching* of minimum total weight (Edmonds' blossom algorithm; sparse blossom explores the paths lazily and runs in near-linear time at low p [55])
4: the correction is the XOR of the matched paths' logical labels: the decoder never touches the qubits, it flips the Pauli frame

MWPM finds the single most likely error, not the most likely *class* of errors; it ignores degeneracy and the correlations between X and Z decoding caused by Y errors, and each of those costs a little threshold. Fowler showed that matching can be made $O(1)$ average time per round with parallel hardware [35]; PyMatching's sparse blossom decodes about a million detection events per second per core [55]; Google's real-time decoder kept up with the 1.1 μs cycle of a distance-5 memory for a million cycles with 63 μs average latency [1], and *latency*, not throughput, is what sets the reaction time that multiplies every non-Clifford gate in Section 10.

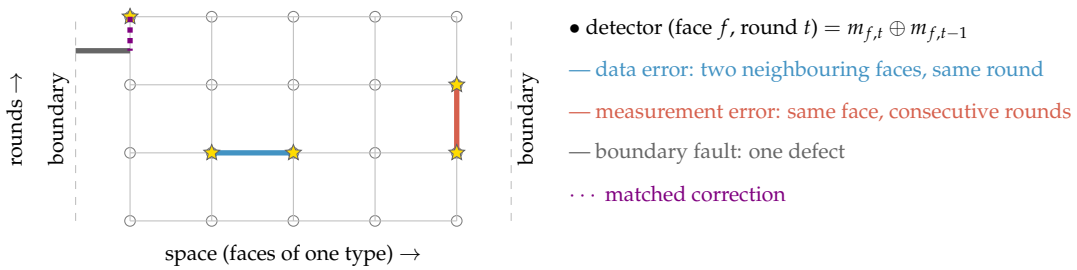


Figure 21: The decoding graph of one check type over four rounds: every fault is an edge, every defect a vertex, and MWPM pairs the defects by lightest edges. A chain of faults that crosses the whole patch (or a matched correction that does) flips the logical operator.

Union-find. Delfosse and Nickerson's decoder [29] trades a little accuracy for almost-linear worst-case time ($O(n\,\alpha(n))$, α the inverse Ackermann function) and a structure that maps to simple hardware:

Algorithm 3 Union-find decoder

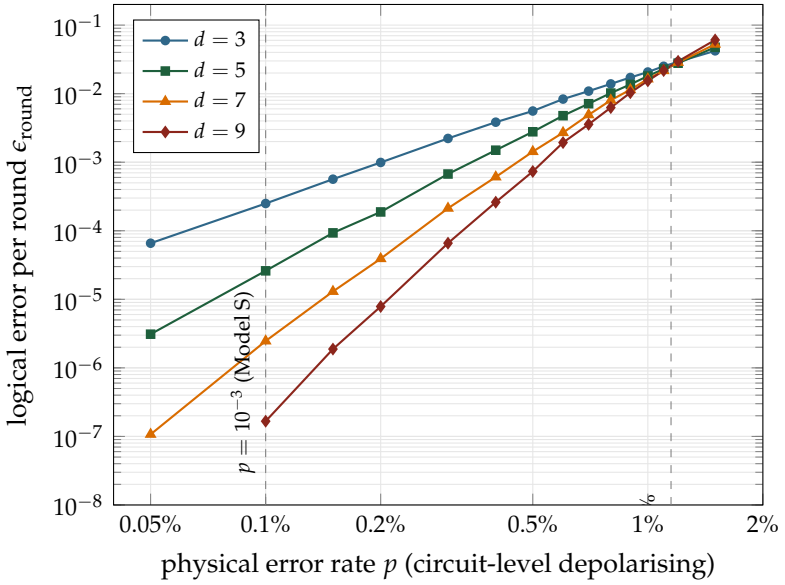
1: start one cluster per defect; a cluster is *odd* if it contains an odd number of defects and does not touch the boundary
2: **while** some cluster is odd **do**
3: grow every odd cluster by half an edge in all directions (*syndrome validation*)
4: when two clusters touch, merge them (union-find with path compression keeps this near constant time); a merged cluster's parity is the sum
5: inside each cluster, build a spanning forest and *peel* it from the leaves: remove a leaf edge, flip it in the correction iff the leaf vertex is an unmatched defect, and toggle the parity of the other endpoint

Its threshold is a few tenths of a percent below MWPM's under the same noise, which is why it is used where decoding hardware is the constraint and why many real-time decoders start from it.

Learned decoders. AlphaQubit [7] (Google DeepMind and Google Quantum AI) is a recurrent transformer trained first on simulated data and then fine-tuned on Sycamore syndrome records; it beat correlated matching on the experimental distance-3 and -5 data by using the soft measurement information and the device's real error correlations, at the price of a latency that is not yet compatible with real-time use.

6.2.4 The threshold, measured

Figure 22 is a circuit-level simulation performed for this document with Stim and PyMatching (`surface_threshold.py`, Appendix B): rotated memory-Z patches of distance 3–9, d rounds each, uniform depolarising noise of strength p after every Clifford, before every round on the data, and on every measurement and reset; at least 1,000 logical failures or 12 million shots per point; the logical error per shot ϵ converted to an error per round by $\epsilon_{\text{round}} = \frac{1}{2}(1-(1-2\epsilon)^{1/d})$. The curves cross at $p \approx 1.1$–1.2%, the circuit-level threshold under this noise model; below it, each $+2$ of distance divides the error by Λ, which the data put at $\Lambda \approx 9.6$, 10.6 and 14.8 for the steps $3 \to 5$, $5 \to 7$ and $7 \to 9$ at $p = 10^{-3}$, ≈ 5 at $p = 2 \times 10^{-3}$ and ≈ 3.2 at $p = 3 \times 10^{-3}$. Google's Willow reached $\Lambda = 2.14 \pm 0.02$ with a $d = 7$ logical error of 0.143% per cycle on 101 qubits, which in this uniform model corresponds to $p \approx 0.45\%$ (real noise is far from uniform: Willow's two-qubit gates are better than that, its measurements worse); the $\Lambda \approx 10$ of the $p = 10^{-3}$ estimates is a factor four or five in physical error away from that device [1].



p	$\Lambda_{3\to5}$	$\Lambda_{5\to7}$	$\Lambda_{7\to9}$
10^{-3}	9.6	10.6	14.8
2×10^{-3}	5.3	4.8	5.0
3×10^{-3}	3.3	3.2	3.2
5×10^{-3}	2.0	1.9	1.9

$\epsilon_{\text{round}}(d{=}9, p{=}10^{-3}) = 1.7\times10^{-7}$ (18 failures)
extrapolated to $d = 25$ at $\Lambda \approx 10$:
$\approx 10^{-15}$

Figure 22: Surface-code threshold computed for this document (Appendix B). Left: logical error per round against physical error for four distances; the crossing is the threshold. Right: the suppression factor Λ between successive distances, read off the same data; the extrapolation to $d = 25$ uses $\Lambda \approx 10$ per step, consistent with Equation (13) at $p_{\text{th}} \approx 1.2\%$. Points with fewer than 200 failures ($d = 7, 9$ at $p \le 10^{-3}$) carry $\pm$10–25% statistical error; the $d = 9$ point at $p = 0.05\%$ had no failure in 12 million shots and is not drawn.

Lattice surgery and logical gates. The surface code has no transversal gate beyond the Paulis, so its logical CNOTs, Hadamards and measurements are made by merging and splitting patches—*lattice surgery*—and its T gates by consuming magic states; both are in Sections 8 and 9.

7 The codes: colour codes and quantum LDPC codes

7.1 Colour codes

7.1.1 Two-dimensional colour codes

Take a trivalent lattice whose faces can be three-coloured so that no two faces of the same colour share an edge (the 4.8.8 tiling of squares and octagons, or the 6.6.6 honeycomb), put a qubit on every vertex, and attach to every face *both* an X-type and a Z-type check on its vertices [11]. Two faces share 0 or 2 vertices (trivalence plus three-colourability), so all checks commute, and the code is CSS with $H_X = H_Z$: it is *self-dual*. A triangular patch with one boundary of each colour encodes one qubit; the 4.8.8 family has $n = \frac{d^2+2d-1}{2}$ $(7, 17, 31, 49, \dots)$ and the 6.6.6 family $n = \frac{3d^2+1}{4}$ $(7, 19, 37, \dots)$; both start at the Steane code (Figure 23). Because $H_X = H_Z$, transversal Hadamard $H^{\otimes n}$ swaps the two check families and so preserves the code, implementing $\bar{H}$; and because every face has a number of vertices divisible by four—every stabiliser weight is doubly even—the transversal $S^{\otimes n}$ (or $S^{\dagger\otimes n}$) is a logical S [66]. With the transversal CNOT of any CSS code this is the whole Clifford group, transversally, in two dimensions: the colour code is the natural home of the Clifford frame (Section 8).

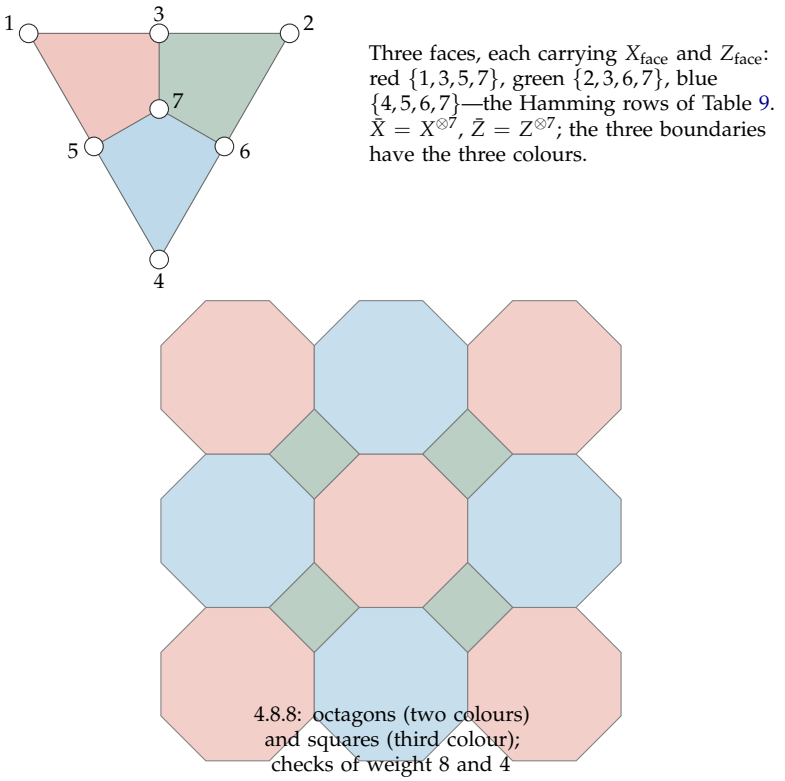


Figure 23: Left: the $[\![7,1,3]\!]$ code as the smallest triangular colour code. Right: a fragment of the 4.8.8 lattice from which the $[\![17,1,5]\!]$, $[\![31,1,7]\!]$, ... codes of Figure 17 are cut.

The price is paid in decoding and in threshold. A single error flips *three* checks (one of each colour), so the decoding graph is a hypergraph and matching does not apply directly. Delfosse's *projection* (restriction) decoder [28] restricts the syndrome to the two colours $\{c_1, c_2\}$—where every error flips exactly two of the remaining checks—runs MWPM on that surface-code-like subproblem, repeats for the other pairs, and lifts the three matchings to a colour-code correction; the *Möbius* decoder and Google's open-source Chromobius refine the lift [43]. Syndrome extraction is also heavier: a weight-eight octagon needs eight CNOTs and the hook problem of Figure 20 is harder to tame, which is what the "superdense" and "middle-out" cycles of Gidney and Jones address. The result is that at $p = 0.1\%$ a colour code needs about 1,250 physical qubits for a 10^{-12}-per-round logical qubit (the *teraquop footprint*) where the surface code with correlated matching needs 650 [43]; Google's 2024 experiment measured $\Lambda_{3/5} = 1.56(4)$ on a colour code, transversal Clifford gates adding only $0.0027(3)$ error per operation, magic-state injection above 99% with 75% post-selection yield, and lattice surgery between $d = 3$ patches [67]. Honeywell/Quantinuum's $[\![7,1,3]\!]$ demonstrations—real-time error correction in 2021, fault-tolerant logical teleportation with transversal gates and lattice surgery in 2024—are colour-code experiments [88, 89].

7.1.2 Three-dimensional colour codes, gauge colour codes

On a three-dimensional four-colourable lattice, with X-checks on the *cells* and Z-checks on the *faces*, every X-stabiliser is triply even and the code has a transversal T [12, 66]; the tetrahedral $d = 3$ member is the $[\![15,1,3]\!]$ code of Section 6.1. By the Bravyi–König theorem a D-dimensional topological code can only have transversal gates in level D of the Clifford hierarchy, so 3D is the minimum for a transversal T and no such code has a transversal H as well [15]. Bombín's *gauge colour code* [10] places both X- and Z-type operators on faces into a gauge group; fixing the gauge one way yields the code with transversal T, fixing it the other way yields a code with transversal Clifford gates, and switching between the two by measuring the gauge operators of the target code is the *gauge fixing* route to universality of Section 9.3. The 3D code also allows *single-shot*

error correction (one noisy syndrome round suffices, because the cell checks are redundant and their redundancy corrects measurement errors), but its qubit count grows as d^3.

7.2 Quantum LDPC codes

A family of codes is *LDPC* if every check has bounded weight and every qubit is in a bounded number of checks as $n \to \infty$; the surface code is LDPC with weight 4, but with $k = 1$. The hope of the field, and the design basis of IBM's, IonQ's and the neutral-atom roadmaps, is codes that are LDPC *and* have $k = \Theta(n)$ (constant rate) with d growing as fast as possible—which by the BPT bound (18) requires long-range checks [20]. Constructing such codes is a problem of commuting checks: random sparse H_X, H_Z never satisfy $H_X H_Z^{\mathsf{T}} = 0$, so every construction builds the orthogonality in algebraically.

7.2.1 Hypergraph product codes

Tillich and Zémor's product [96] takes two classical parity-check matrices H_1 ($m_1 \times n_1$) and H_2 ($m_2 \times n_2$) and sets

$$H_X = \left[\, H_1 \otimes I_{n_2} \;\middle|\; I_{m_1} \otimes H_2^{\mathsf{T}} \,\right], \qquad H_Z = \left[\, I_{n_1} \otimes H_2 \;\middle|\; H_1^{\mathsf{T}} \otimes I_{m_2} \,\right], \tag{19}$$

so that $H_X H_Z^{\mathsf{T}} = H_1 \otimes H_2^{\mathsf{T}} + H_1 \otimes H_2^{\mathsf{T}} = 0$ automatically. The qubits are the $n_1 n_2$ "bit–bit" pairs plus the $m_1 m_2$ "check–check" pairs; the parameters are

$$[\![\, n_1 n_2 + m_1 m_2 \,,\, k_1 k_2 + k_1^{\mathsf{T}} k_2^{\mathsf{T}} \,,\, \min(d_1, d_2, d_1^{\mathsf{T}}, d_2^{\mathsf{T}}) \,]\!],$$

with $k^{\mathsf{T}}, d^{\mathsf{T}}$ the dimension and distance of the transpose code $\ker H^{\mathsf{T}}$. The toric code is the hypergraph product of two cyclic repetition codes ($n_1 = n_2 = m_1 = m_2 = L$, $k = 1 + 1 = 2$, $d = L$). Taking $H_1 = H_2$ a good classical LDPC code with $k = \Theta(n)$ and $d = \Theta(n)$ gives $k = \Theta(n)$ and $d = \Theta(\sqrt{n})$ quantum codes with constant check weight—the first constant-rate quantum LDPC family—and their generalisation, the *lifted product* over a group ring [81], is the code family of the neutral-atom estimates in Section 10 (e.g. $[\![2610, 744, \le 16]\!]$ at a 29% rate) [21].

7.2.2 Bivariate bicycle codes and IBM's gross code

IBM's bivariate bicycle (BB) codes [18] are the lifted product of a one-term code with itself over the abelian group $\mathbb{Z}_\ell \times \mathbb{Z}_m$. Let $x = S_\ell \otimes I_m$ and $y = I_\ell \otimes S_m$ be the two commuting cyclic-shift matrices (S_ℓ the $\ell \times \ell$ shift), and let A and B be sums of three monomials in x, y. Then

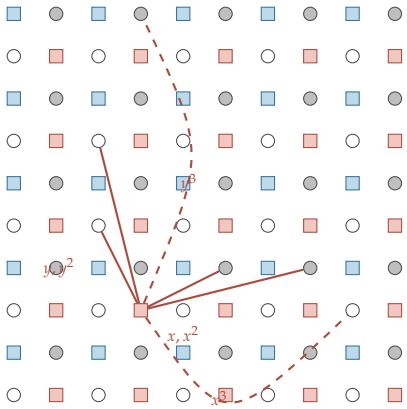
$$H_X = [\, A \mid B \,], \qquad H_Z = [\, B^{\mathsf{T}} \mid A^{\mathsf{T}} \,], \qquad H_X H_Z^{\mathsf{T}} = AB + BA = 0, \tag{20}$$

because polynomials in commuting matrices commute. There are $n = 2\ell m$ data qubits in two halves L and R, ℓm X-checks and ℓm Z-checks, every check has weight six (three L neighbours from the monomials of A or B^{T}, three R neighbours from the other), every data qubit is in three X- and three Z-checks, and $k = 2\dim(\ker A \cap \ker B)$. Table 11 lists the codes of IBM's paper; the parameters n, k, check weights and degrees were recomputed for this document from the polynomials (`scripts/bb_codes.py`), the distances are IBM's (found by integer programming).

code	ℓ, m	n	A	B	k (recomputed)
$[\![72, 12, 6]\!]$	6, 6	72	$x^3 + y + y^2$	$y^3 + x + x^2$	12
$[\![90, 8, 10]\!]$	15, 3	90	$x^9 + y + y^2$	$1 + x^2 + x^7$	8
$[\![108, 8, 10]\!]$	9, 6	108	$x^3 + y + y^2$	$y^3 + x + x^2$	8
$[\![144, 12, 12]\!]$ ("gross")	12, 6	144	$x^3 + y + y^2$	$y^3 + x + x^2$	12
$[\![288, 12, 18]\!]$ ("two-gross")	12, 12	288	$x^3 + y^2 + y^7$	$y^3 + x + x^2$	12

Table 11: Bivariate bicycle codes [18]. Every check has weight 6 and every qubit degree 6 (three X, three Z); $H_X H_Z^{\mathsf{T}} = 0$ and the values of k were verified.

Layout and syndrome cycle. The Tanner graph of a BB code has degree six and is *not* planar, but it is the union of two edge-disjoint planar graphs [18]: on a torus of $\ell \times m$ cells the L qubits, R qubits, X-checks and Z-checks occupy the four sites of each cell, four of the six edges of every check are nearest-neighbour and two are long-range (the x^3, y^3 or x^7 terms), which is what IBM's *c-couplers* on the Loon processor are for [58]. With one ancilla per check the syndrome cycle is seven CNOT layers deep, interleaving the X- and Z-check extraction so that a check qubit talks to its six data qubits while the data qubits are shared with the other type; a full cycle is depth 8 with measurement and reset—the same depth as the surface code's despite the weight-six checks. Under circuit-level depolarising noise with BP+OSD decoding the family has a threshold near 0.8%, and at $p = 10^{-3}$ the gross code's 288 qubits hold 12 logical qubits at a logical error that the surface code would need nearly 3,000 qubits to match: the "tenfold" saving [18]. IBM's later circuit and decoder work puts the gross code's idle error at $10^{-8.8}$ per 8-timestep cycle for the whole block at $p = 10^{-3}$, and the two-gross code's at 10^{-20} [99].



One X check (red square, cell $(1,1)$) of a BB code with $A = x^3 + y + y^2$, $B = y^3 + x + x^2$: three L data qubits (open circles) at the monomials of A and three R data qubits (grey) at those of B. Four edges are nearest-neighbour, two (dashed) are long-range; the Z checks (blue squares) use B^{T} and A^{T}. Opposite edges of the $\ell \times m$ cell grid are identified.

Figure 24: The bivariate bicycle layout: four sites per unit cell on a torus, degree-six checks, two long-range couplers per check.

Decoding: belief propagation with ordered-statistics post-processing. Quantum LDPC codes are decoded like classical ones, by message passing on the Tanner graph of H_Z (for X errors) and H_X (for Z errors) [81, 87]:

Algorithm 4 BP+OSD decoding (one check type; H is $m \times n$, syndrome s, prior error rate p)

1: initialise every variable-to-check message with the prior log-likelihood ratio $\lambda_0 = \log \frac{1-p}{p}$
2: **for** $t = 1, \dots, T_{\max}$ **do** ▷ min-sum belief propagation
3: check i to variable j: $\mu_{i\to j} = (-1)^{s_i}\, \alpha \prod_{j' \in N(i)\setminus j} \operatorname{sgn}(\lambda_{j'\to i}) \cdot \min_{j' \in N(i)\setminus j} |\lambda_{j'\to i}|$ (scaling $\alpha \approx 0.8$)
4: variable j to check i: $\lambda_{j\to i} = \lambda_0 + \sum_{i' \in N(j)\setminus i} \mu_{i'\to j}$; posterior $\Lambda_j = \lambda_0 + \sum_{i' \in N(j)} \mu_{i'\to j}$
5: hard decision $\hat{e}_j = [\Lambda_j < 0]$; **if** $H\hat{e} = s$ **return** $\hat{e}$
6: **OSD-0:** sort the columns of H by increasing Λ_j (most likely error first); Gaussian-eliminate in that order to pick an information set S of rank H independent columns; solve $H_S\, e_S = s$ and set $e_j = 0$ off S; **return** e
7: **OSD-CS(λ)** (optional): also try flipping each of the next λ most-likely columns outside S, one or two at a time, re-solving each time, and keep the lowest-weight solution

Plain BP fails on quantum codes far more often than on classical ones because of degeneracy: two equally likely errors that differ by a stabiliser pull the marginals in opposite directions, the messages oscillate, and BP ends in a configuration that does not satisfy the syndrome. OSD repairs exactly that case, at the cost of an $O(n^3)$ elimination, which is why decoding *speed* is the

main engineering problem of qLDPC architectures. Figure 25 is a small BP+OSD-0 simulation of the gross code written for this document (code-capacity bit-flip noise, i.e. no measurement errors, which is far easier than circuit-level noise and only illustrates the decoder); IBM's answer to the speed problem is **Relay-BP** [79]: BP with *disordered memory strengths* (each variable node mixes its new belief with its previous one by a node-dependent, partly negative weight, which damps the oscillations and breaks the symmetries that trap plain BP), run as a *relay* of several short BP instances with different memory settings that hand their beliefs on, so that several valid corrections are found and the best kept. On BB codes under circuit noise it outperforms BP+OSD with combination sweep, on surface codes it matches MWPM, and it is a parallel message-passing algorithm that fits an FPGA or ASIC—IBM quotes a 5–10× reduction in decoding resources [58]. Cain et al.'s neutral-atom estimates use BP with *localised statistics* decoding in the same spirit [21].

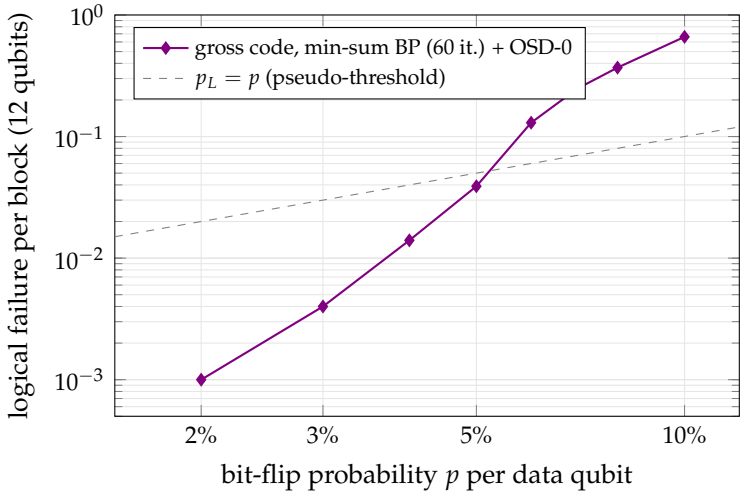


Figure 25: BP+OSD-0 on the $[\![144, 12, 12]\!]$ gross code under independent X errors (computed; 1,000–2,000 trials per point, `scripts/bp_osd_gross.py`). The block fails when the residual error is outside the X-stabiliser group. Code-capacity noise only; the circuit-level numbers of [18, 99] are orders of magnitude lower at $p = 10^{-3}$.

Logical operations and IBM's bicycle architecture. A BB code has no boundaries to merge, so lattice surgery is replaced by two mechanisms. First, the shifts x and y are *automorphisms*: permuting the qubits by $x^a y^b$ maps the code to itself and acts as a logical Clifford on the 12 logical qubits (together with the $L \leftrightarrow R$ "ZX-duality" that swaps H_X and H_Z); IBM's instruction set has twelve such shift automorphisms, each a physical permutation (SWAP network) that costs 14 timesteps [99]. Second, arbitrary logical Pauli measurements are made by *generalised surgery* [25, 26]: to measure a logical $\bar{P}$ one attaches an auxiliary system of ancilla qubits whose checks, together with the code's, form a temporarily larger code in which $\bar{P}$ is a product of new stabilisers; measuring the enlarged code for $\approx d$ rounds measures $\bar{P}$ fault-tolerantly, and the ancillas are then released. IBM's improved construction needs only $\Theta(w)$ ancillas for a weight-w logical and gives the gross code its *logical processing unit* (LPU): 90 extra qubits (158 for two-gross) wired to the code block through which any Pauli in a chosen generating set can be measured, with one of the twelve logical qubits used as a workspace, so that all logical Cliffords are reachable [26, 99]. A *bicycle module* is then a code block (288 or 576 qubits), its LPU, a 22-qubit code-to-code adapter for inter-module joint measurements, and a magic-state factory (454 qubits at $p = 10^{-3}$) with a 29-qubit adapter; in-module logical measurements cost 120 timesteps at $10^{-5.0}$ error (gross) or 216 timesteps at $\approx 10^{-11}$ (two-gross), T injection is a joint $\bar{Z}_T \otimes \bar{P}$ measurement between factory and module followed by an $\bar{X}_T$ readout, and IBM estimates that for a given number of physical qubits the architecture runs circuits about ten times larger than a surface-code machine [99]. The roadmap built on it [58, 60]: Loon (2025) tests the c-couplers; Kookaburra (2026) is the first module

with a qLDPC memory and an LPU; Cockatoo (2027) entangles two modules through the adapter; Starling (2029) targets 200 logical qubits and 10^8 gates; Blue Jay (2033) 2,000 logical qubits and 10^9 gates. IBM's Quantum Learning course on error correction is the pedagogical companion to these papers [59].

7.2.3 Trapped ions: IonQ's walking-cat architecture

Trapped ions have no fixed connectivity—ions are shuttled between zones—so the locality constraint that shapes the superconducting designs disappears, and IonQ's architecture [97] uses quantum LDPC codes with Shor-style syndrome extraction: a *cat factory* continuously prepares and verifies GHZ (cat) states, which are physically transported ("walked") to the code blocks and consumed to measure stabilisers and logical operators transversally, so that no ancilla ever couples to more than one data qubit and hook errors do not exist. Two codes are used: $[\![70, 6, 9]\!]$ for a *fast* design and $[\![102, 22, 9]\!]$ ("Q102", 102 data and 102 ancilla qubits) for a *dense* one in which 110 logical qubits live in 2,514 physical qubits and about a million T gates run per day. Combined with Clifford-frame tracking (Section 8) and a two-level CCZ factory in an LDPC code, this is the architecture of IonQ's ECDLP estimate—1,457 logical qubits and 39×10^6 Toffolis on 19,397 physical qubits in 25.7 days at $p_{2q} = 10^{-4}$—examined in Section 10 [54].

7.2.4 Asymptotically good quantum LDPC codes

Panteleev and Kalachev showed in 2021 that lifted-product codes over non-abelian groups can be *good*—$k = \Theta(n)$ and $d = \Theta(n)$ with constant check weight—and Leverrier and Zémor's quantum Tanner codes gave a second, more explicit family [69, 82]; linear-time decoders that correct a constant fraction of the distance followed [51, 70]. These settle the theory (the Gilbert–Varshamov rate is achievable with sparse checks) but have large constants and expander-graph connectivity, and no architecture uses them yet; the practical frontier is the BB and lifted-product codes above.

7.3 Other codes in use

- **Bacon–Shor and subsystem codes** [6]: the Shor code with its weight-two X_iX_j and Z_iZ_j face operators promoted to gauge operators. Only weight-two measurements are needed (the stabilisers are inferred as products), at the price of no threshold for the family; used in early trapped-ion demonstrations.

- **Concatenated codes**: the proof-of-concept of the threshold theorem and still the basis of several estimates; the Steane code concatenated with itself, or Knill's C_4/C_6 postselection scheme [64], reach thresholds near 10^{-3}–10^{-2} with large qubit overheads. Code switching between Steane and Reed–Muller (Section 9.3) lives here [3, 80].

- **Yoked surface codes** [44]: Google's proposal to concatenate surface-code patches with an outer parity-check code across patches, reducing the storage cost of idle logical qubits by a factor of two or three; used for the idle registers of the RSA-2048 estimate [40].

- **Bosonic codes**: a qubit stored in an oscillator. GKP codes [48] use grid states; *cat qubits* [52] use the two coherent states $|\pm\alpha\rangle$ of a two-photon-stabilised resonator, in which bit flips are suppressed exponentially in the photon number $|\alpha|^2$ while phase flips grow only linearly, so that a one-dimensional repetition code against phase flips suffices. Alice & Bob's ECDLP estimate—126,133 cat qubits for a 256-bit discrete logarithm in nine hours with 19 photons per cat and a 500 ns cycle—is the bosonic point of Section 10 [49].

- **Detection codes** ($[\![4, 2, 2]\!]$, $[\![8, 3, 2]\!]$, iceberg codes): no correction, only post-selection, but transversal gates on many logical qubits; QuEra/Harvard's 48-logical-qubit processor ran its largest circuits in these [9], and its 2024 experiment distilled logical magic states in them [86].

8 Fault-tolerant Clifford gates

A logical gate is only useful if it does not undo the code's work. The requirement, formalised in the threshold theorem, is that a gadget be *fault tolerant*: s faults inside it produce at most s errors on each code block it touches, so that a distance-d code still corrects whatever a sub-threshold number of faults did. The enemy is *propagation*: a CNOT copies an X error from its control to its target and a Z error from its target to its control, so a single fault inside a circuit that couples qubits of one block can become a weight-two or weight-three error on that block. Everything in this section is a way of coupling blocks without coupling the qubits inside a block more than the code can bear.

8.1 Transversal gates and their limits

A gate is *transversal* if it acts as a tensor product of single-qubit gates, or of two-qubit gates each touching the i-th qubit of one block and the i-th qubit of another. Transversal gates are fault tolerant by construction: no gate acts on two qubits of the same block.

- **CNOT, every CSS code.** $\mathrm{CNOT}^{\otimes n}$ between two blocks maps X-stabilisers of the control block onto products with the target's X-stabilisers and Z-stabilisers the other way; since both blocks have the same X- and Z-check matrices the stabiliser group is preserved, and $\bar{X}_1 \mapsto \bar{X}_1\bar{X}_2$, $\bar{Z}_2 \mapsto \bar{Z}_1\bar{Z}_2$: a logical CNOT. This is why the surface, colour, bicycle, hypergraph-product and Reed–Muller codes all have a free logical CNOT *between* blocks—the problem is always the gates *within* a block or between patches that cannot be stacked.
- H **and** S**, self-dual doubly-even CSS codes** (Steane, 2D colour codes, $[\![4,2,2]\!]$): $H^{\otimes n}$ swaps the X- and Z-checks, which coincide, and $S^{\otimes n}$ maps X-checks to $X \cdot Z$ products that are stabilisers when every check weight is divisible by four (Section 7.1). The whole Clifford group is transversal.
- T**, triply-even codes** ($[\![15,1,3]\!]$, 3D colour codes): Section 6.1. These lose the transversal H.
- **The** $[\![5,1,3]\!]$ **code** has a transversal order-three Clifford (SH up to Paulis, cycling $X \to Y \to Z$) and nothing more.

Two theorems say this list cannot be completed. Eastin and Knill [31]: for any code that detects at least one error, the set of logical gates implementable transversally is a finite group, hence not universal—because a transversal gate set would be a continuous group acting locally, and a code detecting single-qubit errors cannot distinguish infinitesimal local rotations from the identity. Bravyi and König [15]: for a topological stabiliser code in D dimensions, any gate implementable by a constant-depth local circuit lies in level D of the Clifford hierarchy, so 2D codes get at most Cliffords (level 2) and a transversal T (level 3) needs three dimensions. The architectures of Section 10 therefore all have the same shape: a 2D (or LDPC) code for memory and Clifford operations, plus a separate mechanism—magic states, code switching—for the one non-Clifford gate.

8.2 Fault-tolerant syndrome extraction

Measuring a weight-w check with one ancilla (Figure 16) is not fault tolerant on its own: an ancilla fault halfway through propagates to $\lfloor w/2 \rfloor$ data qubits. The classical remedies are [24, 64, 92, 95]:

Shor-style (cat states).
Prepare a w-qubit GHZ state $|0^w\rangle + |1^w\rangle$, verify it (measure a few Z_iZ_j parities and discard on failure), then couple ancilla qubit i to data qubit i only; measure all ancillas in the X basis and take the parity. Every data qubit talks to its own ancilla, so a single fault cannot spread within the block; a measurement fault can still give a wrong syndrome bit, so each check is

repeated until two consecutive rounds agree. This is the mechanism of IonQ's architecture, where the cat states are prepared in a factory and shuttled to the data [97].

Steane-style (encoded ancilla).
Prepare a whole ancilla block in $|+_L\rangle$ (fault-tolerantly, by verifying it against a second block), apply a transversal CNOT from data to ancilla, and measure every ancilla qubit in Z: the outcomes give *all* Z-type checks of the data at once, and the transversality keeps propagation at bay. A second block in $|0_L\rangle$ and a reversed CNOT reads the X-type checks. Only for CSS codes.

Knill-style (teleportation).
Prepare an encoded Bell pair $|\Phi_L\rangle$ and Bell-measure the data block against one half of it: the outcomes give every stabiliser and the logical state is teleported into the other half, a fresh block—error correction and a new block in one step. Knill's C_4/C_6 scheme reaches a threshold near 1% with this [64].

Flag qubits.
Chao and Reichardt showed that one or two extra qubits suffice for any distance-three code [24]: a *flag* ancilla is coupled to the syndrome ancilla before and after the middle CNOTs, so that an ancilla fault that would propagate a hook error also flips the flag; a raised flag tells the decoder which correlated error to expect, and a lookup table indexed by (flag, syndrome) corrects it (Figure 26). Flags are what made the small-code trapped-ion and neutral-atom experiments fault tolerant with few ancillas.

Repetition in time.
The surface and bicycle codes use one ancilla per check and *no* verification; instead the syndrome is measured $\approx d$ times and the decoder works on the space-time graph of Figure 21, so that measurement faults and hook errors are just more edges. The circuit's CNOT ordering is chosen so that no single fault reduces the effective distance (Figure 20); for the gross code IBM verified the same property numerically [18].

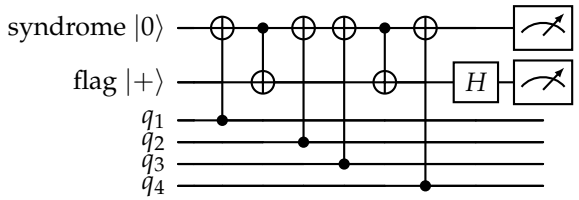
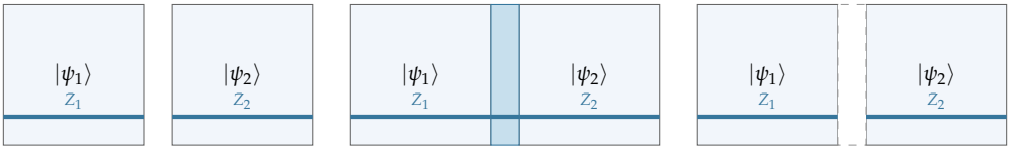


Figure 26: A weight-four Z check with one flag qubit [24]. An X fault on the syndrome qubit between the second and third data CNOT would propagate to q_3q_4 (a hook); it also passes through the second flag CNOT and flips the flag's X-basis outcome, so the decoder learns that a weight-two error of a known shape has occurred and corrects it. Without a fault the flag measures $+1$.

8.3 Lattice surgery: the surface code's logical Cliffords

Surface-code patches cannot be stacked for a transversal CNOT in a planar chip, and the code has no transversal H or S. Horsman, Fowler, Devitt and Van Meter's *lattice surgery* [56] replaces gates by *joint measurements of logical Paulis*, which the code performs natively:



(a) two patches, gap of one row

(b) merge: d rounds with the strip's Z faces on

(c) split: strip measured in X

Figure 27: Measuring $\bar{Z}_1\bar{Z}_2$ by lattice surgery. The two patches' Z boundaries face each other across a one-qubit-wide strip prepared in $|+\rangle$; switching on the strip's Z faces for d rounds measures them, and their product (times already-known boundary checks) is $\bar{Z}_1\bar{Z}_2$, read as the parity of the new outcomes; the merged object is a single wider patch encoding one qubit. Measuring the strip in X splits it again, leaving $|\psi_1\psi_2\rangle$ projected onto the $\bar{Z}_1\bar{Z}_2 = \pm 1$ eigenspace, up to a known Pauli frame update. $\bar{X}_1\bar{X}_2$ is the same with the X boundaries.

- M_{ZZ}, M_{XX} (Figure 27): d rounds each, because the new checks must themselves be measured d times to be trusted. The outcome has a logical error of order $p_L(d)$ and the patches keep their distance throughout.
- **CNOT** = an ancilla patch $|+_L\rangle$, M_{ZZ} between control and ancilla, M_{XX} between ancilla and target, then M_Z of the ancilla, with Pauli corrections from the three outcomes: $2d$ rounds plus the single-qubit measurement, three patches of space. Equivalently, teleport through the ancilla [36, 56].
- **Hadamard**: transversal H on a patch swaps its X and Z boundaries; to restore the standard orientation the patch is *rotated* by growing it into an adjacent ancilla region and shrinking back ($\approx 3d$ rounds), or the orientation is simply remembered.
- S **and** $S^\dagger$: a $\bar{Y}$ measurement, which lattice surgery provides through a *twist defect* on the patch boundary, or M_{ZZ} with an ancilla in $|Y_L\rangle$ (a stabiliser state, preparable once and reused: $S = M_{ZZ}$ with $|Y\rangle$ followed by a Pauli correction) [71].
- **Multi-qubit Pauli product measurements**: an ancilla region touching several patches measures $\bar{P}_1 \otimes \cdots \otimes \bar{P}_k$ in one shot of d rounds; Litinski's layouts arrange patches in rows with one row of ancilla space so that any product is reachable [71].

Pauli and Clifford frames. No Pauli correction is ever applied to the qubits: the decoder's output is a classical *Pauli frame* that is updated and consulted when a measurement outcome is interpreted, because Paulis commute or anticommute with everything else in a known way [64]. Litinski's *game of surface codes* [71] extends this to Cliffords: any Clifford+T circuit is rewritten as a sequence of $\pi/8$ Pauli-product rotations $e^{-i\frac{\pi}{8}P_j}$ (one per T gate, with P_j a multi-qubit Pauli) followed by a final layer of Pauli measurements, by commuting every Clifford to the end of the circuit and absorbing it into the measurement basis. Each $\pi/8$ rotation is then one Pauli product measurement $P_j \otimes \bar{Z}$ against a magic state (Section 9.2), a Clifford correction $e^{\pm i\frac{\pi}{4}P_j}$ that is itself pushed to the end, and the final Pauli measurements are read in whatever basis the accumulated Clifford dictates. Logical Clifford gates thus cost *nothing* in physical time; the entire runtime is the sequence of Pauli product measurements, one *tick* of d cycles each in the "fast" layout—the "reaction-limited" regime of Section 10, where the time per T is the classical reaction time (decode, update the frame, choose the next measurement basis) and not the gate time. IonQ's *Clifford frame* is the same idea applied to a trapped-ion LDPC machine: only Pauli measurements and non-Clifford state injections are physically executed, and the Cliffords—including the transversal ones the codes do have—are classical bookkeeping [54, 97]. This is also why Part I's cost model counts Toffolis and nothing else: Cliffords are free at the logical level, and a Toffoli is four non-Clifford rotations.

Clifford gates on LDPC codes. With no boundaries to merge, bicycle codes get their Cliffords from automorphisms (qubit permutations that preserve the code, each a logical Clifford on all 12 qubits at once) and from the generalised surgery of their LPU (Section 7.2.2), where a logical Pauli product is measured by attaching an ancilla system for $\approx d$ rounds; IBM's instruction set contains 12 automorphisms (14 timesteps), 15 in-module and 36 inter-module Pauli measurements (120 timesteps for the gross code), and the frame does the rest [26, 99]. Colour-code machines (Quantinuum, Google's 2024 experiment) use transversal H, S and CNOT directly and lattice surgery between colour-code patches for the rest [67, 89]; neutral-atom arrays move whole blocks on top of one another for transversal CNOTs between any two blocks [9].

9 Fault-tolerant T gates

9.1 Why T is the expensive gate

Clifford circuits are classically simulable (Gottesman–Knill), so a universal machine needs one gate outside the Clifford group; $T = \mathrm{diag}(1, e^{i\pi/4})$ is the standard choice because it is the cheapest member of the third level of the Clifford hierarchy and because Toffoli = CCZ + Hadamards = seven T gates in the textbook decomposition, four with measurement-based uncomputation (Section 2.5). By Eastin–Knill and Bravyi–König no 2D code has a transversal T, so the T gate is *teleported in* from a resource state:

$$|T\rangle = T|+\rangle = \tfrac{1}{\sqrt{2}}\big(|0\rangle + e^{i\pi/4}|1\rangle\big), \tag{21}$$

by the gadget of Figure 28: a CNOT from the data to the magic state and a Z measurement of the magic state apply T to the data up to a Clifford S correction that depends on the outcome—and that correction goes into the Clifford frame. In lattice-surgery form the gadget is one M_{ZZ} between the data patch and the magic-state patch followed by M_X of the magic patch, i.e. one tick. So the cost of a T gate is the cost of one $|T\rangle$ of sufficient quality, and the whole subject reduces to producing $|T\rangle$ states with error far below $1/(\text{number of } T \text{ gates})$: for the ECDSA.fail circuits, $4 \times 28 \times 1.5 \times 10^6 \approx 1.7 \times 10^8$ T gates per run, i.e. a per-state error of order 10^{-10}.

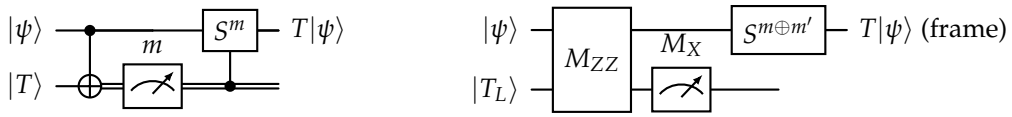


Figure 28: T-gate teleportation. Left: with outcome $m = 0$ the data is $T|\psi\rangle$; with $m = 1$ it is $\omega\, T^\dagger|\psi\rangle$ and $S\,T^\dagger = T$ fixes it ($\omega = e^{i\pi/4}$). Right: the lattice-surgery version, a joint $Z \otimes Z$ measurement (outcome m) and an X measurement of the magic patch (outcome m'); the S correction is never applied, only recorded.

9.2 Magic-state injection and distillation

Injection. A physical $|T\rangle$ is prepared by a single-qubit rotation on one data qubit of a $d = 1$ patch, which is then grown to distance d by switching on checks around it; during the first rounds the state is unprotected, so the injected logical state has an error of order p (a few 10^{-3} for $p = 10^{-3}$, $\approx 1\%$ in Google's colour-code experiment before post-selection) [67]. Injected states are therefore *distilled*.

15-to-1 distillation. Bravyi and Kitaev's protocol [14] uses the $[\![15, 1, 3]\!]$ code's transversal T (Section 6.1). First note that any noisy $|T\rangle$ can be *twirled* (randomly conjugated by the Clifford SX, which fixes $|T\rangle$) into the form $(1-\epsilon)|T\rangle\langle T| + \epsilon\, Z|T\rangle\langle T|Z$: a magic state with error is a magic state with a Z error of probability ϵ. Then:

Algorithm 5 15-to-1 magic-state distillation

1: prepare $|+_L\rangle$ of the $[\![15,1,3]\!]$ Reed–Muller code (a stabiliser state: $|+\rangle^{\otimes 15}$ projected by the ten Z-checks, Section 5.3)
2: apply T transversally by teleporting 15 noisy $|T\rangle$ states, one into each qubit, with the gadget of Figure 28 (15 S corrections tracked)—the block is now $\bar{T}^\dagger|+_L\rangle = |T_L^\dagger\rangle$ up to Z errors at the positions of the faulty input states
3: measure the four X-type stabilisers; **if** any is -1 **reject** (a Z error of weight ≤ 2 has been detected)
4: decode: measure the fourteen non-data qubits so as to map $|T_L^\dagger\rangle$ to a single qubit (the inverse of the encoder of Algorithm 1, or a direct Z-basis readout of the code with the logical value extracted), apply $S^\dagger$ to turn $|T^\dagger\rangle$ into $|T\rangle$

A Z error passes undetected only if it is a Z-logical of the code, and the lightest Z-logicals have weight three: there are 35 of them (the lines of the projective space $PG(3,2)$), so the output error is $35\epsilon^3 + O(\epsilon^4)$ with acceptance probability $1 - 15\epsilon + O(\epsilon^2)$. From $\epsilon = 10^{-3}$ one round gives 3.5×10^{-8} and two rounds 1.5×10^{-21}; in practice the *implementation* of the protocol in the surface code is itself noisy, and that is what sets the floor. Bravyi and Haah's $(3k+8)$-to-k protocols [13] distil k states at once with output error $(3k+1)\epsilon^2$—quadratic, not cubic, but with far better yield—and are used as second stages. Litinski's surface-code *factories* [72] spell out the cost: at $p = 10^{-3}$ a single-level $(15\text{-to-}1)_{17,7,7}$ factory occupies 4,620 physical qubits and produces one $|T\rangle$ of error 4.5×10^{-8} every 42.6 cycles; a two-level $(15\text{-to-}1)^6_{13,5,5} \times (20\text{-to-}4)_{23,11,13}$ factory, 43,300 qubits, one state of error 1.4×10^{-10} every 130 cycles; and $(15\text{-to-}1)^6_{13,7,7} \times (8\text{-to-CCZ})_{25,15,15}$, 47,000 qubits, one $|CCZ\rangle$ of error 5.2×10^{-11} every 60 cycles, where the subscripts are the code distances of the two levels. Gidney and Fowler's CCZ factory with the catalysed $|CCZ\rangle \to 2|T\rangle$ conversion [42] is the design behind the "one CCZ per 150 cycles per factory" of Model S in Section 10: a Toffoli is then *one* magic state, not four.

Cultivation. Gidney, Shutty and Jones [45] replaced the first distillation level by *magic state cultivation*: prepare $|T\rangle$ in a small colour code ($d = 3$, then 5) where T is transversal, repeatedly *check* the state by measuring the transversal $\bar{T}$-conjugated stabilisers and post-select, then "escape" into a surface code of distance 15 by growing the patch. The whole thing fits in one patch, costs about as many physical gates as a lattice-surgery CNOT of the same reliability, and gives $|T\rangle$ states at 2×10^{-9} for $p = 10^{-3}$ (4×10^{-11} at $p = 5 \times 10^{-4}$)—an order of magnitude fewer qubit-rounds than distillation to the same error, and enough that the authors suggest distillation may no longer be needed in practice. Google's RSA-2048 and ECDLP-256 estimates both use it ($\approx 5 \times 10^4$ qubit-rounds per T, $\approx$ 25,000 qubits devoted to magic states) [5, 40]; the neutral-atom estimate cultivates in surface-code patches of $\approx$ 500 qubits and then distils $8T \to CCZ$ inside bicycle-code factories to reach $\leq 10^{-10}$ per CCZ [21]; IonQ's two-level CCZ factory produces states below 10^{-9} in 319 qubits each, four of them making a CCZ in under 28 ms [54].

Experiments. Logical magic-state preparation and distillation have been demonstrated: QuEra/Harvard distilled logical magic states in $[\![5,1,3]\!]$ and $[\![7,1,3]\!]$ codes on a neutral-atom array in 2024 [86]; Google injected magic states into a colour code at $> 99\%$ fidelity with post-selection [67].

9.3 Code switching and gauge fixing

The alternative to magic states is to carry the logical qubit into a code where T *is* transversal and back. Anderson, Duclos-Cianci and Poulin [3] gave the fault-tolerant conversion between the Steane code (transversal Cliffords) and the $[\![15,1,3]\!]$ Reed–Muller code (transversal T): the two are gauge fixings of one $[\![15,7,3]\!]$ subsystem code, and one converts by measuring the stabilisers that

the target code has and the source does not (the weight-four Z-checks of Section 6.1 when going to Reed–Muller, weight-four X-type operators when coming back), correcting with Paulis chosen from the outcomes, and repeating the measurements to make them fault tolerant [80]. Bombín's gauge colour codes do the same in a topological setting (Section 7.1): the 3D code's face operators are gauge degrees of freedom, fixing them as Z-type stabilisers gives transversal T, fixing them as X-type gives transversal Cliffords, and the switch is a measurement of the operators being fixed [10]. Code switching avoids factories but needs 3D codes ($n \sim d^3$) or small concatenated codes with low thresholds, which is why no large-scale estimate to date uses it; its cost per T is a full syndrome-extraction cycle of the larger code rather than a magic state.

9.4 What a Toffoli costs, by architecture

architecture	non-Clifford resource	per-Toffoli time	qubits for magic states
surface code, superconducting, Model S (Section 10)	$\lvert CCZ\rangle$ from distillation factories [42]	150 cycles per factory; 6 factories $\Rightarrow$ 25 μs	six factories inside the 7×18-patch compute block (170,352 physical qubits with workspace)
surface code, Google 2025–26 [5, 40]	cultivation [45], $\approx 5 \times 10^4$ qubit-rounds per T	reaction-limited: $\approx 10\,\mu$s reaction $\times$ 1.5 per Toffoli	$\approx$ 25,000
bicycle architecture, IBM [99]	factory in a bicycle module	T injection = one inter-module measurement, 120 timesteps	454 per module at $p = 10^{-3}$ (810 at 10^{-4})
walking cat, IonQ [54, 97]	two-level CCZ factory in an LDPC code, $< 10^{-9}$	29.5 ms (one measurement layer)	$4 \times 319 = 1{,}276$
neutral atoms, Cain et al. [21]	cultivation $+ 8T \to CCZ$ in $[\![248, 10, 18]\!]$ factories, $\leq 10^{-10}$	$\geq 4d$ stabiliser cycles of 1 ms	$\approx$ 500 per cultivation site plus factory codes
cat qubits, Alice & Bob [49]	off-line Toffoli-state preparation by projective measurements	500 ns cycle; 9 h total	included in 126,133

Table 12: The non-Clifford gate across the architectures compared in Section 10.

10 Physical-qubit estimates

10.1 From a logical circuit to a machine

A logical circuit is characterised by three numbers: its logical qubits Q, its non-Clifford gate count T (Toffolis here), and its non-Clifford *depth*. A fault-tolerant machine is characterised by its code, its physical error rate p, its cycle time t_c and its reaction time t_r (the classical latency from a measurement to the next basis choice). The estimate proceeds in four steps, which every published number in Table 14 follows in some form [36, 40, 41, 71]:

1. **Error budget.** The run must succeed with reasonable probability, so the total logical error must be $O(1)$: with Q' logical qubits (circuit plus workspace) alive for R rounds, $Q' R\, p_L(d) \lesssim 1$, where $R = t_{\text{run}}/t_c$. For $Q' \approx 1{,}300$ and a run of 10^9 rounds this is $p_L \lesssim 10^{-12}$; Model S below uses 10^{-15} per patch per round, i.e. $p_{\text{ok}} \approx 0.999$ for the fast circuits.

2. **Distance.** From Equation (13), the smallest odd d with $A(p/p_{\text{th}})^{(d+1)/2} \leq \varepsilon$; the footprint of a logical qubit is $2d^2 - 1$ physical qubits plus routing space, $2(d+1)^2$ in the layouts of [40, 41].

3. **Magic-state supply.** Enough factories (or cultivation sites) to deliver T states at the rate the circuit consumes them; their footprint is added to Q'.

4. **Time.** The larger of the *throughput* bound (gates $\times$ time per magic state) and the *reaction* bound (non-Clifford depth $\times$ t_r, because each layer of T gates must be decoded before the next basis is chosen, Section 8.3); then divide by the success probability and multiply by the expected number of shots.

10.2 Model S: a surface-code machine at $p = 10^{-3}$

Model S is the superconducting surface-code model of `shor/shor_qiskit/physical.py`, re-implemented in `scripts/physical_estimates.py` so that the two agree to the qubit: $p = 10^{-3}$, $t_c = 1\,\mu$s, $t_r = 10\,\mu$s, $A = 0.1$ and $p_{\text{th}} = 1.22\%$ (the crossing of Figure 22), target 10^{-15} per patch per round $\Rightarrow$ $d = 25$, 1,352 physical qubits per logical qubit, a compute block of 7×18 patches (170,352 physical qubits: six CCZ factories of the Gidney–Fowler type and three columns of workspace) delivering one $|CCZ\rangle$ every 25 μs, so that

$$N_{\text{phys}} = 1{,}352\,Q + 170{,}352, \qquad t_{\text{run}} = \max\left(25\,\mu\text{s} \times T,\ 10\,\mu\text{s} \times \text{depth}\right),$$
$$p_{\text{ok}} = (1 - 10^{-15})^{(Q+126)\,t_{\text{run}}/t_c}. \tag{22}$$

The ripple-carry circuits of Part I have a Toffoli depth of 0.8–0.95 $\times\ T$, so they are throughput-bound in this model (the reaction bound would need depth $< 0.4\,T$), and the time is simply 25 μs per Toffoli. For the ECDSA.fail rows the challenge's point-addition numbers are converted to a whole attack by Equation (2): 28 additions, each with its $3 \cdot 2^{16}$ lookup Toffolis, and 16 window qubits added to Q.

circuit (whole attack unless noted)	Q	Toffolis	physical	time/run	p_{ok}	expected
ECDSA.fail `8e9c9a2` (26 Jul, 1,151 × 1.30 M) +16 window qubits	1,167	41.9 M	1,748,136	17 min	0.999	17 min
ECDSA.fail windowed `8e9c9a2` (Note F, lookups included)	1,162	47.1 M	1,741,376	20 min	0.998	20 min
ECDSA.fail `a02c716` (7 Oct, Leapfrog, 1,244 × 0.80 M) +16	1,260	27.8 M	1,873,872	12 min	0.999	12 min
ECDSA.fail `b6f2b0a` low-Q (825 × 489 M) +16	841	13.70 B	1,307,384	4.0 d	0.718	5.5 d
ECDSA.fail `10177cd` low-Q (792 × 657 M, 7 Oct) +16	808	18.39 B	1,262,768	5.3 d	0.651	8.2 d
Google, Babbush et al. low-qubit variant [5]	1,200	90.0 M	1,792,752	38 min	0.997	38 min
Google, Babbush et al. low-gate variant [5]	1,450	70.0 M	2,130,752	29 min	0.997	29 min
IonQ, Häner et al. [54]	1,457	39.0 M	2,140,216	16 min	0.998	16 min
Schrottenloher, space-optimised [90]	1,208	72.5 M	1,803,568	30 min	0.998	30 min
Schrottenloher, gate-optimised [90]	1,462	57.7 M	2,146,976	24 min	0.998	24 min
`shor/`: IonQ cells, signed windows, SELECT-SWAP (executed)	1,457	36.8 M	2,140,216	15 min	0.999	15 min
`shor/`: dialog GCD on the space cells	1,246	79.1 M	1,854,944	33 min	0.997	33 min
`shor/`: three field registers, Luo EEA	851	2.39 B	1,320,904	16.6 h	0.943	17.6 h
RSA-2048, Gidney 2025 inputs (for calibration) [40]	1,399	6.50 B	2,061,800	45.1 h	0.781	2.4 d

Table 13: Model S applied to the ECDSA.fail circuits, to the published logical costs, and to the `shor/` circuits (computed, `scripts/physical_estimates.py`; the logical inputs of each row are cited in Table 5 and Section 4). "expected" divides the run time by p_{ok} (and multiplies by the expected number of shots where a source specifies more than one). The RSA-2048 row feeds Gidney's logical inputs through the same model; his published figure—under a million qubits, under a week—uses yoked storage for idle qubits and cultivation, which this model does not, so the row shows the model's conservatism rather than a discrepancy.

Three things stand out. First, the *physical* cost of every fast 256-bit ECDLP circuit in this model is 1.7–2.1 million qubits and 12–40 minutes: the Q axis of the challenge moves the qubit count by at most 20% because Q is multiplied by one patch footprint, and the T axis moves the time linearly. Second, the low-Q track buys a 25% smaller machine (1.26–1.31 million qubits) at the price of five to eight days and a success probability of 0.65–0.72 per run, which is why no architecture paper optimises for Q alone: in spacetime volume (qubits $\times$ seconds) the low-Q circuits are 400$\times$ more expensive. Third, the ECDSA.fail frontier's $Q \times T$ is a reasonable proxy for this volume—$N_{\text{phys}}\,t_{\text{run}} \approx (1{,}352Q + 170{,}352) \times 25\,\mu\text{s} \times 28(T + 3 \cdot 2^{16})$—but with a 170k-qubit constant and a $3 \cdot 2^{16}$-Toffoli constant that the challenge metric omits, both of which favour the fast circuits more than the metric does.

10.3 Per-gate accounting

For the reader who wants to rebuild the table from the gate level, at $d = 25$ and $p = 10^{-3}$:

- a **logical qubit** is a 25×25 data array with 624 measure qubits, 1,249 physical qubits, 1,352 with its share of routing; it costs one cycle of 1 μs per round to keep alive at $\approx 10^{-15}$ error;
- a **lattice-surgery CNOT** or Pauli product measurement occupies an ancilla region of the same width for $d = 25$ rounds, 25 μs; a Hadamard with patch rotation about $3d$ rounds; Paulis and the Clifford frame are free;
- a T **gate** consumes one magic state and one tick of d rounds; a Litinski $(15\text{-to-}1)_{17,7,7}$ factory (4,620 qubits) delivers one state of error 4.5×10^{-8} every 42.6 cycles, the two-level factories of Section 9.2 deliver 10^{-10}–10^{-11} states in 43–47k qubits every 60–130 cycles, and cultivation delivers 2×10^{-9} states for $\approx 5 \times 10^4$ qubit-rounds each [45, 72];
- a **Toffoli** is one $|CCZ\rangle$ (Model S: one every 25 μs from six factories) or four T states plus the measurement-based uncomputation of Section 2.5; the ECDSA.fail 26-July circuit's 4.2×10^7 Toffolis are 1.7×10^8 T gates, so the magic states must be below $\approx 10^{-9}$ each—which the two-level factories and cultivation provide, and the single-level factory does not.

10.4 The published estimates

source	platform, code	logical cost	physical qubits	time	assumptions
Google, Babbush et al. 2026 [5]	superconducting, surface code with yoked storage, cultivation	$<$ 1,200 Q, $<$ 90 M Toffoli; or $<$ 1,450 Q, $<$ 70 M	$<$ 500,000	$\approx$ 23 / 18 min	$p = 10^{-3}$, 1 μs cycle, 10 μs reaction +50% per Toffoli; $\approx$ 25,000 qubits for magic states; circuits withheld
IonQ, Häner et al. 2026 [54, 97]	trapped ions, $[\![102, 22, 9]\!]$ walking cat	1,457 Q, 39 M Toffoli	19,397	25.7 days (63% success)	$p_{2q} = 10^{-4}$, $p_{1q} = 10^{-5}$, 29.5 ms per Toffoli layer; four 319-qubit CCZ factories
PsiQuantum, Litinski 2023 [73, 74]	photonic, active-volume architecture	$\approx$ 50 M Toffoli (title)	6,000 modules $\times$1,152	one key per 10 min	non-local module connections; 38 h per key for a strictly 2D-local layout
Alice & Bob, Gouzien et al. 2023 [49]	cat qubits, repetition code	(256-bit ECDLP, Shor)	126,133 cat qubits	9 h	$\kappa_1/\kappa_2 = 10^{-5}$, 19 photons, 500 ns cycle, lattice surgery, off-line Toffoli states
Neutral atoms, Cain et al. 2026 [21]	reconfigurable atoms, lifted-product $[\![2610, 744, \leq 16]\!]$ memory, $[\![248, 10, \leq 18]\!]$ processor	(P-256, Shor)	$\approx$ 26,000 (balanced 11,961; space-efficient 9,739)	"a few days" (fastest $\approx$ 10 days at 10–26k)	$p = 10^{-3}$ (0.093% for the fastest), 1 ms stabiliser cycle, cultivation + $8T \to CCZ$; RSA-2048 $\approx 100\times$ longer
Kim et al. 2026 [62]	surface code; or QLDPC (Pinnacle model)	P-256: 7,454 Q, 1.39 B Toffoli (in-place)	11.7 M (surface) / 765,234 (QLDPC)	23.9 min / 21.9 h	$p = 10^{-3}$, 1 μs, 10 μs reaction; 15-to-1 distillation
Iceberg, Pinnacle 2026 [98]	QLDPC	RSA-2048	$<$ 100,000	–	$p = 10^{-3}$, 1 μs, 10 μs reaction
Google, Gidney 2025 [40]	surface code, yoked storage, cultivation	RSA-2048: $\approx$ 6.5 B Toffoli	$<$ 1 M	$<$ 1 week	$p = 10^{-3}$, 1 μs, 10 μs reaction
Model S, this document	surface code, factories	26-Jul ECDSA.fail + windows	1.75 M	17 min	Equation (22)

Table 14: Published physical estimates for 256-bit ECDLP (and the RSA-2048 points that use the same machinery), from the companies' and authors' arXiv/ePrint papers (Appendix A). "Q" is logical qubits. The platforms differ by two orders of magnitude in clock speed and in physical qubit count in opposite directions, which is why the time-to-key and the qubit count must be read together.

The ordering in Table 14 is the ordering of *clock speeds*: superconducting circuits at 1 μs per cycle run the 4×10^7-Toffoli circuit in minutes on 10^5–10^6 qubits; trapped ions at 30 ms per Toffoli layer

take weeks but need 20,000 ions because high-rate codes and all-to-all connectivity shrink both the code and the factories; neutral atoms sit between, with 1 ms cycles and 10–26k atoms; cat qubits use a biased-noise code that needs only a repetition code (126k cat qubits, nine hours). Google's half-million figure for the same logical circuit as Model S's two million comes from three choices this document's model does not make—storing idle logical qubits in yoked codes, which packs them two to three times denser, cultivating magic states instead of running factories (25k qubits instead of 170k), and tuning the distance to the actual run length—and from reaction-limited rather than throughput-limited timing (10 μs $\times$1.5 per Toffoli $\approx$ 15 μs instead of 25 μs) [5].

An LDPC illustration. IBM has published no ECDLP estimate, but its per-instruction numbers [99] allow an order-of-magnitude placement of the bicycle architecture. A two-gross module (576 code +158 LPU +34 adapter +463 factory at $p = 10^{-3}$, $\approx$ 1,230 physical qubits) holds 12 logical qubits with an in-module measurement error of $\approx 10^{-11}$ per 216 timesteps; 1,457 logical qubits are 122 modules, $\approx$ 150,000 physical qubits, and the 1.7×10^8 T gates of the 26-July circuit would accumulate $\approx 2 \times 10^{-3}$ logical error through their measurements. Each T costs one 216-timestep measurement; with the circuit's Toffoli depth close to its count the serial time is $\approx 1.7 \times 10^8 \times$ 216 μs $\approx$ 10 hours at a 1 μs timestep. This is *not* IBM's number—it ignores inter-module routing (the 36 inter-module measurements cost the same 216 steps but have higher error on the gross code), the factory throughput and the fact that the gross code, not the two-gross, is the near-term target—but it shows the shape of the LDPC trade: an order of magnitude fewer qubits than the surface code at $p = 10^{-3}$, and an order of magnitude more time, because logical operations on an LDPC block are serialised through its LPU rather than performed on all patches at once.

10.5 What the logical optimisations buy, physically

Translating Part I's frontier through Equation (22): the 2,715 $\times$ 3.96 M baseline of May is 3.9 M qubits and 49 min; the 26-July incumbent 1.75 M and 17 min; the 7-October frontier 1.87 M and 12 min. The six inversion architectures and the arithmetic levers together bought a factor of two in physical qubits and four in time in this model—the time coming almost entirely from the Toffoli count, the qubits from the width—and the remaining distance to Google's half-million is a matter of the *physical* layer (yoked storage, cultivation), not of the logical circuit. On the trapped-ion side the same 39 M-Toffoli circuit costs 19,397 ions and 26 days, so a further halving of the Toffoli count (the frontier's 0.80 M per addition against IonQ's 1.39 M) would be worth two weeks there and five minutes on a superconducting machine.

11 Conclusion

The whole picture in six statements

1. The cost of breaking secp256k1 is 28 windowed point additions, and the cost of a point addition is one modular inversion plus a handful of multiplications; ECDSA.fail spent five months on that inversion and moved its $Q \times T$ from 1.08×10^{10} to 1.0×10^{9} by replacing Kaliski's walk with a sequence of ever-cheaper recorded walks (dialog, Jump-2, ping-pong, Skywalk, Leapfrog), by pricing every comparator and fold against a validator-failure budget λ, and by sharing registers and venting qubits until the peak was set by a single owner (Section 2).
2. The frontier is two-dimensional and its two ends are far apart: $1{,}244 \times 0.80\,\mathrm{M}$ at the fast end, 792 qubits at the narrow end, with the narrow end costing $800\times$ more Toffolis (Section 3); the `shor/` package sits at $1{,}457 \times 1.31\,\mathrm{M}$ with exact or measured-error cells and the generic version of most of the frontier's levers (Section 4).
3. Error correction turns each logical qubit into $\sim 10^{3}$ physical ones and each T gate into a magic state, because the Knill–Laflamme conditions can be met by stabiliser codes whose checks are measured repeatedly and decoded classically, but no 2D code can be universal transversally (Sections 5 and 8).
4. The surface code is the baseline—threshold $\approx 1\%$, $\Lambda \approx 10$ per two units of distance at $p = 10^{-3}$, matching decoders, lattice surgery for Cliffords, factories or cultivation for T—and the quantum LDPC codes (IBM's bicycle codes, IonQ's walking-cat codes, lifted-product codes for atoms) trade a tenfold saving in qubits for serialised logical operations and harder decoding (Sections 6 and 7).
5. On a superconducting surface-code machine at $p = 10^{-3}$ the fast ECDSA.fail circuits need 1.7–1.9 million physical qubits and 12–20 minutes in this document's model, or under half a million qubits and $\approx$ 20 minutes with yoked storage and cultivation in Google's; the same circuits need $\approx$ 20,000 ions and 26 days, 10–26,000 atoms and days, or 126,000 cat qubits and nine hours (Section 10).
6. What the logical optimisations buy physically is time on fast machines and weeks on slow ones; what they cannot buy is the factor between two million and half a million qubits, which belongs to the physical layer.

For the `shor/` project. The package's natural next steps, in the order of their physical payoff: a Leapfrog backend behind `ec_gcd.mul` (the whole remaining Toffoli gap of the frontier is in the walk); the fused replay cell and the base-5 codec for the Jump-2 backend; and, on the physical side, a yoked-storage and cultivation variant of `physical.py` so that its surface-code numbers can be compared with Google's rather than only with Model S, plus an LDPC module model built from IBM's and IonQ's per-instruction tables (Table 12) for the platforms where logical-depth, not Toffoli count, sets the clock.

A Sources and how they were checked

The brief for this document restricted sources to arXiv, the IACR ePrint archive, and the published material of established quantum-computing companies, with the additional instruction to consult IBM Quantum's resources. The policy applied:

- **arXiv and ePrint.** Every paper cited with an arXiv or ePrint identifier was located through its arXiv abstract page (`arxiv.org/abs/...`) or its ePrint record, and the numbers quoted from it were taken from the abstract or from the paper's own tables (fetched as the arXiv HTML rendering where the abstract was not enough: [5, 21, 54, 99]). Two papers were read from the user's own copies in `shor/papers/`: Schrottenloher [90] (ePrint 2026/1128) and Kim et al. [62] (ePrint 2026/106).

- **Company material.** A source counts as a company source when the paper's arXiv record lists the company as the authors' affiliation or the page is served from the company's own domain. The companies whose material is used, and the form it takes:

  Google Quantum AI (`quantumai.google`).
  Willow below-threshold memory [1]; colour code on a superconducting processor [67]; AlphaQubit (with Google DeepMind) [7]; yoked surface codes [44]; magic state cultivation [45]; the colour-code circuits and Chromobius [43]; the RSA-2048 estimate [40]; the ECDLP-256 estimates and mitigations paper [5]; Stim and PyMatching [39, 55].

  IBM Quantum (`ibm.com/quantum`).
  The bivariate bicycle (gross) code paper [18]; the bicycle architecture ("Tour de gross") [99]; improved QLDPC surgery [26]; Relay-BP [79]; the June 2025 fault-tolerance announcement [58] and the development roadmap [60], both read from `ibm.com`; and the IBM Quantum Learning course "Foundations of quantum error correction" [59], used as the reference for the pedagogical ordering of Sections 5 and 6.

  IonQ (`ionq.com`).
  The walking-cat architecture [97] and the ECDLP-256 trapped-ion estimate [54] (both arXiv, IonQ affiliations; the latter is also ePrint 2026/1916).

  Quantinuum / Honeywell (`quantinuum.com`).
  Real-time fault-tolerant error correction on the $[\![7, 1, 3]\!]$ code [88] and fault-tolerant logical teleportation with transversal gates and lattice surgery [89].

  PsiQuantum (`psiquantum.com`).
  The active-volume architecture [74] and the "50 million Toffoli gates" ECDLP estimate [73].

  QuEra / Harvard (`quera.com`).
  The logical processor on reconfigurable atom arrays [9] and logical magic-state distillation [86]; the 2026 neutral-atom Shor estimate [21] is from the same Harvard/Caltech group with QuEra co-authors.

  Alice & Bob (`alice-bob.com`).
  The repetition-cat ECDLP-256 estimate [49] and the repetition-cat proposal [52].

  Iceberg Quantum's Pinnacle architecture [98] is included in Table 14 as an arXiv paper, not as an established-company source.

- **The ECDSA.fail material** is on GitHub: the challenge repository, the supplemental materials of the paper's authors, and the per-commit wiki (Section 1.3); the paper itself is on arXiv [75].

- **Textbook material** (stabiliser formalism, the small codes, the threshold theorem, the bounds) is cited to the original arXiv/journal papers [2, 8, 23, 30, 46, 63, 65, 68, 91, 94] and to Gottesman's textbook draft [47].

- **Computed for this document**: the encoders and syndrome tables of Section 6.1, the surface-code threshold data of Figure 22, the BB-code verification of Table 11, the BP+OSD simulation of Figure 25, the bounds of Figure 17, the Pareto and trajectory data of Section 3, and the Model S table. Every such number is marked "computed" at the point of use and reproducible from Appendix B.

B Scripts and data

All scripts are Python 3 and were run with Stim 1.16, PyMatching 2.4 and NumPy; the data files they produce are shipped with the LaTeX source in `data/`. Run them from the document's root directory.

script	what it does	output
`surface_threshold.py`	Stim rotated memory-Z circuits, $d = 3,5,7,9$, circuit-level depolarising noise, PyMatching MWPM, $\geq$ 1,000 failures or 12 M shots per point; per-round conversion	`surface_threshold.dat` (Figure 22)
`standard_form_encoder.py`	Gottesman standard form by Gaussian elimination, encoder emission, Stim verification of stabilisers and logicals for $[\![5,1,3]\!]$, Steane, Shor, $[\![15,1,3]\!]$	printed circuits (Figure 18)
`verify_codes.py`	Stim check of the textbook Shor and Steane encoders and of the $[\![5,1,3]\!]$ circuit; syndrome uniqueness; RM_{15} CSS structure	printed
`syndrome_tables.py`	lookup tables of the small codes; splits `codes.dat` by family	`syn_*.tex`, `codes_*.dat`
`bb_codes.py`	builds $H_X = [A\|B]$, $H_Z = [B^{\mathsf{T}}\|A^{\mathsf{T}}]$ for the five BB codes, checks orthogonality, weights, degrees and k	printed (Table 11)
`bp_osd_gross.py`	min-sum BP (60 iterations, $\alpha = 0.8$) + OSD-0 on the gross code under code-capacity X noise; logical failure test by rank	`bb_bposd.dat` (Figure 25)
`physical_estimates.py`	Model S (Equation (22)) applied to every row of Table 13; per-gate accounting	`physical_modelS.json`
`modelS_table.py`	formats the JSON as the table	`modelS_table.tex`

The Pareto, trajectory and census data of Section 3 (`incumbents.dat`, `promoted_qt.dat`, `rejected_qt.dat`, `frontier_oct7_*.dat`) were extracted from the challenge repository's leaderboard files, the supplemental materials' `frontier_reconstruction/` CSVs and the wiki's 7-October TSV snapshot; `bounds.dat` tabulates Equations (15) to (17) and `codes.dat` the (n,k,d) of the codes in Figure 17.

`scripts/standard_form_encoder.py`

```
"""Gottesman's standard-form encoder for a stabiliser code, built and verified with Stim.

Given stabiliser generators as Pauli strings, bring the binary check matrix [X|Z] to the
standard form  [ I  A1 A2 | B 0 C ; 0 0 0 | D I E ]  by Gaussian elimination (allowing
qubit permutations), derive logical X operators  [0 E^T I | C^T 0 0], and emit the
encoding circuit:  data qubits are the last k; for each logical X_j apply its Pauli
(restricted to the first n-k qubits) controlled on data qubit j; then for i < r apply H
on qubit i followed by generator i's Pauli on the other qubits controlled on qubit i.
"""
import sys, numpy as np, stim

def pauli_to_bits(s):
    n=len(s); x=np.zeros(n,int); z=np.zeros(n,int)
    for i,c in enumerate(s):
        if c in "XY": x[i]=1
        if c in "ZY": z[i]=1
```

```
    return x,z

def standard_form(stabs):
    m=len(stabs); n=len(stabs[0])
    M=np.array([np.concatenate(pauli_to_bits(s)) for s in stabs])%2
    perm=list(range(n))
    def swapcols(i,j):
        nonlocal M,perm
        if i!=j:
            M[:,[i,j]]=M[:,[j,i]]; M[:,[n+i,n+j]]=M[:,[n+j,n+i]]
            perm[i],perm[j]=perm[j],perm[i]
    # eliminate X part
    r=0
    for col in range(n):
        if r>=m: break
        piv=[i for i in range(r,m) if M[i,col]]
        if not piv:
            # try to bring a column with a pivot here
            found=False
            for c2 in range(col+1,n):
                if any(M[i,c2] for i in range(r,m)):
                    swapcols(col,c2); found=True; break
            if not found: break
            piv=[i for i in range(r,m) if M[i,col]]
        i=piv[0]; M[[r,i]]=M[[i,r]]
        for i2 in range(m):
            if i2!=r and M[i2,col]: M[i2]^=M[r]
        r+=1
    # eliminate Z part of the remaining rows on columns r..n-1
    r2=r
    for col in range(r,n):
        if r2>=m: break
        piv=[i for i in range(r2,m) if M[i,n+col]]
        if not piv:
            found=False
            for c2 in range(col+1,n):
                if any(M[i,n+c2] for i in range(r2,m)):
                    swapcols(col,c2); found=True; break
            if not found: break
            piv=[i for i in range(r2,m) if M[i,n+col]]
        i=piv[0]; M[[r2,i]]=M[[i,r2]]
        for i2 in range(m):
            if i2!=r2 and M[i2,n+col]: M[i2]^=M[r2]
        r2+=1
    return M,perm,r

def encoder_circuit(stabs):
    m=len(stabs); n=len(stabs[0]); k=n-m
    M,perm,r=standard_form(stabs)
    X=M[:,:n]; Z=M[:,n:]
    # blocks
    A2=X[:r, n-k:]; C=Z[:r, n-k:]; E=Z[r:, n-k:]
    # logical X: [0 E^T I | C^T 0 0]
    circ=stim.Circuit()
    for j in range(k):
        dq=n-k+j
        for q in range(r, n-k):          # E^T part: X on qubits r..n-k-1
            if E[q-r, j]: circ.append("CX",[dq,q])
        for q in range(r):               # C^T part: Z on qubits 0..r-1
            if C[q, j]: circ.append("CZ",[dq,q])
    for i in range(r):
        circ.append("H",[i])
        for q in range(n):
            if q==i: continue
            if X[i,q] and Z[i,q]: circ.append("CY",[i,q])
            elif X[i,q]: circ.append("CX",[i,q])
            elif Z[i,q]: circ.append("CZ",[i,q])
        if Z[i,i]:                      # generator i has Y on its pivot: (I+Y_i M')/sqrt2 needs
    the phase i
            circ.append("S",[i])
    return circ, perm, r

def verify(name, stabs, xl, zl):
    n=len(stabs[0]); m=len(stabs); k=n-m
    circ,perm,r=encoder_circuit(stabs)
```

```
    # the circuit acts on permuted qubits: physical qubit perm[i] <- circuit qubit i
    inv=[0]*n
    for i,p in enumerate(perm): inv[p]=i
    def relabel(s):  # pauli string on physical qubits -> circuit qubit order
        out=["_"]*n
        for p,c in enumerate(s): out[inv[p]]=c
        return "".join(out)
    P=stim.PauliString
    S=[P(relabel(s)) for s in stabs]; XL=P(relabel(xl)); ZL=P(relabel(zl))
    dq=n-k
    sim=stim.TableauSimulator(); sim.do(circ)
    ok0=all(sim.peek_observable_expectation(s)==1 for s in S)
    zexp=sim.peek_observable_expectation(ZL)
    sim=stim.TableauSimulator(); sim.h(dq); sim.do(circ)
    okp=all(sim.peek_observable_expectation(s)==1 for s in S)
    xexp=sim.peek_observable_expectation(XL)
    print(f"{name}: r={r} perm={perm} stabilisers ok: {ok0 and okp}; <Z_L> on |0_L> = {zexp}, <
    X_L> on |+_L> = {xexp}; ops={len(circ)}")
    print(circ)
    return circ,perm

if __name__=="__main__":
    verify("[[5,1,3]]", ["XZZX_","_XZZX","X_XZZ","ZX_XZ"], "XXXXX", "ZZZZZ")
    verify("Steane", ["___XXXX","_XX__XX","X_X_X_X","___ZZZZ","_ZZ__ZZ","Z_Z_Z_Z"], "XXXXXXX","
    ZZZZZZZ")
    verify("Shor", ["ZZ_______","_ZZ______","___ZZ____","____ZZ___","______ZZ_","_______ZZ","
    XXXXXX___","___XXXXXX"],
           "ZZZZZZZZZ","XXXXXXXXX")
    # [[15,1,3]] punctured Reed-Muller: X checks = the four weight-8 coordinate hyperplanes of
    the
    # 4-cube minus the origin, Z checks = those four plus the six weight-4 pairwise products.
    import itertools
    xs=["".join("X" if ((i>>b)&1) else "_" for i in range(1,16)) for b in range(4)]
    zs=["".join("Z" if ((i>>b)&1) else "_" for i in range(1,16)) for b in range(4)]
    zs+=["".join("Z" if (((i>>b1)&1) and ((i>>b2)&1)) else "_" for i in range(1,16)) for b1,b2 in
     itertools.combinations(range(4),2)]
    verify("[[15,1,3]] RM", xs+zs, "X"*15, "Z"*15)
```

scripts/surface_threshold.py

```
"""Rotated surface-code memory (Z basis) under circuit-level depolarising noise,
decoded with minimum-weight perfect matching (PyMatching / sparse blossom).

For each distance d and physical error rate p the circuit runs d rounds; the
logical error per shot eps is converted to an error per round via
    eps_round = (1 - (1 - 2 eps)^(1/d)) / 2 .
Output: data/surface_threshold.dat with columns d p shots errors eps eps_round
"""
import sys, time, numpy as np, stim, pymatching

out = sys.argv[1] if len(sys.argv) > 1 else "surface_threshold.dat"
distances = [3, 5, 7, 9]
ps = [0.0005, 0.001, 0.0015, 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.008, 0.009, 0.010,
    0.011, 0.012, 0.015]
MAX_SHOTS = 12_000_000
MIN_ERRORS = 1000
BATCH = 50_000
rng = np.random.default_rng(12345)
rows = []
t0 = time.time()
for d in distances:
    for p in ps:
        circ = stim.Circuit.generated(
            "surface_code:rotated_memory_z", distance=d, rounds=d,
            after_clifford_depolarization=p, before_round_data_depolarization=p,
            before_measure_flip_probability=p, after_reset_flip_probability=p)
        dem = circ.detector_error_model(decompose_errors=True)
        matcher = pymatching.Matching.from_detector_error_model(dem)
        sampler = circ.compile_detector_sampler(seed=int(rng.integers(1 << 30)))
        shots = errors = 0
        while shots < MAX_SHOTS and errors < MIN_ERRORS:
            dets, obs = sampler.sample(shots=BATCH, separate_observables=True)
```

```
            pred = matcher.decode_batch(dets)
            errors += int(np.sum(np.any(pred != obs, axis=1)))
            shots += BATCH
        eps = errors / shots
        eps_round = (1 - (1 - 2 * eps) ** (1 / d)) / 2 if eps < 0.5 else 0.5
        rows.append((d, p, shots, errors, eps, eps_round))
        print(f"d={d} p={p:.4f} shots={shots} errors={errors} eps={eps:.3e} "
              f"eps_round={eps_round:.3e}  [{time.time()-t0:.0f}s]", flush=True)
with open(out, "w") as f:
    f.write("d p shots errors eps eps_round\n")
    for r in rows:
        f.write(f"{r[0]} {r[1]} {r[2]} {r[3]} {r[4]:.6e} {r[5]:.6e}\n")
print("wrote", out)
```

scripts/bb_codes.py

```
"""Bivariate bicycle codes: build H_X=[A|B], H_Z=[B^T|A^T] from the polynomials and verify k = n
    - rank H_X - rank H_Z."""
import numpy as np
def shift(l):  # cyclic shift matrix S_l
    return np.roll(np.eye(l,dtype=int),1,axis=1)
def xy(l,m):
    x=np.kron(shift(l),np.eye(m,dtype=int)); y=np.kron(np.eye(l,dtype=int),shift(m)); return x,y
def mpow(M,e):
    R=np.eye(M.shape[0],dtype=int)
    for _ in range(e): R=(R@M)%2
    return R
def rank2(M):
    M=M.copy()%2; r=0; rows,cols=M.shape
    for c in range(cols):
        piv=[i for i in range(r,rows) if M[i,c]]
        if not piv: continue
        M[[r,piv[0]]]=M[[piv[0],r]]
        for i in range(rows):
            if i!=r and M[i,c]: M[i]^=M[r]
        r+=1
        if r==rows: break
    return r
codes={ # name: (l, m, A terms as (xe,ye), B terms)
 "[[72,12,6]]":  (6,6,  [(3,0),(0,1),(0,2)], [(0,3),(1,0),(2,0)]),
 "[[90,8,10]]":  (15,3, [(9,0),(0,1),(0,2)], [(0,0),(2,0),(7,0)]),
 "[[108,8,10]]": (9,6,  [(3,0),(0,1),(0,2)], [(0,3),(1,0),(2,0)]),
 "[[144,12,12]]":(12,6, [(3,0),(0,1),(0,2)], [(0,3),(1,0),(2,0)]),
 "[[288,12,18]]":(12,12,[(3,0),(0,2),(0,7)], [(0,3),(1,0),(2,0)]),
}
for name,(l,m,At,Bt) in codes.items():
    x,y=xy(l,m); n=2*l*m
    A=sum(mpow(x,a)@mpow(y,b) for a,b in At)%2; B=sum(mpow(x,a)@mpow(y,b) for a,b in Bt)%2
    HX=np.concatenate([A,B],axis=1); HZ=np.concatenate([B.T,A.T],axis=1)
    assert not ((HX@HZ.T)%2).any()
    k=n-rank2(HX)-rank2(HZ)
    wX=HX.sum(axis=1); wZ=HZ.sum(axis=1); colw=HX.sum(axis=0)+HZ.sum(axis=0)
    print(f"{name}: n={n} k={k} check weight X={set(wX)} Z={set(wZ)} data-qubit degree={set(colw)
    }")
```

scripts/bp_osd_gross.py

```
"""Minimal BP+OSD decoder (min-sum belief propagation, then order-0 ordered-statistics
post-processing) for the [[144,12,12]] gross code under code-capacity bit-flip noise.
Z-type checks H_Z=[B^T|A^T] detect X errors; logical failure = residual error not in
the stabiliser group (i.e. anticommutes with some logical Z).  Output: data/bb_bposd.dat
"""
import sys, numpy as np
sys.path.insert(0, __file__.rsplit('/',1)[0])
from bb_codes import xy, mpow, rank2

def gf2_solve_rows(H, s, order):
    """OSD-0: Gaussian elimination on columns in the given reliability order; return e with H e =
     s."""
    m, n = H.shape
```

```
    M = np.concatenate([H[:, order] % 2, s.reshape(-1, 1)], axis=1)
    pivcols = []
    r = 0
    for c in range(n):
        piv = [i for i in range(r, m) if M[i, c]]
        if not piv: continue
        M[[r, piv[0]]] = M[[piv[0], r]]
        for i in range(m):
            if i != r and M[i, c]: M[i] ^= M[r]
        pivcols.append(c); r += 1
        if r == m: break
    e = np.zeros(n, dtype=int)
    for i, c in enumerate(pivcols):
        e[c] = M[i, n]
    out = np.zeros(n, dtype=int); out[order] = e
    return out

def bp_minsum(H, s, p, iters=60, alpha=0.8):
    m, n = H.shape
    llr0 = np.log((1 - p) / p)
    rows = [np.flatnonzero(H[i]) for i in range(m)]
    cols = [np.flatnonzero(H[:, j]) for j in range(n)]
    msg_v2c = {(i, j): llr0 for i in range(m) for j in rows[i]}
    msg_c2v = {(i, j): 0.0 for i in range(m) for j in rows[i]}
    for it in range(iters):
        for i in range(m):
            js = rows[i]
            vals = np.array([msg_v2c[(i, j)] for j in js])
            sg = np.prod(np.sign(vals)) * (-1 if s[i] else 1)
            ab = np.abs(vals)
            for idx, j in enumerate(js):
                others = np.delete(ab, idx)
                msg_c2v[(i, j)] = alpha * sg * np.sign(vals[idx]) * others.min()
        post = np.full(n, llr0)
        for j in range(n):
            post[j] += sum(msg_c2v[(i, j)] for i in cols[j])
        for i in range(m):
            for j in rows[i]:
                msg_v2c[(i, j)] = post[j] - msg_c2v[(i, j)]
        e = (post < 0).astype(int)
        if np.array_equal((H @ e) % 2, s):
            return e, post, True
    return e, post, False

def main(out):
    l, m = 12, 6
    x, y = xy(l, m)
    A = (mpow(x,3) + mpow(y,1) + mpow(y,2)) % 2
    B = (mpow(y,3) + mpow(x,1) + mpow(x,2)) % 2
    HZ = np.concatenate([B.T, A.T], axis=1) % 2       # detects X errors
    HX = np.concatenate([A, B], axis=1) % 2
    n = 2 * l * m
    # logical Z operators = kernel(HX) mod rowspace(HZ); test failure via commutation with a
    basis of ker(HX)
    # compute ker(HX) basis
    def nullspace(Mx):
        Mx = Mx.copy() % 2; rows, cols = Mx.shape; piv = []; r = 0
        for c in range(cols):
            pv = [i for i in range(r, rows) if Mx[i, c]]
            if not pv: continue
            Mx[[r, pv[0]]] = Mx[[pv[0], r]]
            for i in range(rows):
                if i != r and Mx[i, c]: Mx[i] ^= Mx[r]
            piv.append(c); r += 1
            if r == rows: break
        free = [c for c in range(cols) if c not in piv]
        basis = []
        for f in free:
            v = np.zeros(cols, dtype=int); v[f] = 1
            for i, c in enumerate(piv):
                if Mx[i, f]: v[c] = 1
            basis.append(v)
        return np.array(basis)
    K = nullspace(HX)                                  # all Z-type operators commuting with X
    checks (n - rank HX of them)
```

```
    rng = np.random.default_rng(7)
    rows_out = []
    for p in [0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.10]:
        trials = 2000 if p <= 0.04 else 1000
        fails_bp = fails_osd = 0
        for t in range(trials):
            e = (rng.random(n) < p).astype(int)
            s = (HZ @ e) % 2
            ehat, post, conv = bp_minsum(HZ, s, p)
            if not conv:
                order = np.argsort(-np.abs(post) * np.where(post < 0, 1.0, 1.0))  # most reliable
     first
                # OSD: columns sorted by decreasing probability of being in error (post small/
    negative first)
                order = np.argsort(post)
                ehat = gf2_solve_rows(HZ, s, order)
            res = (e + ehat) % 2
            # the residual X error is harmless iff it lies in the X-stabiliser group = rowspace(
    HX);
            # (it is syndrome-free by construction) -> logical failure iff rank([HX; res]) > rank
    (HX)
            if rank2(np.vstack([HX, res])) > rank2(HX):
                fails_osd += 1
            if not conv:
                fails_bp += 1
        rows_out.append((p, trials, fails_bp, fails_osd))
        print(f"p={p} trials={trials} BP-nonconverged={fails_bp} logical failures (BP+OSD0)={
    fails_osd} -> {fails_osd/trials:.4f}", flush=True)
    with open(out, "w") as f:
        f.write("p trials bp_fail osd_fail pL\n")
        for r in rows_out: f.write(f"{r[0]} {r[1]} {r[2]} {r[3]} {r[3]/r[1]:.5f}\n")

if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "bb_bposd.dat")
```

scripts/physical_estimates.py

```
"""Physical-resource estimates used in the document.

Model S ("superconducting surface code, Gidney-2025 conventions", identical to the model in
Julian's shor_qiskit/physical.py so that the two documents agree):
  p = 1e-3, 1 us cycle, 10 us reaction time, per-patch per-round target 1e-15 -> d = 25,
  hot logical qubit = 2(d+1)^2 = 1352 physical, compute block 7x18 patches = 170,352 physical
  (six CCZ factories + three columns of workspace), one CCZ per 25 us -> time = Toffolis * 25 us.
Also prints the Litinski-style per-gate accounting.
"""
import math, json

class Surface:
    def __init__(s, p=1e-3, cycle_us=1.0, reaction_us=10.0, target=1e-15, a=0.1, p_th=0.0122,
    factories=6):
        s.p, s.cycle_us, s.reaction_us, s.target, s.a, s.p_th, s.factories = p, cycle_us,
    reaction_us, target, a, p_th, factories
    def pl(s, d): return s.a * (s.p / s.p_th) ** ((d + 1) / 2)
    def distance(s):
        d = 3
        while s.pl(d) > s.target: d += 2
        return d
    def hot(s, d=None): d = d or s.distance(); return 2 * (d + 1) ** 2
    def compute(s, d=None): return 7 * 18 * s.hot(d)
    def ccz_us(s): return 150.0 / s.factories * s.cycle_us

def estimate(label, logical, toffoli, code=None, shots=1.0, depth=None):
    code = code or Surface()
    d = code.distance(); hot = code.hot(d)
    phys = logical * hot + code.compute(d)
    t_ccz = toffoli * code.ccz_us() * 1e-6
    t_react = (depth or 0) * code.reaction_us * 1e-6
    secs = max(t_ccz, t_react)
    rounds = secs / (code.cycle_us * 1e-6)
    logical_all = logical + 7 * 18
    p_ok = (1 - code.target) ** (logical_all * rounds)
```

```
    return dict(label=label, logical=logical, toffoli=toffoli, d=d, hot=hot, physical=phys,
                hours=secs / 3600, p_ok=p_ok, days=secs * shots / 86400 / p_ok)

LOOKUPS = 3 * 2 ** 16   # per windowed addition, IonQ/ECDSA.fail convention
NADD = 28
rows = [
 # label, logical qubits (incl. 16 window qubits where the source excludes them), Toffolis per
    run
 ("ECDSA.fail 8e9c9a2 (26 Jul incumbent) + 16 window qubits, 28 adds + lookups", 1151 + 16, NADD
    * (1_299_453 + LOOKUPS)),
 ("ECDSA.fail windowed 8e9c9a2 (Note F; lookups included)", 1162, NADD * 1_683_828),
 ("ECDSA.fail a02c716 (7 Oct frontier, Leapfrog) + 16, 28 adds + lookups", 1244 + 16, NADD * (797
    _905 + LOOKUPS)),
 ("ECDSA.fail b6f2b0a low-Q (825) + 16, 28 adds + lookups", 825 + 16, NADD * (489_161_900 +
    LOOKUPS)),
 ("ECDSA.fail 10177cd low-Q (792, 7 Oct) + 16, 28 adds + lookups", 792 + 16, NADD * (656_602_578
    + LOOKUPS)),
 ("Google/Babbush et al. low-qubit variant (whole algorithm)", 1200, 90_000_000),
 ("Google/Babbush et al. low-gate variant (whole algorithm)", 1450, 70_000_000),
 ("IonQ/Haener et al. (whole algorithm)", 1457, 39_000_000),
 ("Schrottenloher space-optimised (whole algorithm)", 1208, 72_490_961),
 ("Schrottenloher gate-optimised (whole algorithm)", 1462, 57_698_094),
 ("Julian: IonQ cells, signed windows, SELECT-SWAP (whole algorithm, executed)", 1457, 36_758_053
    ),
 ("Julian: dialog on the space cells (whole algorithm)", 1246, 79_056_082),
 ("Julian: three field registers, Luo EEA (whole algorithm)", 851, 2_385_774_798),
 ("RSA-2048, Gidney 2025 (published inputs; 9.2 shots of 12.07 h)", 1399 + 0, 6_500_000_000),
]
code = Surface()
print(f"Model S: d={code.distance()} hot={code.hot()} compute={code.compute()} CCZ period={code.
    ccz_us()} us")
out = []
for label, q, t in rows:
    r = estimate(label, q, t, code)
    out.append(r)
    print(f"{label[:60]:60s} Q={q:5d} T={t:.3e} phys={r['physical']:,.0f} hours={r['hours']:.3f}
    days={r['days']:.3f} p_ok={r['p_ok']:.3f}")
json.dump(out, open("data/physical_modelS.json", "w"), indent=1)

# Litinski-style per-gate accounting at p=1e-3, d=25
d = 25
print("\nPer-gate accounting (surface code, d=25, p=1e-3):")
print(f" logical qubit patch: d^2 data + (d^2-1) ancilla = {2*d*d-1} physical; with routing share
     2(d+1)^2 = {2*(d+1)**2}")
print(f" lattice-surgery CNOT/joint measurement: + one ancilla tile {d*d} data (~{2*d*d} physical
    ) for d = {d} rounds = {d*code.cycle_us} us")
print(f" T gate (fast layout): 1 tick = d rounds = {d} us, consumes one |T> from a factory")
fac = {"(15-to-1)_{17,7,7}": (4620, 42.6, 4.5e-8), "(15-to-1)^6_{13,5,5} x (20-to-4)_{23,11,13}":
     (43300, 130, 1.4e-10),
       "(15-to-1)^6_{13,7,7} x (8-to-CCZ)_{25,15,15}": (47000, 60.0, 5.2e-11)}
for k, (q, c, e) in fac.items():
    print(f"  factory {k}: {q:,} qubits, {c} cycles per state, p_out={e:.1e}")
print(" cultivation (Gidney-Shutty-Jones 2024): ~2e-9 per T at p=1e-3, ~5e4 qubit-rounds per T (
    Babbush et al.)")
T_total = 4 * NADD * (1_299_453 + LOOKUPS)
print(f" ECDSA.fail 8e9c9a2 full attack: {NADD*(1_299_453+LOOKUPS):,} Toffoli = {T_total:,} T (4T
     Toffoli) -> at 1e-10/T budget error {T_total*1.4e-10:.3f}")
```

scripts/syndrome_tables.py

```
"""Syndrome (lookup-table) decoders for the small codes, emitted as LaTeX tables.

For every single-qubit Pauli error E on every qubit, the syndrome bit s_i is 1 iff E
anticommutes with generator S_i.  A lookup-table decoder inverts this map.  Also
splits data/codes.dat into one file per family for the rate-distance plot.
Outputs: data/syn_steane.tex, data/syn_513.tex, data/syn_shor.tex, data/codes_*.dat
"""
import stim, itertools, collections

def syndrome(err, gens):
    return "".join("1" if not err.commutes(g) else "0" for g in gens)
```

```
def P(s): return stim.PauliString(s)

def single_errors(n):
    for q in range(n):
        for p in "XYZ":
            s = ["_"] * n; s[q] = p
            yield f"{p}_{{{q+1}}}", P("".join(s))

# ---------------------------------------------------------------- Steane: Hamming rows
ham = [[1,3,5,7],[2,3,6,7],[4,5,6,7]]           # 1-indexed supports of the three checks
def css(sup, t, n=7):
    return P("".join(t if (i+1) in sup else "_" for i in range(n)))
steane_z = [css(s,"Z") for s in ham]            # detect X errors
steane_x = [css(s,"X") for s in ham]            # detect Z errors
rows = []
for q in range(1,8):
    ex = ["_"]*7; ex[q-1] = "X"; ez = ["_"]*7; ez[q-1] = "Z"
    sz = syndrome(P("".join(ex)), steane_z); sx = syndrome(P("".join(ez)), steane_x)
    assert sz == sx == format(q, "03b")[::-1], (q, sz, sx)
    rows.append((q, sz[::-1]))
with open("data/syn_steane.tex", "w") as f:
    f.write("\\begin{tabular}{@{}c" + "c"*7 + "@{}}\\toprule\n")
    f.write("qubit $j$ & " + " & ".join(str(q) for q,_ in rows) + "\\\\\n")
    f.write("$(s_4,s_2,s_1)$ of $X_j$ (from $Z$-checks) = of $Z_j$ (from $X$-checks) & " + " & ".
    join(s for _,s in rows) + "\\\\\n")
    f.write("\\bottomrule\\end{tabular}\n")

# ---------------------------------------------------------------- [[5,1,3]]
gens513 = [P("XZZX_"), P("_XZZX"), P("X_XZZ"), P("ZX_XZ")]
table = {}
for name, e in single_errors(5):
    s = syndrome(e, gens513)
    assert s not in table, "collision"
    table[s] = name
assert len(table) == 15 and "0000" not in table
with open("data/syn_513.tex", "w") as f:
    keys = sorted(table)
    f.write("\\begin{tabular}{@{}l" + "c"*15 + "@{}}\\toprule\n")
    f.write("$s_1s_2s_3s_4$ & " + " & ".join(keys) + "\\\\\n")
    f.write("error & " + " & ".join("$"+table[k]+"$" for k in keys) + "\\\\\n")
    f.write("\\bottomrule\\end{tabular}\n")

# ---------------------------------------------------------------- Shor
shor_z = [P("ZZ_______"),P("_ZZ______"),P("___ZZ____"),P("____ZZ___"),P("______ZZ_"),P("_______ZZ
    ")]
shor_x = [P("XXXXXX___"),P("___XXXXXX")]
with open("data/syn_shor.tex", "w") as f:
    f.write("\\begin{tabular}{@{}lccccccccc@{}}\\toprule\n")
    f.write("qubit $j$ & " + " & ".join(str(q) for q in range(1,10)) + "\\\\\n\\midrule\n")
    xs = []
    zs = []
    for q in range(9):
        e = ["_"]*9; e[q]="X"; xs.append(syndrome(P("".join(e)), shor_z))
        e = ["_"]*9; e[q]="Z"; zs.append(syndrome(P("".join(e)), shor_x))
    f.write("$X_j$ ($Z$-checks, by block) & " + " & ".join(s[:2]+"\\,"+s[2:4]+"\\,"+s[4:] for s
    in xs) + "\\\\\n")
    f.write("$Z_j$ ($X_{1..6},X_{4..9}$) & " + " & ".join(s for s in zs) + "\\\\\n")
    f.write("\\bottomrule\\end{tabular}\n")
    # degeneracy: Z_1,Z_2,Z_3 share a syndrome and differ by a stabiliser
    assert zs[0]==zs[1]==zs[2] and zs[3]==zs[4]==zs[5] and zs[6]==zs[7]==zs[8]

# ---------------------------------------------------------------- split codes.dat by family
fam = collections.defaultdict(list)
hdr = None
for line in open("data/codes.dat"):
    if line.startswith("n "): hdr = line; continue
    parts = line.split(None, 6)
    fam[parts[5]].append(line)
for k, lines in fam.items():
    with open(f"data/codes_{k}.dat", "w") as f:
        f.write(hdr); f.writelines(lines)
print("wrote", sorted(fam), {k: len(v) for k, v in fam.items()})
```

scripts/verify_codes.py

```
"""Verify the encoding circuits and syndrome tables quoted in the document with Stim.

Checks, for each code:
  * the encoder maps |0>|0..0> to a +1 eigenstate of every stabiliser and of Z_L,
    and |+>|0..0> to a +1 eigenstate of X_L  (TableauSimulator.peek_observable_expectation)
  * the Heisenberg picture: X on the data qubit -> X_L (mod stabilisers), Z -> Z_L
  * syndrome tables for all single-qubit Paulis
"""
import itertools, stim

def paulistr(s):
    return stim.PauliString(s)

def check_encoder(name, n, data_q, enc, stabs, xl, zl):
    print(f"=== {name} ===")
    # |0_L>
    sim = stim.TableauSimulator()
    sim.do(enc)
    for s in stabs + [zl]:
        e = sim.peek_observable_expectation(paulistr(s))
        assert e == 1, (name, s, e)
    # |+_L>
    sim = stim.TableauSimulator()
    sim.h(data_q)
    sim.do(enc)
    for s in stabs + [xl]:
        e = sim.peek_observable_expectation(paulistr(s))
        assert e == 1, (name, s, e)
    # Heisenberg check: conjugate X_data and Z_data through the encoder
    tab = enc.to_tableau()
    xs = tab.x_output(data_q); zs = tab.z_output(data_q)
    # membership of xs*xl^-1 in the stabiliser group: check it commutes with all logicals and
    stabilisers
    def in_stab_group(p):
        # p is in the stabiliser group iff it commutes with everything in the normaliser's
        # generators AND is +1 on the code space; test on |0_L>,|+_L> expectation
        for st in (stabs + [xl, zl]):
            if not p.commutes(paulistr(st)):
                return False
        for prep in (lambda s: None, lambda s: s.h(data_q)):
            sim = stim.TableauSimulator(); prep(sim); sim.do(enc)
            if sim.peek_observable_expectation(p) != 1:
                return False
        return True
    assert in_stab_group(xs * paulistr(xl)), ("X_L mismatch", xs)
    assert in_stab_group(zs * paulistr(zl)), ("Z_L mismatch", zs)
    print(f"encoder OK: X_data -> {xs}  (= X_L mod S),  Z_data -> {zs}  (= Z_L mod S)")
    print("gates:", enc.num_operations if hasattr(enc,'num_operations') else len(enc))
    return tab

def syndrome_table(name, n, stabs):
    table = {}
    for q in range(n):
        for P in "XYZ":
            err = ["I"]*n; err[q] = P
            e = paulistr("".join(err))
            syn = tuple(0 if e.commutes(paulistr(s)) else 1 for s in stabs)
            table.setdefault(syn, []).append(f"{P}{q}")
    collisions = {k:v for k,v in table.items() if len(v) > 1}
    print(f"{name}: {len(table)} distinct syndromes for {3*n} single-qubit errors; collisions: {
    collisions}")
    return table

# ---------------- Steane [[7,1,3]] ----------------
steane_stabs = ["___XXXX", "_XX__XX", "X_X_X_X", "___ZZZZ", "_ZZ__ZZ", "Z_Z_Z_Z"]
steane_enc = stim.Circuit("""
CX 0 1 0 2
H 3 4 5
CX 3 0 3 1 3 6
CX 4 0 4 2 4 6
CX 5 1 5 2 5 6
""")
```

```
check_encoder("Steane [[7,1,3]]", 7, 0, steane_enc, steane_stabs, "XXX____", "ZZZ____")
st = syndrome_table("Steane", 7, steane_stabs)
print("Steane X-error syndromes (Z-checks s4 s5 s6):")
for q in range(7):
    err=["_"]*7; err[q]="X"; e=paulistr("".join(err))
    print(" X%d ->"%q, "".join("1" if not e.commutes(paulistr(s)) else "0" for s in steane_stabs
    [3:]))

# ---------------- Shor [[9,1,3]] ----------------
shor_stabs = ["ZZ_______","_ZZ______","___ZZ____","____ZZ___","______ZZ_","_______ZZ",
              "XXXXXX___","___XXXXXX"]
shor_enc = stim.Circuit("""
CX 0 3 0 6
H 0 3 6
CX 0 1 0 2 3 4 3 5 6 7 6 8
""")
check_encoder("Shor [[9,1,3]]", 9, 0, shor_enc, shor_stabs, "ZZZZZZZZZ", "XXXXXXXXX")
syndrome_table("Shor", 9, shor_stabs)

# ---------------- [[5,1,3]] ----------------
five_stabs = ["XZZX_", "_XZZX", "X_XZZ", "ZX_XZ"]
# derive an encoder: tableau whose Z-images are S1..S4 and Z_L ; data qubit = 4
tab = stim.Tableau.from_stabilizers([paulistr(s) for s in five_stabs] + [paulistr("ZZZZZ")],
                                    allow_redundant=False, allow_underconstrained=False)
five_enc = tab.to_circuit(method="elimination")
print("[[5,1,3]] derived encoder:\n", five_enc)
check_encoder("[[5,1,3]]", 5, 4, five_enc, five_stabs, "XXXXX", "ZZZZZ")
tbl = syndrome_table("[[5,1,3]]", 5, five_stabs)
print("[[5,1,3]] syndrome table (s1 s2 s3 s4):")
for syn, errs in sorted(tbl.items()):
    print(" ", "".join(map(str,syn)), errs)

# ---------------- [[15,1,3]] Reed-Muller (punctured RM(1,4)) ----------------
# X stabilisers: rows of the [15,4] simplex-code-like H (weight 8), Z stabilisers: all weight-4
    ...
# We only verify transversal-T later in the text; here verify the CSS structure counts.
import numpy as np
def rm_x_stabs():
    rows=[]
    for b in range(4):
        rows.append("".join("X" if ((i>>b)&1) else "_" for i in range(1,16)))
    return rows
def rm_z_stabs():
    # Z stabilisers: products of pairs of the X-type rows (weight 4 patterns) -> generators of RM
    (2,4)* dual
    rows=[]
    for b1,b2 in itertools.combinations(range(4),2):
        rows.append("".join("Z" if (((i>>b1)&1) and ((i>>b2)&1)) else "_" for i in range(1,16)))
    for b in range(4):
        rows.append("".join("Z" if ((i>>b)&1) else "_" for i in range(1,16)))
    return rows
xs=rm_x_stabs(); zs=rm_z_stabs()
ok = all(paulistr(a).commutes(paulistr(b)) for a in xs for b in zs)
print("[[15,1,3]] RM: X stabs", len(xs), "Z stabs", len(zs), "all commute:", ok)
```

References

[1] Rajeev Acharya et al. Quantum error correction below the surface code threshold. *Nature*, 638:920–926, 2025. arXiv:2408.13687 (Google Quantum AI).

[2] Dorit Aharonov and Michael Ben-Or. Fault-tolerant quantum computation with constant error rate. *SIAM Journal on Computing*, 38(4):1207–1282, 2008. arXiv:quant-ph/9906129.

[3] Jonas T. Anderson, Guillaume Duclos-Cianci, and David Poulin. Fault-tolerant conversion between the Steane and Reed–Muller quantum codes. *Physical Review Letters*, 113:080501, 2014. arXiv:1403.2734.

[4] Alexei Ashikhmin and Simon Litsyn. Upper bounds on the size of quantum codes. *IEEE Transactions on Information Theory*, 45(4):1206–1215, 1999. arXiv:quant-ph/9709049.

[5] Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake, and Dan Boneh. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: resource estimates and mitigations. *PRX Quantum*, 7:031001, 2026. arXiv:2603.28846; IACR ePrint 2026/625 (Google Quantum AI).

[6] Dave Bacon. Operator quantum error-correcting subsystems for self-correcting quantum memories. *Physical Review A*, 73:012340, 2006. arXiv:quant-ph/0506023.

[7] Johannes Bausch, Andrew W. Senior, Francisco J. H. Heras, Thomas Edlich, Alex Davies, Michael Newman, Cody Jones, Kevin Satzinger, et al. Learning high-accuracy error decoding for quantum processors, 2024. Nature 635, 834–840; arXiv:2310.05900 (Google DeepMind / Google Quantum AI).

[8] Charles H. Bennett, David P. DiVincenzo, John A. Smolin, and William K. Wootters. Mixed-state entanglement and quantum error correction. *Physical Review A*, 54:3824–3851, 1996. arXiv:quant-ph/9604024.

[9] Dolev Bluvstein, Simon J. Evered, Alexandra A. Geim, et al. Logical quantum processor based on reconfigurable atom arrays. *Nature*, 626:58–65, 2024. arXiv:2312.03982 (Harvard/-QuEra).

[10] Héctor Bombín. Gauge color codes: optimal transversal gates and gauge fixing in topological stabilizer codes. *New Journal of Physics*, 17:083002, 2015. arXiv:1311.0879.

[11] Héctor Bombín and Miguel Angel Martin-Delgado. Topological quantum distillation. *Physical Review Letters*, 97:180501, 2006. arXiv:quant-ph/0605138.

[12] Héctor Bombín and Miguel Angel Martin-Delgado. Exact topological quantum order in $d = 3$ and beyond: Branyons and brane-net condensates. *Physical Review B*, 75:075103, 2007. arXiv:cond-mat/0607736.

[13] Sergey Bravyi and Jeongwan Haah. Magic-state distillation with low overhead. *Physical Review A*, 86:052329, 2012. arXiv:1209.2426.

[14] Sergey Bravyi and Alexei Kitaev. Universal quantum computation with ideal Clifford gates and noisy ancillas. *Physical Review A*, 71:022316, 2005. arXiv:quant-ph/0403025.

[15] Sergey Bravyi and Robert König. Classification of topologically protected gates for local stabilizer codes. *Physical Review Letters*, 110:170503, 2013. arXiv:1206.1609.

[16] Sergey Bravyi and Barbara Terhal. A no-go theorem for a two-dimensional self-correcting quantum memory based on stabilizer codes. *New Journal of Physics*, 11:043029, 2009. arXiv:0810.1983.

[17] Sergey Bravyi, David Poulin, and Barbara Terhal. Tradeoffs for reliable quantum information storage in 2D systems. *Physical Review Letters*, 104:050503, 2010. arXiv:0909.5200.

[18] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627:778–782, 2024. arXiv:2308.07915 (IBM Quantum).

[19] Sergey B. Bravyi and A. Yu. Kitaev. Quantum codes on a lattice with boundary, 1998. arXiv:quant-ph/9811052.

[20] Nikolas P. Breuckmann and Jens Niklas Eberhardt. Quantum low-density parity-check codes, 2021. PRX Quantum 2, 040101; arXiv:2103.06309.

[21] Madelyn Cain, Qian Xu, Robbie King, Lewis R. B. Picard, Harry Levine, Manuel Endres, John Preskill, Hsin-Yuan Huang, and Dolev Bluvstein. Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits, 2026. arXiv:2603.28627.

[22] A. R. Calderbank and Peter W. Shor. Good quantum error-correcting codes exist. *Physical Review A*, 54:1098–1105, 1996. arXiv:quant-ph/9512032.

[23] A. Robert Calderbank, Eric M. Rains, Peter W. Shor, and Neil J. A. Sloane. Quantum error correction via codes over GF(4). *IEEE Transactions on Information Theory*, 44(4):1369–1387, 1998. arXiv:quant-ph/9608006.

[24] Rui Chao and Ben W. Reichardt. Quantum error correction with only two extra qubits. *Physical Review Letters*, 121:050502, 2018. arXiv:1705.02329.

[25] Lawrence Z. Cohen, Isaac H. Kim, Stephen D. Bartlett, and Benjamin J. Brown. Low-overhead fault-tolerant quantum computing using long-range connectivity. *Science Advances*, 8:eabn1717, 2022. arXiv:2110.10794.

[26] Andrew Cross, Zhiyang He, Patrick Rall, and Theodore Yoder. Improved QLDPC surgery: logical measurements and bridging codes, 2024. arXiv:2407.18393 (IBM Quantum).

[27] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit, 2004. arXiv:quant-ph/0410184.

[28] Nicolas Delfosse. Decoding color codes by projection onto surface codes. *Physical Review A*, 89:012317, 2014. arXiv:1308.6207.

[29] Nicolas Delfosse and Naomi H. Nickerson. Almost-linear time decoding algorithm for topological codes. *Quantum*, 5:595, 2021. arXiv:1709.06218.

[30] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43:4452–4505, 2002. arXiv:quant-ph/0110143.

[31] Bryan Eastin and Emanuel Knill. Restrictions on transversal encoded quantum gate sets. *Physical Review Letters*, 102:110502, 2009. arXiv:0811.4262.

[32] Eigen Labs. The secp256k1 point-addition challenge (`ecdsafail-challenge`, commit `c672dda`, 11 oct. 2026). https://github.com/Layr-Labs/ecdsafail-challenge; https://ecdsa.fail, 2026.

[33] Martin Ekerå. Revisiting Shor's quantum algorithm for computing general discrete logarithms, 2019. arXiv:1905.09084.

[34] Artur Ekert and Chiara Macchiavello. Quantum error correction for communication. *Physical Review Letters*, 77:2585–2588, 1996. arXiv:quant-ph/9602022.

[35] Austin G. Fowler. Minimum weight perfect matching of fault-tolerant topological quantum error correction in average $o(1)$ parallel time, 2013. Quantum Information and Computation 15, 145–158 (2015); arXiv:1307.1740.

[36] Austin G. Fowler and Craig Gidney. Low overhead quantum computation using lattice surgery, 2018. arXiv:1808.06709.

[37] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: towards practical large-scale quantum computation. *Physical Review A*, 86:032324, 2012. arXiv:1208.0928.

[38] Craig Gidney. Halving the cost of quantum addition. *Quantum*, 2:74, 2018. arXiv:1709.06648.

[39] Craig Gidney. Stim: a fast stabilizer circuit simulator, 2021. Quantum 5, 497; arXiv:2103.02202.

[40] Craig Gidney. How to factor 2048 bit RSA integers with less than a million noisy qubits, 2025. arXiv:2505.15917 (Google Quantum AI).

[41] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021. arXiv:1905.09749.

[42] Craig Gidney and Austin G. Fowler. Efficient magic state factories with a catalyzed $|ccz\rangle \rightarrow 2|t\rangle$ transformation, 2019. Quantum 3, 135; arXiv:1812.01238.

[43] Craig Gidney and Cody Jones. New circuits and an open source decoder for the color code, 2023. arXiv:2312.08813 (Google Quantum AI).

[44] Craig Gidney, Michael Newman, Peter Brooks, and Cody Jones. Yoked surface codes, 2023. arXiv:2312.04522 (Google Quantum AI).

[45] Craig Gidney, Noah Shutty, and Cody Jones. Magic state cultivation: growing T states as cheap as CNOT gates, 2024. arXiv:2409.17595 (Google Quantum AI).

[46] Daniel Gottesman. *Stabilizer codes and quantum error correction*. PhD thesis, California Institute of Technology, 1997. arXiv:quant-ph/9705052.

[47] Daniel Gottesman. *Surviving as a Quantum Computer in a Classical World*. 2024. textbook draft, available from the author's web page.

[48] Daniel Gottesman, Alexei Kitaev, and John Preskill. Encoding a qubit in an oscillator. *Physical Review A*, 64:012310, 2001. arXiv:quant-ph/0008040.

[49] Élie Gouzien, Diego Ruiz, Francois-Marie Le Régent, Jérémie Guillaud, and Nicolas Sangouard. Performance analysis of a repetition cat code architecture: computing 256-bit elliptic curve logarithm in 9 hours with 126,133 cat qubits, 2023. Physical Review Letters 131, 040602; arXiv:2302.06639 (Alice & Bob).

[50] Robert B. Griffiths and Chi-Sheng Niu. Semiclassical Fourier transform for quantum computation. *Physical Review Letters*, 76:3228–3231, 1996.

[51] Shouzhen Gu, Christopher A. Pattison, and Eugene Tang. An efficient decoder for a linear distance quantum LDPC code, 2022. STOC 2023; arXiv:2206.06557.

[52] Jérémie Guillaud and Mazyar Mirrahimi. Repetition cat qubits for fault-tolerant quantum computation. *Physical Review X*, 9:041053, 2019. arXiv:1904.09474.

[53] Thomas Häner, Samuel Jaques, Michael Naehrig, Martin Roetteler, and Mathias Soeken. Improved quantum circuits for elliptic curve discrete logarithms. In *PQCrypto 2020*, volume 12100 of *LNCS*, pages 425–444, 2020. arXiv:2001.09580.

[54] Thomas Häner, Felix Tripier, Jacob Young, Michael Naehrig, Andrii Maksymov, Safwan Alam, Dmitri Maslov, Matthew Parrott, Yvette de Sereville, Jordan Sullivan, Mark Webster, Nicolas Delfosse, John Gamble, and Martin Roetteler. Computing 256-bit elliptic curve discrete logarithms in 26 days on a fault-tolerant trapped-ion quantum computer with 20,000 qubits, 2026. arXiv:2609.05625; IACR ePrint 2026/1916 (IonQ).

[55] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching, 2023. arXiv:2303.15933.

[56] Clare Horsman, Austin G. Fowler, Simon Devitt, and Rodney Van Meter. Surface code quantum computing by lattice surgery. *New Journal of Physics*, 14:123011, 2012. arXiv:1111.4022.

[57] Ben Huang. EcdsafailWiki: per-commit analysis of every accepted source commit of the ECDSA.Fail competition, with concept pages and leaderboard snapshots (584 commits, 29 may – 7 oct. 2026). https://github.com/benhuang2025/ecdsafail-wiki, 2026.

[58] IBM Quantum. IBM sets the course to build the world's first large-scale, fault-tolerant quantum computer. https://www.ibm.com/quantum/blog/large-scale-ftqc, 2025.

[59] IBM Quantum. Foundations of quantum error correction (IBM quantum learning course). https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction, 2025.

[60] IBM Quantum. IBM quantum development roadmap. https://www.ibm.com/roadmaps/quantum/, 2026.

[61] Tanuj Khattar, Noah Shutty, Craig Gidney, Adam Zalcman, Noureldin Yosri, Dmitri Maslov, Ryan Babbush, and Stephen P. Jordan. Verifiable quantum advantage via optimized DQI circuits, 2025. arXiv:2510.10967.

[62] Hyunji Kim, Kyungbae Jang, Siyi Wang, Vikas Srivastava, Anubhab Baksi, Gyeongju Song, Hwajeong Seo, and Anupam Chattopadhyay. New quantum circuits for ECDLP: breaking prime elliptic curve cryptography, 2026. IACR ePrint 2026/106.

[63] A. Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303:2–30, 2003. arXiv:quant-ph/9707021.

[64] Emanuel Knill. Quantum computing with realistically noisy devices. *Nature*, 434:39–44, 2005. arXiv:quant-ph/0410199.

[65] Emanuel Knill and Raymond Laflamme. Theory of quantum error-correcting codes. *Physical Review A*, 55:900–911, 1997. arXiv:quant-ph/9604034.

[66] Aleksander Kubica and Michael E. Beverland. Universal transversal gates with color codes: a simplified approach. *Physical Review A*, 91:032330, 2015. arXiv:1410.0069.

[67] Nathan Lacroix, Alexandre Bourassa, Francisco J. H. Heras, Lei M. Zhang, Johannes Bausch, et al. Scaling and logic in the color code on a superconducting quantum processor, 2024. arXiv:2412.14256 (Google Quantum AI).

[68] Raymond Laflamme, Cesar Miquel, Juan Pablo Paz, and Wojciech Hubert Zurek. Perfect quantum error correcting code. *Physical Review Letters*, 77:198–201, 1996. arXiv:quant-ph/9602019.

[69] Anthony Leverrier and Gilles Zémor. Quantum Tanner codes, 2022. FOCS 2022; arXiv:2202.13641.

[70] Anthony Leverrier and Gilles Zémor. Decoding quantum Tanner codes, 2022. IEEE Transactions on Information Theory 69, 5100–5115 (2023); arXiv:2208.05537.

[71] Daniel Litinski. A game of surface codes: large-scale quantum computing with lattice surgery. *Quantum*, 3:128, 2019. arXiv:1808.02892.

[72] Daniel Litinski. Magic state distillation: not as costly as you think. *Quantum*, 3:205, 2019. arXiv:1905.06903.

[73] Daniel Litinski. How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates, 2023. arXiv:2306.08585 (PsiQuantum).

[74] Daniel Litinski and Naomi Nickerson. Active volume: an architecture for efficient fault-tolerant quantum computers with limited non-local connections, 2022. arXiv:2211.15465 (PsiQuantum).

[75] Jieyi Long, Theodore Pender, Zhao Huang, Manuel B. Santos, Samrendra Kumar Singh, Bartosz Naskręcki, Bit Wonka, Pierre-Luc Dallaire-Demers, Francesco Giannicola, Ruben M. L. Paschoarelli, Oli Freuler, Jackie Chia-Hsun Lee, Vasily Gnuchev, Gopi Kannappan, John Boyer, Xavier Butler, Akash Balasubramani, Jordan Newman, Bereket Dereje, Alexander Hertlein, Robert Kodra, Lucas Levy, Shaan Patel, JT Rose, Matt Zweil, Okechukwu Wisdom, Tarek El-Eter, Edison Lee, Michael Dong, Alan Li, Anto Joseph, Gajesh Naik, Gautham Anant, Soubhik Deb, and Justin Drake. ECDSA.Fail: open autoresearch for optimizing elliptic-curve point addition in Shor's algorithm, 2026. arXiv:2609.09582v2.

[76] Jieyi Long et al. ECDSA.Fail supplemental materials: Supplemental notes A–F (appendices, circuit details, frontier reconstruction). https://github.com/jieyilong/ecdsafail-supplemental-materials, 2026.

[77] Guang Hao Low, Vadym Kliuchnikov, and Luke Schaeffer. Trading T-gates for dirty qubits in state preparation and unitary synthesis, 2018. arXiv:1812.00954.

[78] Hao Luo, Zhe Yang, Jun Luo, Zeyu Wang, Yue Su, Xiaoming Sun, Lvzhou Li, and Tao Li. Quantum algorithm for elliptic curve discrete logarithms with space-efficient point addition, 2026. arXiv:2607.13816.

[79] Tristan Müller, Thomas Alexander, Michael E. Beverland, Markus Bühler, Blake R. Johnson, Thilo Maurer, and Drew Vandeth. Improved belief propagation is sufficient for real-time decoding of quantum memory, 2025. arXiv:2506.01779 (IBM Quantum).

[80] Adam Paetznick and Ben W. Reichardt. Universal fault-tolerant quantum computation with only transversal gates and error correction. *Physical Review Letters*, 111:090505, 2013. arXiv:1304.3709.

[81] Pavel Panteleev and Gleb Kalachev. Degenerate quantum LDPC codes with good finite length performance. *Quantum*, 5:585, 2021. arXiv:1904.02703.

[82] Pavel Panteleev and Gleb Kalachev. Asymptotically good quantum and locally testable classical LDPC codes, 2021. STOC 2022; arXiv:2111.03654.

[83] John Proos and Christof Zalka. Shor's discrete logarithm quantum algorithm for elliptic curves. *Quantum Information and Computation*, 3(4):317–344, 2003. arXiv:quant-ph/0301141.

[84] Eric M. Rains. Quantum shadow enumerators. *IEEE Transactions on Information Theory*, 45(7): 2361–2366, 1999. arXiv:quant-ph/9611001.

[85] Robert Raussendorf and Jim Harrington. Fault-tolerant quantum computation with high threshold in two dimensions. *Physical Review Letters*, 98:190504, 2007. arXiv:quant-ph/0610082.

[86] Pedro Sales Rodriguez et al. Experimental demonstration of logical magic state distillation, 2024. arXiv:2412.15165 (QuEra/Harvard).

[87] Joschka Roffe, David R. White, Simon Burton, and Earl Campbell. Decoding across the quantum low-density parity-check code landscape. *Physical Review Research*, 2:043423, 2020. arXiv:2005.07016.

[88] Ciaran Ryan-Anderson et al. Realization of real-time fault-tolerant quantum error correction. *Physical Review X*, 11:041058, 2021. arXiv:2107.07505 (Honeywell/Quantinuum).

[89] Ciaran Ryan-Anderson et al. High-fidelity and fault-tolerant teleportation of a logical qubit using transversal gates and lattice surgery on a trapped-ion quantum computer, 2024. arXiv:2404.16728 (Quantinuum).

[90] André Schrottenloher. Optimized point addition circuits for elliptic curve discrete logarithms, 2026. IACR ePrint 2026/1128.

[91] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52:R2493–R2496, 1995.

[92] Peter W. Shor. Fault-tolerant quantum computation. In *Proceedings of the 37th Symposium on Foundations of Computer Science (FOCS)*, pages 56–65, 1996. arXiv:quant-ph/9605011.

[93] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997. arXiv:quant-ph/9508027.

[94] Andrew M. Steane. Error correcting codes in quantum theory. *Physical Review Letters*, 77: 793–797, 1996.

[95] Andrew M. Steane. Active stabilization, quantum computation, and quantum state synthesis. *Physical Review Letters*, 78:2252–2255, 1997. arXiv:quant-ph/9611027.

[96] Jean-Pierre Tillich and Gilles Zémor. Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength. *IEEE Transactions on Information Theory*, 60(2):1193–1202, 2014. arXiv:0903.0566.

[97] Felix Tripier, Woo Chang Chung, Jacob Young, Safwan Alam, Bryce Bjork, Aharon Brodutch, Finn Lasse Buessen, Nolan J. Coble, Thomas Dellaert, Dmitri Maslov, Martin Roetteler, Edwin Tham, Mark Webster, Min Ye, John Gamble, Andrii Maksymov, J. P. Marceaux, and Nicolas Delfosse. Fault-tolerant quantum computing with trapped ions: the walking cat architecture, 2026. arXiv:2604.19481 (IonQ).

[98] Paul Webster, Lucas Berent, Omprakash Chandra, Evan T. Hockings, Nouédyn Baspin, Felix Thomsen, Samuel C. Smith, and Lawrence Z. Cohen. The Pinnacle architecture: reducing the cost of breaking RSA-2048 to 100,000 physical qubits using quantum LDPC codes, 2026. arXiv:2602.11457 (Iceberg Quantum).

[99] Theodore J. Yoder, Eddie Schoute, Patrick Rall, Emily Pritchett, Jay M. Gambetta, Andrew W. Cross, Malcolm Carroll, and Michael E. Beverland. Tour de gross: a modular quantum computer based on bivariate bicycle codes, 2025. arXiv:2506.03094 (IBM Quantum).